

UNIT I

C#.NET .NET Enterprise Architecture , .NET Framework , Base Class Libraries (BCL) , Common Language Run Time (CLR) , Microsoft Intermediate Language (MSIL) , Common Language Specification (CLS) , .NET Products , .NET Services , Download Installation of Visual Studio Community edition 2022.Core 6.0, SQL Server Console Applications

C#.NET

C# is a modern, object-oriented, and type-safe programming language. C# enables developers to build many types of secure and robust applications that run in .NET. C# has its roots in the C family of languages and will be immediately familiar to C, C++, Java, and JavaScript programmers. C# is an object-oriented, component-oriented programming language. C# provides language constructs to directly support these concepts, making C# a natural language in which to create and use software components.

Several C# features help create robust and durable applications.

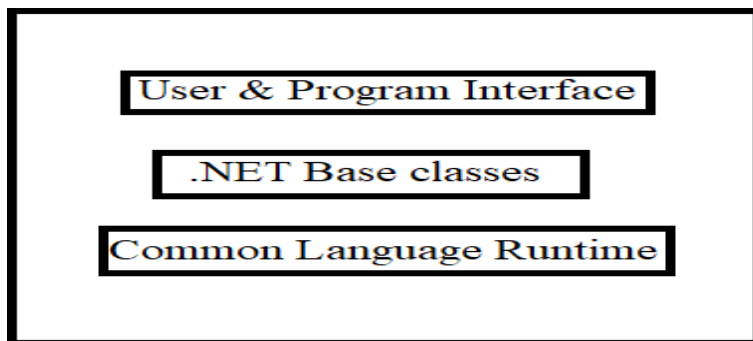
Garbage collection automatically reclaims memory occupied by unreachable unused objects. **Nullable types** guard against variables that don't refer to allocated objects.

Exception handling provides a structured and extensible approach to error detection and recovery.

Lambda expressions support functional programming techniques.

Language Integrated Query (LINQ) syntax creates a common pattern for working with data from any source.

.NET Enterprise Architecture



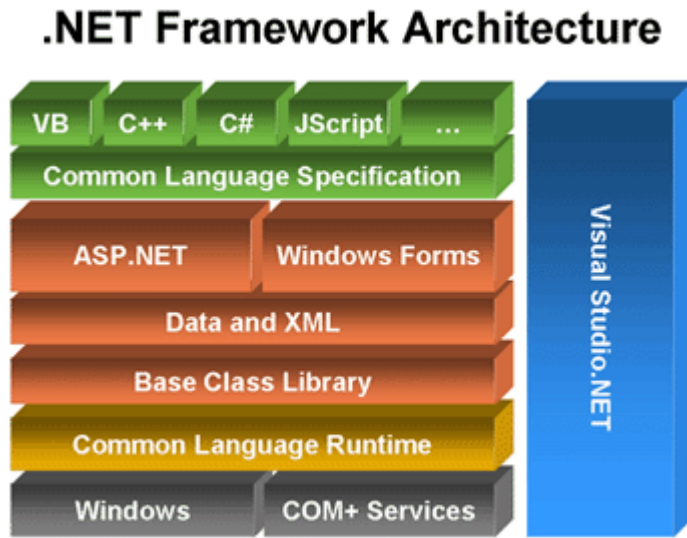
There are 3 important parts in .NET Architecture

1. Common language Runtime
2. .NET Base classes
3. User & Program Interface

.NET Framework:

The **.NET Framework** is a software development framework developed by Microsoft that provides a runtime environment and a set of libraries and tools for building and running applications on Windows operating systems. The framework includes a variety of programming languages, such as C#, F#, and Visual Basic, and supports a range of application types, including desktop, web, mobile, and gaming applications.

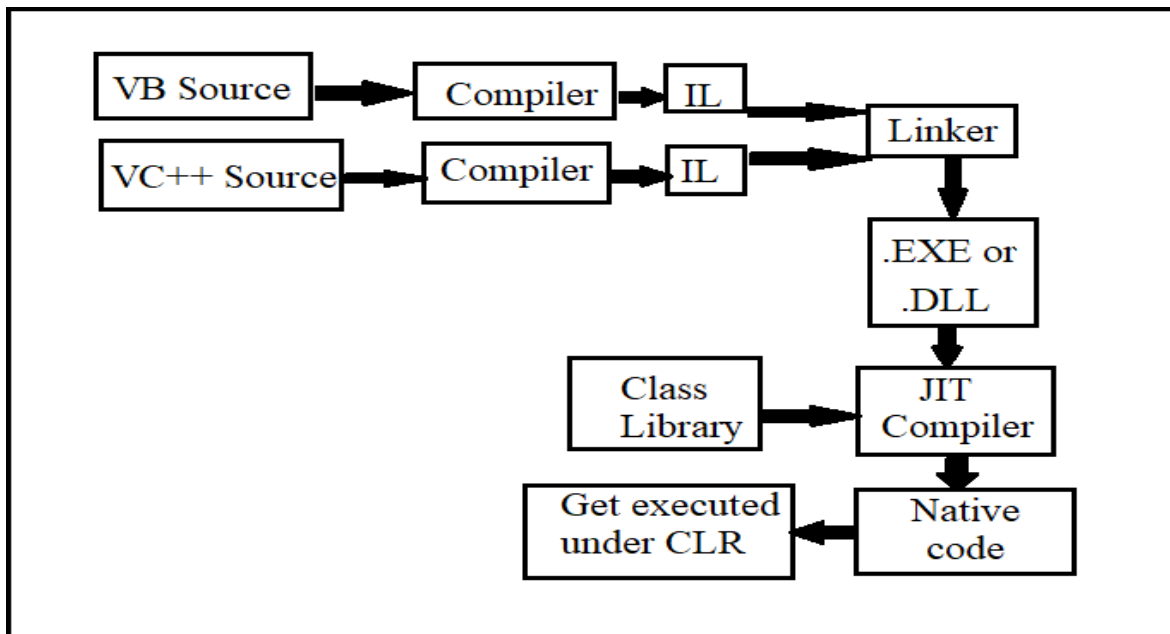
.NET framework architecture is shown in the below diagram:



1. The .NET Framework includes two main components: the Common Language Runtime (CLR) and the .NET Framework Class Library. The CLR is responsible for managing the execution of code written in any of the supported languages, while the class library provides a large set of pre-built functions and classes that can be used to create a wide range of applications.
2. One of the key advantages of the .NET Framework is its support for a variety of programming languages. This means that developers can choose the language that best fits their needs and expertise, while still being able to use the same set of libraries and tools provided by the framework.
3. Another advantage of the .NET Framework is its support for a variety of application types. The framework includes libraries and tools for creating desktop, web, mobile, and gaming applications, which makes it a versatile choice for developers working on a wide range of projects.
4. The .NET Framework also provides a number of features that help improve the security, reliability, and performance of applications. These include features such as code access security, automatic memory management, and just-in-time (JIT) compilation, which helps improve the speed of application execution.
5. The .NET Framework is also designed to integrate with other Microsoft technologies, such as Microsoft SQL Server, Microsoft SharePoint, and Microsoft Office, which can make it easier to build applications that work seamlessly with other Microsoft products.

Common Language Runtime:

The Common Language Runtime (CLR) is a component of the Microsoft .NET Framework that manages the execution of .NET applications. It is responsible for loading and executing the code written in various .NET programming languages, including C#, VB.NET, F#, and others.



VB source and VC++ source provide the source code to the compiler.

Compilers are tools that translate source code to machine understandable language. In .Net, the CLR provides a compiler that converts that source code to Microsoft Intermediate Language (MSIL). However, the MSIL code cannot be executed by the machine.

Intermediate language (IL) is an object-oriented programming language designed to be used by compilers for the .NET Framework. The IL is used by the .NET Framework to generate machine-independent code as the output of compilation of the source code written in any .NET programming language.

A linker or link editor is a computer system program that takes one or more object files (generated by a compiler or an assembler) and combines them into a single executable file, library file, or another "object" file(.EXE or .DLL(Dynamic Link Library)).

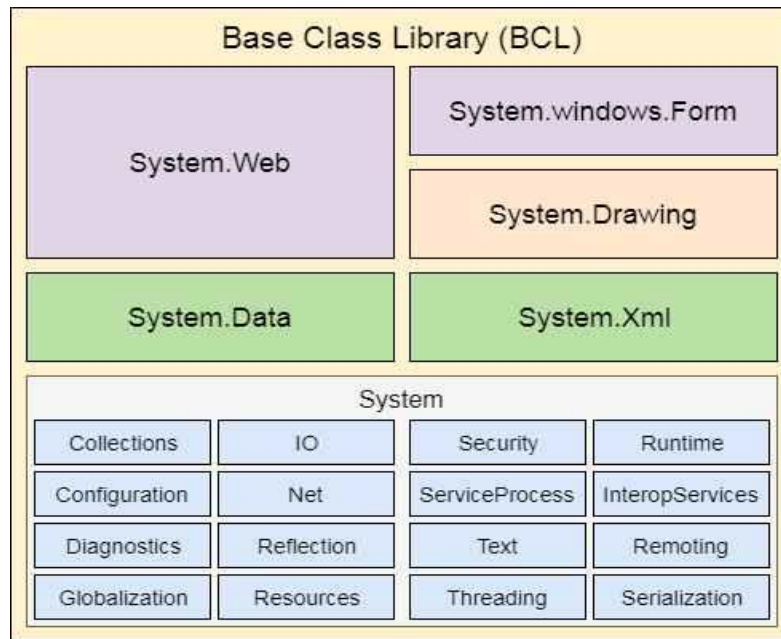
Class Library is the sub part of the Framework that provides library support to Common Language Runtime to work properly. It is the collection of classes, namespaces, interfaces and value types that are used for .NET applications.

Just In Time or JIT Compilation is the process of taking those binaries of the Intermediate Language and converting them to native instructions that are understood by the machine.

Finally after completion of all these process we will execute our program under this CLR.

.NET Base classes

It is the sub part of the Framework that provides library support to Common Language Runtime to work properly. It is the collection of classes, namespaces, interfaces and value types that are used for .NET applications. The developer can use these classes to implement the application quickly.



It includes several kinds of classes

- classes and types related to basic database management
- classes to debug in application to trace execution
- classes that allow reading writing to files and other data sheets
- classes that perform common mathematical operations
- classes to inspect metadata of an assembly
- classes to enable security capabilities

User & Program Interface:

It is the top layer of the dotnet architecture.

The .NET framework provides the following tools for managing user and application interfaces:

- Windows forms
- Web Forms
- Console Applications
- Web Services

Windows Forms is a UI framework for building Windows desktop apps. It provides one of the most productive ways to create desktop apps based on the visual designer provided in Visual Studio.

Web Forms is a UI framework for building web application. ASP.NET Web Forms is a web application framework and one of several programming models supported by the Microsoft ASP.NET technology. Web Forms applications can be written in any programming language which supports the Common Language Runtime, such as C# or Visual Basic

(Web applications require an Internet connection and use a web browser as a runtime environment. On the other hand, desktop applications are installed on a computer and run locally.)

Console applications are command line oriented applications that allow user to read characters from the console, write characters to the console.

Console applications are typically designed without graphical user interface and are compiled in a stand-alone executable file.

A console application is run from the command line with input and output [information](#) being exchanged between the command prompt and the running application. Because information can be written to and read from the console window, this makes the console application a great way to learn new programming techniques without having to be concerned with the user interface.

A web service is an XML-based information exchange system that creates direct interaction between the two applications over the internet or network in order to exchange data or information. It enables us to communicate and exchange data or information between the two different applications created in the same or different languages over the internet or network.

.NET Products:

R# (ReSharper):

ReSharper makes Visual Studio a much better IDE. ReSharper extends Visual Studio with over 2200 on-the-fly code inspections for C#, VB.NET, ASP.NET, JavaScript, TypeScript and other technologies. For most inspections, ReSharper provides quick-fixes to improve the code.

ReSharper features are Code

analysis

ReSharper extends Visual Studio with over 2200 on-the-fly code inspections for C#, VB.NET, ASP.NET, JavaScript, TypeScript and other technologies. Find and remove unused code, Migrate your code to the latest C# version, Convert loops to LINQ at will, Find and prevent possible exceptions, Use a common naming standard. All are made possible with ReSharper's code analysis.

Refactorings

60+ refactorings and 450+ context actions help safely organize code and move it around the solution, distribute responsibility, decouple, decrease complexity, or simply use alternative language syntax.

Navigation and search

ReSharper helps instantly get to any code in a solution, no matter how large the solution is. It can also navigate you from any symbol to its related code such as implementations of a given interface, extension methods of a class, or usages of a field.

Code generation

ReSharper can create a lot of code for you: from new files and classes to conditional blocks and GUIDs; from methods and properties required by an interface being implemented to equality checks and formatting members.

RD(Rider):

Rider is a cross-platform .NET IDE based on the IntelliJ platform and ReSharper. Rider provides the power to productively develop a wide range of applications, including .NET desktop applications, services and libraries, Unity and Unreal Engine games, Xamarin apps, ASP.NET, and ASP.NET Core web applications, and more. On Windows, macOS, and Linux.

Rider features are

Code analysis: Rider boasts 2,200+ live code inspections, with automated quick-fixes to resolve detected issues individually or in bulk.

Code editing: Rider's rich editor features different kinds of code completion and code templates, auto-inserting matching braces and import directives, quick info tooltips and gutter icons for inheritance navigation, context actions, and much more.

Refactorings: Most of ReSharper's 60+ refactorings are already available in Rider, and its 450+ context actions are all there. Rename, extract methods, interfaces and classes, move and copy types, use alternative syntax, and a lot more!

Unit test runner:

Rider helps you run and debug unit tests based on NUnit, xUnit.net, or MSTest. You can explore tests, group them in different ways, break them down into individual sessions, see test output and navigate to source code from stack traces.

Debugger and more tools

Rider includes a debugger that works with .NET Framework, Mono and .NET Core applications, letting you step, watch, evaluate, and run to cursor. Other tools include a stack trace explorer, NuGet browser, and VCS and database support.

Databases and SQL

Work with SQL and databases without leaving Rider. Connect to databases, edit schemas and table data, run queries, and even analyze schemas with UML diagrams.

Navigation and search

Jump to any file, type, or member in your code base instantly, as well as quickly find settings and actions. Find usages of any symbol, or navigate from a symbol to the base and derived symbols, extension methods, or implementations.

Front-end technologies

Rider comes with JavaScript, TypeScript, HTML, CSS and Sass support built in. Take advantage of the refactorings, debugging, and unit testing capabilities included from WebStorm.

Extensibility

True to its roots, Rider supports a wide array of plugins developed for ReSharper and IntelliJ Platform.

DC(dotCover)

dotCover is a .NET unit test runner and code coverage tool that integrates with Visual Studio and JetBrains Rider. Make sure you know to what extent your code is covered with unit tests. dotCover calculates and reports statement-level code coverage in applications targeting .NET Framework, .NET Core, Mono, or Mono for Unity.

dotCover features are Unit

tests

Execute and debug unit tests and run coverage analysis of unit tests in Visual Studio or using the command-line utility. dotCover supports many unit testing frameworks including MSTest, NUnit, xUnit, and MSpec.

Manual test sessions

For manual test sessions, dotCover shows what parts of code were called during the session and which parts were not reached. Using the coverage analysis data, you can easily get to the source code of the tested application.

Detecting hot spots

Detect potential risk areas at a glance with Hot Spots view, which shows the most complex methods with the least coverage. Get your coverage analysis off to a good start with an intuitive graphical representation

Seamless integration with Visual Studio and JetBrains Rider

dotCover integrates with Visual Studio and JetBrains Rider, giving you the advantage of analyzing and visualizing code coverage without leaving the code editor. It supports Visual Studio 2010–2017 and JetBrains Rider 2018.2 or later.

Continuous testing

dotCover can figure out on-the-fly which unit tests are affected by your latest code changes, and automatically re-runs the affected tests for you. The continuous testing mode can be switched on for any unit test session: this way you can choose which tests you want to run continuously and which to run the traditional way.

DM(dotMemory)

dotMemory allows you to analyze memory usage in a variety of .NET and .NET Core applications: desktop applications, Windows services, ASP.NET web applications, IIS, IIS Express, arbitrary .NET processes, and more.

dotMemory features are

Easily control the profiling process

Use the timeline view to monitor the memory consumed by your application in real time. Collect the info on allocations as you need it.

Collect profiling data

Take snapshots with a single click without stopping your profiling session, add them to compare or browse memory traffic during a particular timeframe.

Auto-detect issues

Thanks to the multiple powerful built-in inspections, most common memory issues are detected with a single click, no manual effort required. Just get a snapshot and be guided toward optimizing the memory usage.

Analyse deeper

For in-depth analysis, dotMemory offers many different criteria to apply to memory usage data, so you can view the data from thousands of different angles, and drill-down, dice, slice or pivot as you wish.

Determine traffic

Analyse memory traffic in a particular timeframe to understand the cause of excessive allocations and garbage collections. Determine how objects were allocated and collected and what methods are behind these allocations.

DT(dot.Trace)

dotTrace helps you detect performance bottlenecks in a variety of .NET and .NET Core applications: WPF and Universal Windows Platform, ASP.NET, Windows services, WCF services, and unit tests. Mono and Unity applications are supported as well.

dotTrace features are

Analyze any performance issues

Get accurate data on calls execution time. In the timeline profiling mode, get the same call time data but bound to a timeline. Use timeline data to analyze performance issues where the order of events matters, such as UI freezes, excessive garbage collection, uneven workload distribution, inefficient file I/O, and others.

Profile SQL queries and HTTP requests

Find out how a particular query affects application performance and why this query is running slow. If you develop a web application, learn how it processes incoming HTTP requests.

Profile static methods and unit tests

You can instantly get comprehensive profiling data without having to build your solution, run the profiler and perform a usage scenario. Just add a static method alongside the code that you're optimizing, and profile it! If you employ unit testing in your project, dotTrace helps you profile any test in just a few clicks. In addition, dotTrace offers a special TeamCity plugin which you can use to profile integration tests on your CI server and prevent performance regressions.

Profile and analyze results in Rider and Visual Studio

With dotTrace, you can profile and analyze call stacks without leaving your IDE. It is already integrated into Visual Studio and JetBrains Rider. You can even navigate to a method declaration from a hot spot in the call tree.

.NET Services

.NET services are quite popular these days. The development of Windows-based services and web-based services are on rise. The users can easily make use of the services without having to look into the implementation details or know how they work in the first place. There are few advantages of going for .NET services and they are:

- Makes communication between applications easier
- Rapid development
- Easy to distribute information
- Less cost and easy maintenance
- Web services enjoy data interoperability due to XML use
- Lightweight architecture
- Less application and image loading time
- .NET APIs comes with built-in testing feature which detects mistakes automatically while coding

UNIT II

C# Languages Basics, Data Types, Type Conversion, Boxing & Unboxing, Conditional Statements, Looping, Methods in C#, Properties, Arrays, Indexes, Structures, Enumerations, Strings, Regular Expressions, Collections

C# Languages Basics:

A C# program consists of the following parts:

- Namespace declaration
- A class
- Class methods
- Class attributes
- A Main method
- Statements and Expressions
- Comments Important

points

- C# is case sensitive.
- All statements and expression must end with a semicolon (;).
- The program execution starts at the Main method.
- Unlike Java, program file name could be different from the class name.

Example program

```
using System;
namespace HelloWorldApplication
{
    Class HelloWorld
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World");
            Console.ReadKey();
        }
    }
}
```

Output:

Hello World

The first line of the program **using System;** - the **using** keyword is used to include the **System** namespace in the program. A program generally has multiple **using** statements.

The next line has the **namespace** declaration. A **namespace** is a collection of classes. The *HelloWorldApplication* namespace contains the class *HelloWorld*.

The next line has a **class** declaration, the class *HelloWorld* contains the data and method definitions that your program uses. Classes generally contain multiple methods. Methods define the behavior of the class. However, the *HelloWorld* class has only one method **Main**.

The next line defines the **Main** method, which is the **entry point** for all C# programs. The**Main** method states what the class does when executed.

The next line `/*...*/` is ignored by the compiler and it is put to add **comments**in the program.

The Main method specifies its behavior with the statement **Console.WriteLine("Hello World");**

WriteLine is a method of the *Console* class defined in the *System* namespace. This statement causes the message "Hello, World!" to be displayed on the screen.

The last line **Console.ReadKey();** is for the VS.NET Users. This makes the program wait for a key press and it prevents the screen from running and closing quickly when the program is launched from Visual Studio .NET.

Data Types:

A data type specifies the size and type of variable values.

It is important to use the correct data type for the corresponding variable; to avoid errors, to save time and memory, but it will also make your code more maintainable and readable. The most common data types are:

Data Type	Size	Description
int	4 bytes	Stores whole numbers from - 2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from - 9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
bool	1 bit	Stores true or false values

char	2 bytes	Stores a single character/letter, surrounded by single quotes
string	2 bytes per character	Stores a sequence of characters, surrounded by double quotes

Examples:

Int:

```
int myNum = 100000;  
Console.WriteLine(myNum); // Output 100000
```

Long:

```
long myNum = 15000000000L;  
Console.WriteLine(myNum); // Output 15000000000
```

Float:

```
float myNum = 5.75F;  
Console.WriteLine(myNum); // Outputs 5.75
```

Double:

```
double myNum = 19.99D;  
Console.WriteLine(myNum); // Outputs 19.99
```

Boolean:

```
bool isCSharpFun = true;  
bool isCSharpHard = false;  
Console.WriteLine(isCSharpFun); // Output True  
Console.WriteLine(isCSharpHard); // Output False
```

Character:

```
char myGrade = 'B';  
Console.WriteLine(myGrade); // Output B
```

String:

```
string greeting = "Hello World";  
Console.WriteLine(greeting); // Output Hello World
```

Type conversion:

Type casting is when you assign a value of one data type to another type.

In C#, there are two types of casting:

1. **Implicit Casting** (automatically) - converting a smaller type to a larger type size. Implicit casting is done automatically when passing a smaller size type to a larger size type:

char -> int -> long -> float -> double

Example:

```
using System;  
namespace MyApplication  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            int myInt = 9;  
            double myDouble = myInt; // Automatic casting: int to double  
            Console.WriteLine(myInt); // Output 9  
            Console.WriteLine(myDouble); // Output 9  
        }  
    }  
}
```

2. **Explicit Casting** (manually) - converting a larger type to a smaller size type. Explicit casting must be done manually by placing the type in parentheses in front of the value. These conversions are done explicitly by users using the predefined functions. Explicit conversions require a cast operator.

double -> float -> long -> int -> char

Example:

```
using System;  
namespace MyApplication  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            double myDouble = 9.78;  
            int myInt = (int) myDouble; // Manual casting: double to int  
            Console.WriteLine(myDouble); // Output 9.78  
        }  
    }  
}
```

```

        Console.WriteLine(myInt); // Output 9
    }
}
}

```

Type Conversion Methods

It is also possible to convert data types explicitly by using built-in methods, such as:

- 1 . **ToBoolean**-----Converts a type to a Boolean value, where possible.
- 2 . **ToByte** ----- Converts a type to a byte.
- 3 . **ToChar** ----- Converts a type to a single Unicode character, where possible.
- 4 . **ToDateTime** --- Converts a type *integer or string type* to date-time structures.
- 5 . **ToDecimal** ---- Converts a floating point or integer type to a decimal type.
- 6 **ToDouble** ----- Converts a type to a double type.
- 7 . **ToInt16** ----- Converts a type to a 16-bit integer.
- 8 . **ToInt32** --- Converts a type to a 32-bit integer.
- 9 . **ToInt64** --- Converts a type to a 64-bit integer.
- 10 . **ToSbyte**-- Converts a type to a signed byte type.
- 11 . **ToSingle**--- Converts a type to a small floating point number.
- 12 . **ToString** -- Converts a type to a string.
14. **ToUInt16** -- Converts a type to an unsigned int type.
15. **ToUInt32** -- Converts a type to an unsigned long type.
16. **ToUInt64** -- Converts a type to an unsigned big integer.

Eg:

```

using System ;
namespace TypeConversionApplication
{
    class StringConversion
    {
        static void Main(string[] args)
        {
            int i = 75;
            float f = 53.005f;
            double d = 2345.7652;
            bool b = true;
            Console.WriteLine(i.ToString());
            Console.WriteLine(f.ToString());
            Console.WriteLine(d.ToString());
            Console.WriteLine(b.ToString());
            Console.ReadKey();
        }
    }
}

```

Output:

```

75
53.005
2345.7652

```

True

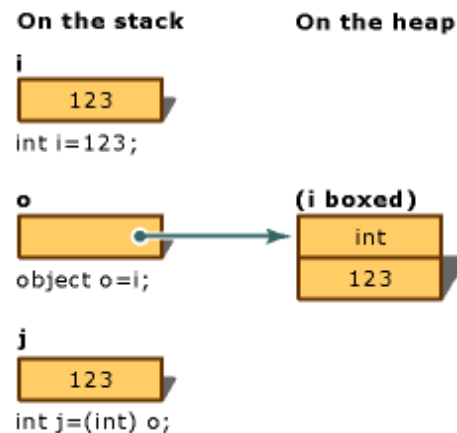
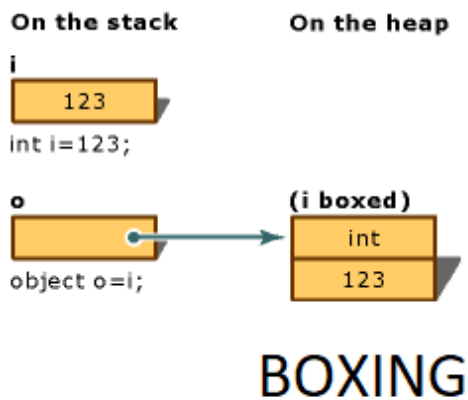
Boxing and Unboxing:

Boxing

Implicit conversion of a value type (int, char etc.) to a reference type (object), is known as Boxing. In boxing process, a value type is being allocated on the heap rather than the stack.

Unboxing

Explicit conversion of same reference type (which is being created by boxing process); back to a value type is known as unboxing. In unboxing process, boxed value type is unboxed from the heap and assigned to a value type which is being allocated on the stack.



Example:

```
using System;
namespace BoxingUnboxingApplication
{
    class BoxingUnboxing
    {
        // Main Method
        static public void Main(string[] args)
        {
            // assigned int value
            // 23 to num
            int i = 123;

            // boxing
            object o = i;

            // unboxing
            int j = (int)o;

            // Display result
            Console.WriteLine("Value of Boxing is : " + o);
            Console.WriteLine("Value of unboxing is : " + j);
        }
    }
}
```

```
}
```

Output:

Value of Boxing is : 23

Value of Unboxing is : 23

Conditional Statements:

C# supports the usual logical conditions from mathematics:

Less than: $a < b$

Less than or equal to: $a \leq b$

Greater than: $a > b$

Greater than or equal to: $a \geq b$

Equal to: $a == b$

Not Equal to: $a != b$

C# has the following conditional statements:

If : Use if to specify a block of code to be executed, if a specified condition is True

Syntax:

```
if (condition)
{
    // block of code to be executed if the condition is True
}
```

Example:

```
using System;
namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            if (20 > 18)
            {
                Console.WriteLine("20 is greater than 18");
            }
        }
    }
}
```

Output:

20 is greater than 18

If –else: Use the **else** statement to specify a block of code to be executed if the condition is **False**.

Syntax:

```
if (condition)
```

```

{
    // block of code to be executed if the condition is True
}
else
{
    // block of code to be executed if the condition is False
}

```

Example:

```

using System;
namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            int a=5
            int b=15
            if (a > b)
            {
                Console.WriteLine("a is greater than b");
            }
            else
            {
                Console.WriteLine("a is not greater than b");
            }
        }
    }
}

```

Output:

a is not greater than b

The else if Statement

Use the **else if** statement to specify a new condition if the first condition is **False**.

Syntax:

```

if (condition1)
{
    // block of code to be executed if condition1 is True
}
else if (condition2)
{
    // block of code to be executed if the condition1 is false and condition2 is True
}
else
{
    // block of code to be executed if the condition1 is false and condition2 is False
}

```

Example:

```

using System;
namespace MyApplication
{

```

```

class Program
{
    static void Main(string[] args)
    {
        int a=15
        int b=15
        if (a > b)
        {
            Console.WriteLine("a is greater than b");
        }
        else (a < b)
        {
            Console.WriteLine("a is less than b");
        }
        else
        {
            Console.WriteLine("a is equal to b");
        }
    }
}

```

Output:

a is equal to b

Short if: There is also a short-hand if else, which is known as the **ternary operator** because it consists of three operands. It can be used to replace multiple lines of code with a single line. It is often used to replace simple if else statements:

syntax : *variable = (condition) ? expressionTrue : expressionFalse;*

Example:

```

using System;
namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 10;
            string result = (a < 18) ? "a is greater than 18" : "a is not greater than 18";
            Console.WriteLine(result);
        }
    }
}

```

Output:

a is greater than 18

Switch: Use the **switch** statement to select one of many code blocks to be executed.

switch syntax:

```
switch(expression)
{
    case x:
        // code block break;
    case y:
        // code block break; default:
        // code block break;
}
```

Example:

```
int day = 5;
switch (day)
{
    case 1:
        Console.WriteLine("Sunday");
        break;

    case 2:
        Console.WriteLine("Monday");
        break;
    case 3:
        Console.WriteLine("Tuesday");
        break;
    case 4:
        Console.WriteLine("Wednesday");
        break;
    case 5:
        Console.WriteLine("Thursday");
        break;
    case 6:
        Console.WriteLine("Friday");
        break;
    case 7:
        Console.WriteLine("Saturday");
        break;
default:
    Console.WriteLine("Not in week");
    break;
}
```

Output:

Thursday

Loop statements:

1.While loop:

The **while** loop loops through a block of code as long as a specified condition is **True**:

Syntax:

```
while (condition)
{
    // code block to be executed
}
```


Example:

```
using System;
namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            int i = 0;
            while (i < 5)
            {
                Console.WriteLine(i);
                i++;
            }
        }
    }
}
```

Output:

```
0
1
2
3
4
```

2.Do-while loop:

The do/while loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

Syntax:

```
do
{
    // code block to be executed
}
while (condition);
```

Example:

```
using System;

namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            int i = 0;
            do
            {
```

```

    Console.WriteLine(i);
    i++;
}
while (i < 5);
}
}

```

Output:

```

0
1
2
3
4

```

3.For loop:

When you know exactly how many times you want to loop through a block of code, use the for loop instead of a while loop:

Syntax:

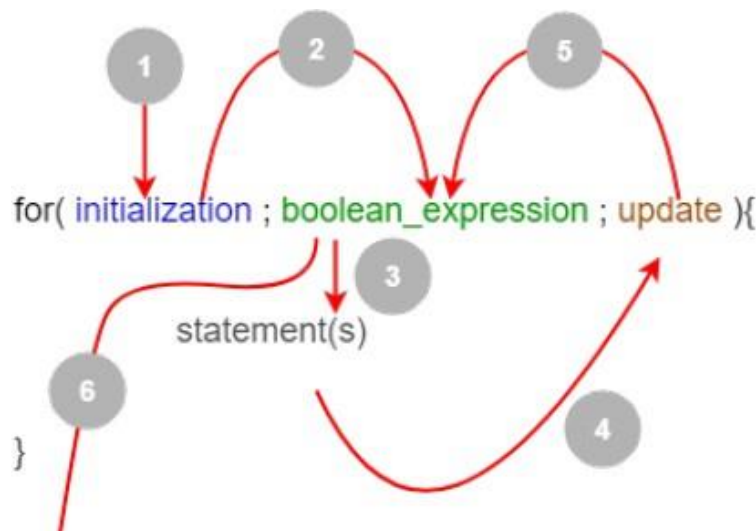
```

for (statement 1; statement 2; statement 3)
{
    // code block to be executed
}

```

Statement 1 is executed (one time) before the execution of the code block. Statement 2 defines the condition for executing the code block.

Statement 3 is executed (every time) after the code block has been executed.



For loop can be in the following forms

- For Loop without Update**

```

for (statement 1; statement 2; )
{
    // code block to be executed statement 3
}

```

```
}
```

- **For Loop without Initialization**

statement 1

for (; statement 2; statement 3)

```
{
```

// code block to be executed

```
}
```

- **For Loop without Boolean Expression or Condition**

for (statement 1;; statement 3)

```
{
```

statement 2

// code block to be executed

```
}
```

Example:

```
using System;
namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int i = 0; i < 5; i++)
            {
                Console.WriteLine(i);
            }
        }
    }
}
```

Output:

```
0
1
2
3
4
```

4.Nested Loops

It is also possible to place a loop inside another loop. This is called a **nested loop**. The "inner loop" will be executed one time for each iteration of the "outer loop":

i)Nested for loop: An inner for loop inside the outer for loop is called nested for loop. The "inner for loop" will be executed one time for each iteration of the "outer for loop":

Syntax:

```
// Outer loop
while (condition)
{
    // code block to be executed
    // Inner loop
    while (condition)
    {
        // code block to be executed
    }
}
```

Example:

```
using System;
namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            // Outer loop
            for (int i = 1; i <= 2; ++i)
            {
                Console.WriteLine("Outer: " + i); // Executes 2 times

                // Inner loop
                for (int j = 1; j <= 3; j++)
                {
                    Console.WriteLine(" Inner: " + j); // Executes 6 times (2 * 3)
                }
            }
        }
    }
}
```

Output:

```
Outer:1
Inner:1
Inner:2
Inner:3
Outer:2
Inner:1
Inner:2
Inner:3
```

i)Nested while loop: An inner while loop inside the outer for loop is called nested while loop. The "inner while loop" will be executed one time for each iteration of the "outer while loop":

Syntax:

```

// Outer loop
for (statement 1; statement 2; statement 3)
{
    // code block to be executed
// Inner loop
for (statement 1; statement 2; statement 3)
{
    // code block to be executed
}
}

```

Example:

```

using System;
namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            // Outer loop
            int i = 1;
            while (i < =2)
            {
                Console.WriteLine("Outer: " + i); // Executes 2 times
            // Inner loop
            int j = 1;
            while (j < =3)
            {
                Console.WriteLine(" Inner: " + j); // Executes 6 times (2 * 3)
                j++;
            }
        }
    }
}

```

Output:

```

Outer:1
Inner:1
Inner:2
Inner:3
Outer:2
Inner:1
Inner:2
Inner:3

```

5.Foreach loop:

There is also a foreach loop, which is used exclusively to loop through elements in an array:

Syntax:

```
foreach (type variableName in arrayName)
{
    // code block to be executed
}
```

Example:

```
using System;
class Program
{
    static void Main(string[] args)
    {
        int[] numbers = { 1, 2, 3, 4, 5 };
        foreach (int number in numbers)
        {
            Console.WriteLine(number);
        }
    }
}
```

Output:

```
1
2
3
4
5
```

Methods:

A method is a block of code which only runs when it is called. Every C# program has at least one class with a method named Main.

To use a method, you need to:

1. Define the method
2. Call the method

1. Defining Methods in C#

When you define a method, you basically declare the elements of its structure. The syntax for defining a method in C# is as follows:

```
<Access Specifier> <Return Type> <Method Name>(Parameter List)
{
    Method Body
}
```

Following are the various elements of a method:

Access Specifier: This determines the visibility of a variable or a method from another class.

In C#, an access specifier is used to control the accessibility of a method. Access specifiers determine which parts of the code have access to the method. There are several access specifiers in C#, including:

public: A method declared with the public access specifier is accessible from any part of the code.

private: A method declared with the private access specifier is only accessible within the same class.

internal: A method declared with the internal access specifier is accessible within the same assembly.

protected: A method declared with the protected access specifier is accessible within the same class and any derived classes.

Ex:

```
using System;
namespace MyApplication
{
    class Program
    {
        private void MyMethod()
        {
            Console.WriteLine("Hello World");
        }
        static void Main(string[] args)
        {
            MyMethod();
        }
    }
}
```

Output:

compile-time error

In this example, the DoSomething method is declared with a private access specifier, which means it can only be accessed within the Program class. If you tried to call the DoSomething method from outside the Program class, you would get a compile-time error.

Return type: A method may return a value. The return type is the data type of the value the method returns. If the method is not returning any values, then the return type is void.

Method name: Method name is a unique identifier and it is case sensitive. It cannot be same as any other identifier declared in the class.

Parameter list: Enclosed between parentheses, the parameters are used to pass and receive data from a method. The parameter list refers to the type, order, and number of the parameters of a method. Parameters are optional; that is, a method may contain no parameters.

Method body: This contains the set of instructions needed to complete the required activity.

2. Calling Methods in C#:

You can call a method using the name of the method. The following example illustrates this: using System ;

EX:

```
using System;
namespace MyApplication
{
    class Program
    {
        static void MyMethod()
        {
```

```

    Console.WriteLine("Hello World");
}
static void Main(string[] args)
{
    MyMethod();
}
}
}

```

Output:

Hello World

3. Method with multiple times calling:

```

using System;
namespace MyApplication
{
    class Program
    {
        static void MyMethod()
        {
            Console.WriteLine("Hello, World!");
        }

        static void Main(string[] args)
        {
            MyMethod();
            MyMethod();
            MyMethod();
        }
    }
}

```

Output:

Hello, World!

Hello, World!

Hello, World!

Properties:

private variables can only be accessed within the same class (an outside class has no access to it). However, sometimes we need to access them - and it can be done with properties.

A property is like a combination of a variable and a method, and it has two methods: a **get** and a **set** method.

A property is a member that provides a flexible mechanism to read, write, or compute the value of a private field. Properties can be used as if they're public data members, but they're special methods called accessors

Accessors

The accessor of a property contains the executable statements that helps in getting

Reading or computing or setting writing the property. The accessor declarations can contain a get accessor, a set accessor, or both.

Syntax:

```

// Declare a Code property of type string:
public string Code

```



```

{
get
{
return code;
}
set
{
code = value;
}
}
// Declare a Name property of type string:
public string Name
{
get
{
return name;
}
set
{
name = value;
}
}
// Declare a Age property of type int:
public int Age
{
get
{
return age;
}
set
{
age = value;
}
}

```

Ex:

```

using System;
namespace MyApplication
{
class Person
{
private string name; // field
public string Name // property
{
get
{
return name;
}
set
{

```

```

        name = value;
    }
}
}

class Program
{
    static void Main(string[] args)
    {
        Person myObj = new Person();
        myObj.Name = "Lakshmi";
        Console.WriteLine(myObj.Name);
    }
}
}

```

Output:

Lakshmi

Array:

An array stores a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type stored at contiguous memory locations. Instead of declaring individual variables, such as number0, number1, ..., and number99, you declare one array variable such as numbers and use numbers[0], numbers[1], and ..., numbers[99] to represent individual variables. A specific element in an array is accessed by an index. All arrays consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.

Declaring Arrays

To declare an array in C#, you can use the following syntax:

```
datatype[] arrayName;
```

where,

datatype is used to specify the type of elements in the array.

[] specifies the rank of the array. The rank specifies the size of the array. *arrayName* specifies the name of the array.

For example,

```
double[] balance;
```

Initializing an Array

Declaring an array does not initialize the array in the memory. When the array variable is initialized, you can assign values to the array.

Array is a reference type, so you need to use the new keyword to create an instance of the array. For example,

- *double[] balance = new double[10];*

Assigning Values to an Array

You can assign values to individual array elements, by using the index number, like:

- `double[] balance = new double[10];`
- `balance[0] = 4500.0;`

You can assign values to the array at the time of declaration, as shown:

- `double[] balance = { 2340.0, 4523.69, 3421.0};`

When you create an array, C# compiler implicitly initializes each array element to a default value depending on the array type. For example, for an int array all elements are initialized to 0.

Accessing Array Elements

An element is accessed by indexing the array name. This is done by placing the index of the element within square brackets after the name of the array. For example,

- `double salary = balance[9];`

EX:

```
using System;
namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            string[] branch = {"ECE", "CSE", "EEE", "MECH", "CIVIL"};
            Console.WriteLine(branch[0]);
        }
    }
}
```

Output:

ECE

Indexes:

An indexer allows an object to be indexed such as an array. When you define an indexer for a class, this class behaves similar to a virtual array. You can then access the instance of this class using the array access operator `[]`.

Syntax

A one dimensional indexer has the following syntax:

```
element-type this[int index]
{
    // The get accessor.
    get
    {
        // return the value specified by index
    }
    // The set accessor.
    set
    {
```

```
// set the value specified by index
}  
}
```

Use of Indexers

Declaration of behavior of an indexer is to some extent similar to a property. similar to the properties, you use get and set accessors for defining an indexer. However, properties return or set a specific data member, whereas indexers returns or sets a particular value from the object instance. In other words, it breaks the instance data into smaller parts and indexes each part, gets or sets each part. Defining a property involves providing a property name. Indexers are not defined with names, but with the this keyword, which refers to the object instance. The following example demonstrates the concept:

```
using System;  
namespace myApplication  
// class declaration  
class IndexerCreation  
{  
  
    // class members  
    private string[] val = new string[3];  
  
    // Indexer declaration  
    // public - access modifier  
    // string - the return type of the Indexer  
    // this - is the keyword having a parameters list  
    public string this[int index]  
    {  
  
        // get Accessor  
        // retrieving the values  
        // stored in val[] array  
        // of strings  
        get  
        {  
  
            return val[index];  
        }  
  
        // set Accessor  
        // setting the value at  
        // passed index of val  
        set  
        {  
  
            // value keyword is used  
            // to define the value  
            // being assigned by the  
            // set indexer.  
            val[index] = value;  
        }  
    }  
}  
  
// Driver Class  
class main {  
  
    // Main Method  
    public static void Main() {
```

```

// creating an object of parent class which
// acts as primary address for using Indexer
IndexerCreation ic = new IndexerCreation();

// Inserting values in ic[]
// Here we are using the object
// of class as an array
ic[0] = "C";
ic[1] = "CPP";
ic[2] = "CSHARP";

Console.WriteLine("Printing values stored in objects used as arrays\n");

// printing values
Console.WriteLine("First value = {0}", ic[0]);
Console.WriteLine("Second value = {0}", ic[1]);
Console.WriteLine("Third value = {0}", ic[2]);

}
}

```

Output:

Printing values stored in objects used as arrays
First value = C
Second value = CPP
Third value = CSHARP

Structure:

In C#, a structure is a value type data type. It helps you to make a single variable hold related data of various data types. The **struct** keyword is used for creating a structure. Structures are used to represent a record. Suppose you want to keep track of your books in a library. You might want to track the following attributes about each book:

Title Author Subject
Book ID

Defining a Structure

To define a structure, you must use the struct statement. The struct statement defines a new data type, with more than one member for your program.

For example, here is the way you can declare the Book structure:

```

struct Books
{
    public string title;
    public string author;
    public string subject;
    public int book_id;
}

```

Features of C# Structures

You have already used a simple structure named Books. Structures in C# are quite different from that in traditional C or C++.

The C# structures have the following features:

- Structures can have methods, fields, indexers, properties, operator methods, and events.
- Structures can have defined constructors, but not destructors. However, you cannot define a default constructor for a structure. The default constructor is automatically defined and cannot be changed.
- Unlike classes, structures cannot inherit other structures or classes.

- Structures cannot be used as a base for other structures or classes.
- A structure can implement one or more interfaces.
- Structure members cannot be specified as abstract, virtual, or protected.
- When you create a struct object using the New operator, it gets created and the appropriate constructor is called. Unlike classes, structs can be instantiated without using the New operator.
- If the New operator is not used, the fields remain unassigned and the object cannot be used until all the fields are initialized.

Class versus Structure

Classes and Structures have the following basic differences:

- classes are reference types and structs are value types
- structures do not support inheritance
- structures cannot have default constructor

Enumerations:

An enum is a special "class" that represents a group of constants (unchanges/read-only variables). To create an enum, use the enum keyword (instead of class or interface), and separate the enum items with a comma:

In C#, enumeration is done using the keyword enum . C# enum defines a list of elements assigned with an ordered set of integer values.

The syntax to declare an enum is shown below:

```
enum EnumName{
    CONSTANT_NAME_1,
    CONSTANT_NAME_2,
    CONSTANT_NAME_N
}
```

where EnumName is the name given to this enum and with which we can reference. CONSTANT_NAME_1 , CONSTANT_NAME_2 , . . etc., are the constants inside the enum.

Ex:

```
using System;
namespace MyApplication
{
    enum Level
    {
        Low,
        Medium,
        High
    }
    class Program
    {
        static void Main(string[] args)
        {
            Level myVar = Level.Medium;
            Console.WriteLine(myVar);
        }
    }
}
```

Output:

Medium

Strings:

Strings are used for storing text. A **string** variable contains a collection of characters surrounded by double quotes:

In C#, you can use strings as array of characters, However, more common practice is to use the string keyword to declare a string variable. The string keyword is an alias for the System.String class.

Creating a String Object

You can create string object using one of the following methods:

- By assigning a string literal to a String variable
- By using a String class constructor
- By using the string concatenation operator +
- By retrieving a property or calling a method that returns a string
- By calling a formatting method to convert a value or an object to its string representation

Ex:

```
using System;
namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            string greeting = "Hello";
            Console.WriteLine(greeting);
        }
    }
}
```

Output:

Hello

Properties of the String Class

The String class has the following two properties:

- 1.Chars--Gets the Char object at a specified position in the current String object.
- 2 .Length--Gets the number of characters in the current String object.

String Length

A string in C# is actually an object, which contain properties and methods that can perform certain operations on strings. For example, the length of a string can be found with the **Length** property:

Ex:

```
using System;
namespace MyApplication
{
    class Program
    {
```

```

static void Main(string[] args)
{
    string txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    Console.WriteLine("The length of the txt string is: " + txt.Length);
}
}

```

Output:

The length of the txt string is: 26

Uppercase and Lowercase Methods:

There are many string methods available, for example **ToUpper()** and **ToLower()**, which returns a copy of the string converted to uppercase or lowercase:

Ex:

```

using System;
namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            string txt = "Hello World";
            Console.WriteLine(txt.ToUpper()); // Outputs "HELLO WORLD"
            Console.WriteLine(txt.ToLower()); // Outputs "hello world"
        }
    }
}

```

Regular Expressions:

A regular expression is a pattern that could be matched against an input text. The .Net framework provides a regular expression engine that allows such matching. A pattern consists of one or more character literals, operators, or constructs.

Constructs for Defining Regular Expressions

There are various categories of characters, operators, and constructs that lets you to define regular expression.

- Character escapes
- Character classes
- Anchors
- Grouping constructs
- Quantifiers
- Backreference constructs
- Alternation constructs
- Substitutions
- Miscellaneous constructs The Regex
- Class

The Regex class is used for representing a regular expression. It has the following commonly used methods:

- 1 .public bool IsMatch(string input
- 2 .public bool IsMatch(string input, int startat


```
3 .public static bool IsMatch(string input, string pattern)
4 .public MatchCollection Matches(string input)
5 .public string Replace(string input, string replacement)
6 .public string[] Split(string input)
```

Collections

Collection classes are specialized classes for data storage and retrieval. These classes provide support for stacks, queues, lists, and hash tables. Most collection classes implement the same interfaces. Collection classes serve various purposes, such as allocating memory dynamically to elements and accessing a list of items on the basis of an index etc. These classes create collections of objects of the Object class, which is the base class for all data types in C#.

Various Collection Classes and Their Usage

ArrayList : It represents ordered collection of an object that can be indexed individually. It is basically an alternative to an array. However, unlike array you can add and remove items from a list at a specified position using an index and the array resizes itself automatically. It also allows dynamic memory allocation, adding, searching and sorting items in the list.

Hashtable It uses a key to access the elements in the collection. A hash table is used when you need to access elements by using key, and you can identify a useful key value. Each item in the hash table has a key/value pair. The key is used to access the items in the collection.

SortedList : It uses a key as well as an index to access the items in a list. A sorted list is a combination of an array and a hash table. It contains a list of items that can be accessed using a key or an index. If you access items using an index, it is an ArrayList, and if you access items using a key, it is a Hashtable. The collection of items is always sorted by the key value.

Stack : It represents a last-in, first out collection of object. It is used when you need a last-in, first-out access of items. When you add an item in the list, it is called pushing the item and when you remove it, it is called popping the item.

Queue : It represents a first-in, first out collection of object. It is used when you need a first-in, first-out access of items. When you add an item in the list, it is called enqueue and when you remove an item, it is called deque.

BitArray : It represents an array of the binary representation using the values 1 and 0. It is used when you need to store the bits but do not know the number of bits in advance. You can access items from the BitArray collection by using an integer index, which starts from zero.

UNIT-III

GUI Applications Development, Windows forms and controls , Creating Menus ,Toolbars, Image list , Tree View, List view Data Base Programming , ADO.NET Introduction , Data Providers in .NET , Connection, Command , Data reader , DataAdapter , Data Set, Command Builder , Data Relations , ADO.NET and XML

GUI Applications Development:

GUI in .NET stands for Graphical User Interface, which refers to the visual elements and controls used to build software applications using the .NET framework. The .NET framework provides a rich set of graphical user interface components, such as buttons, menus, text boxes, and dialog boxes, that developers can use to create Windows applications. Developers can use several programming languages such as C#, Visual Basic .NET, and F# to create GUI-based applications using .NET. The GUI is typically created using a visual designer tool, such as Visual Studio, which allows developers to drag and drop GUI components onto a form and configure their properties using a graphical interface. The .NET framework provides a rich set of libraries and APIs for building GUI-based applications, including Windows Forms, WPF (Windows Presentation Foundation), and ASP.NET (Active Server Pages .NET) for web-based applications. These frameworks provide a wide range of features and functionality to create responsive and visually appealing user interfaces for desktop and web applications.

The design elements of GUI in .NET refer to the visual components and controls that can be used to create a user interface for a software application. Some of the key design elements of GUI in .NET include:

Windows Forms: Windows Forms is a graphical user interface framework that provides a set of controls for building Windows-based applications. Windows Forms controls include buttons, labels, text boxes, menus, and many others.

WPF (Windows Presentation Foundation): WPF is a graphical user interface framework that provides a rich set of controls for building Windows applications. It includes advanced controls such as 3D graphics, animation, and multimedia.

Dialog Boxes: Dialog boxes are windows that appear on top of an application's main window to prompt the user for input or to display information. Dialog boxes can be used to perform tasks such as file selection, message display, and error reporting.

Toolbars and Menus: Toolbars and menus are graphical elements that provide access to application commands and functionality. Toolbars typically contain buttons for frequently used commands, while menus provide access to a broader range of commands organized into hierarchical menus.

Status Bar: A status bar is a horizontal bar at the bottom of an application window that displays information about the current status of the application or the task being performed. Status bars can be used to display information such as progress indicators, status messages, and help text.

Tree View and List View: Tree view and list view controls provide a hierarchical or flat view of data, respectively. They can be used to display and navigate through large amounts of data in a structured way.

Tab Control: A tab control is a graphical element that allows the user to switch between different views or modes of an application. Tab controls are typically used to organize related functionality or data into separate tabs.

These design elements can be combined and customized to create a visually appealing and functional GUI for a .NET application.

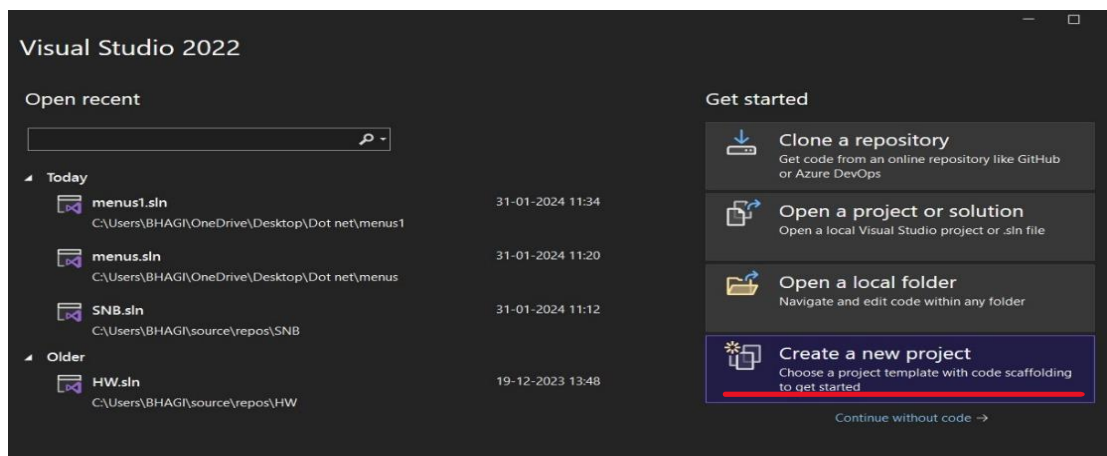
Windows forms and controls:

- Windows Forms is a graphical user interface (GUI) framework for developing desktop applications in the Microsoft .NET framework using the Visual Basic .NET (VB.NET) programming language. Windows Forms provide a set of classes, controls, and components that allow developers to create visually appealing and responsive user interfaces.
- Controls are the building blocks of Windows Forms. They are UI elements such as buttons, labels, text boxes, drop-down lists, and more. Controls can be added to a form and then manipulated programmatically to create a functional application.
- In VB.NET, controls are represented by classes that inherit from the Control class. Controls can be customized by setting various properties such as color, font, and size. They can also be programmed to respond to user events such as mouse clicks, key presses, and form resizing.
- Developers can use the drag-and-drop interface of the Visual Studio Integrated Development Environment (IDE) to create Windows Forms applications quickly. The IDE provides a toolbox with all the controls and components needed for building Windows Forms applications. Developers can simply drag and drop controls onto the form, and then use the Properties window to set their properties.
- In summary, Windows Forms and controls in VB.NET provide a powerful platform for building desktop applications with a rich user interface that can respond to user events and interact with data.

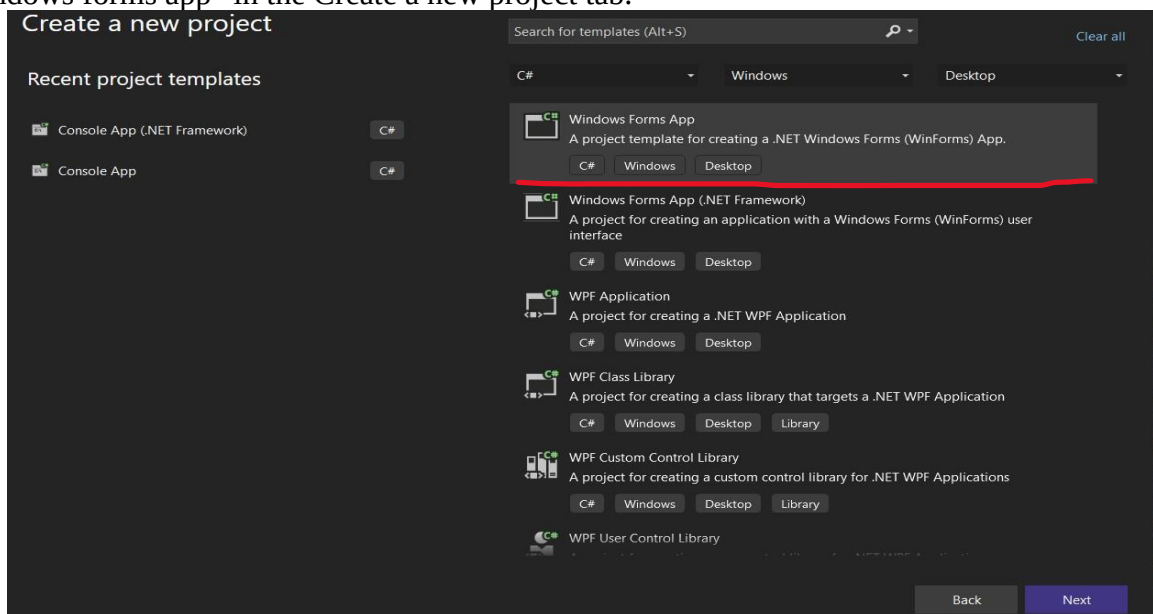
Creating Menus:

To create a menu in a Windows Forms application using Visual Studio, follow these steps:

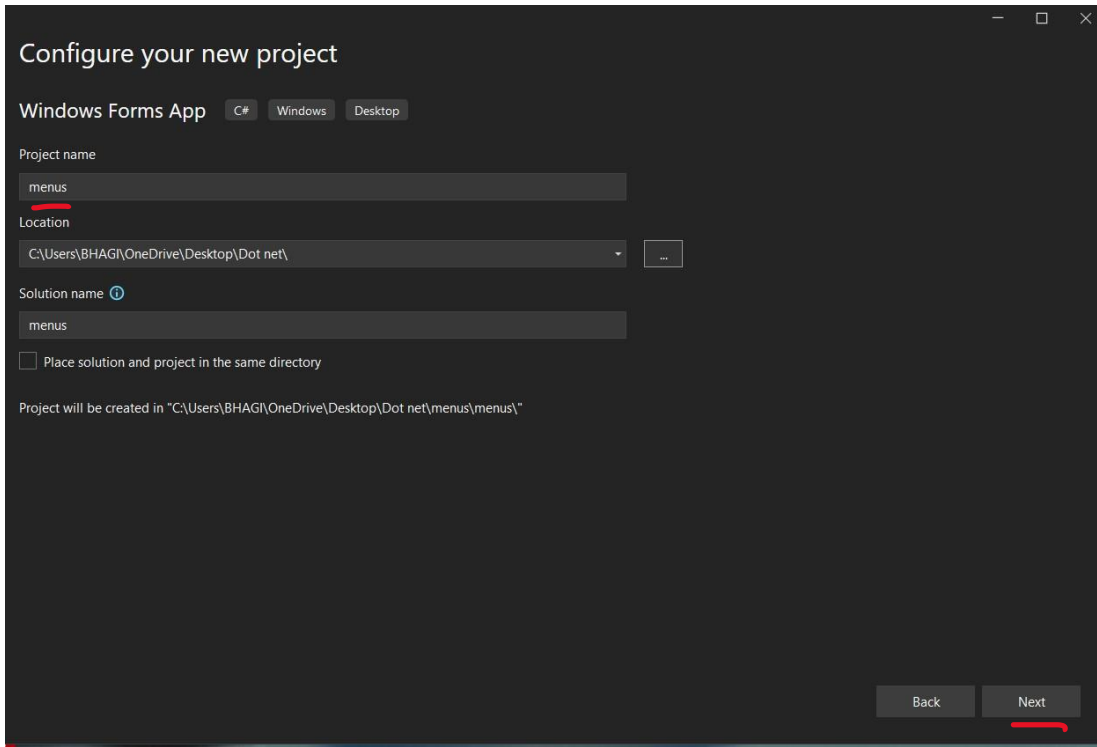
1. Open Visual Studio 2022 and “create a new project”.



2. Select “Windows forms app” in the Create a new project tab.

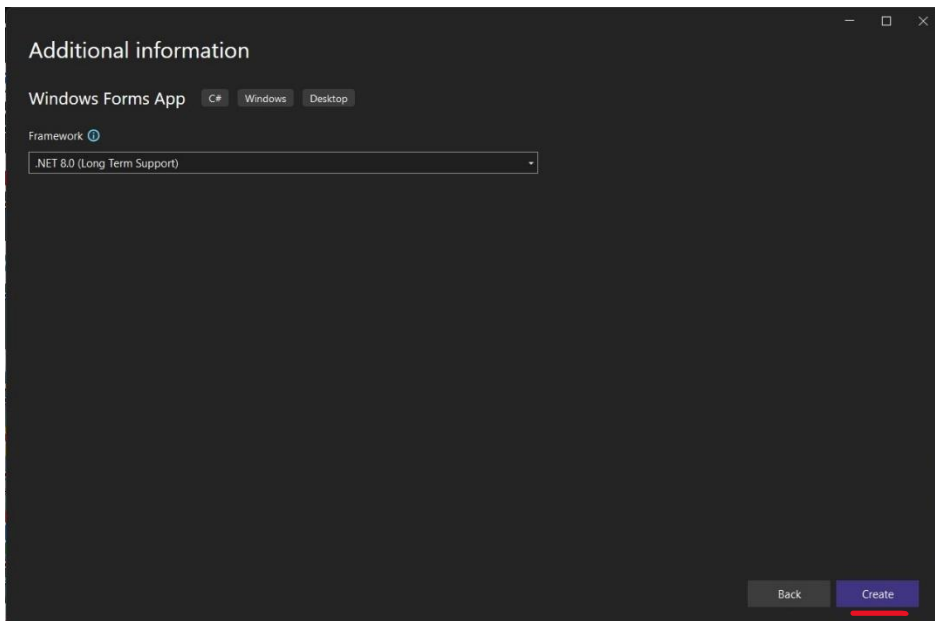


3. Give the “project name” Ex: menus. And then click on “Next” button



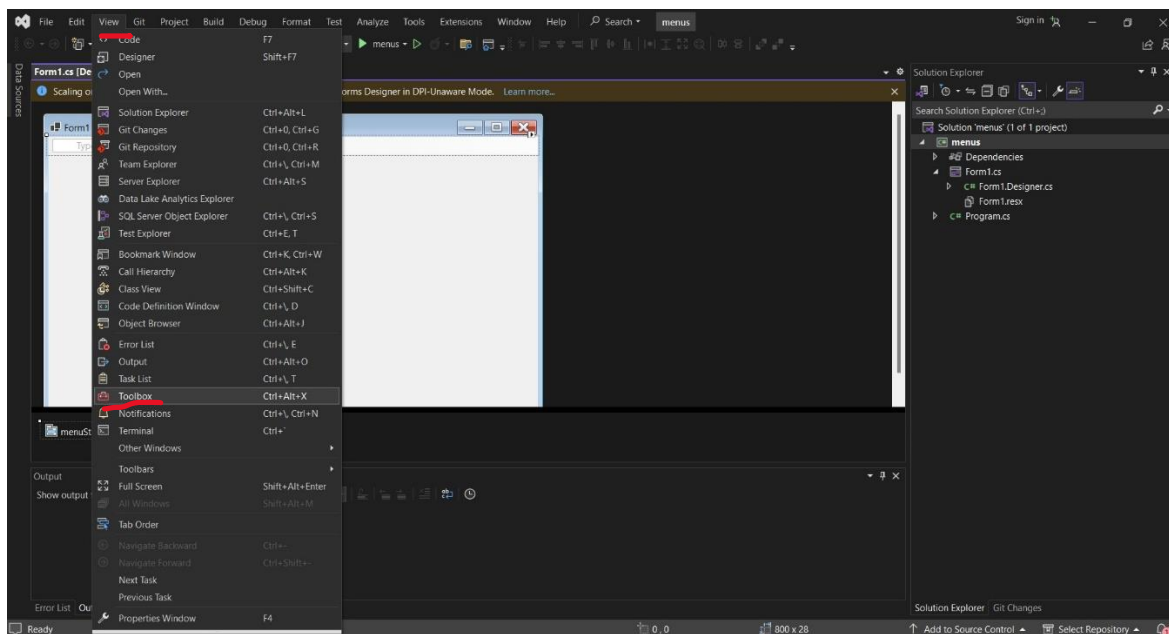
The screenshot shows the 'Configure your new project' window. At the top, it says 'Windows Forms App' with tabs for 'C#', 'Windows', and 'Desktop'. The 'Project name' field contains 'menus'. The 'Location' field shows the path 'C:\Users\BHAGI\OneDrive\Desktop\Dot net\'. The 'Solution name' field also contains 'menus'. There is an unchecked checkbox for 'Place solution and project in the same directory'. At the bottom, it states 'Project will be created in "C:\Users\BHAGI\OneDrive\Desktop\Dot net\menus\menus\"'. The 'Next' button is highlighted with a red underline.

4. Click on “Create” button in the Additional information tab.

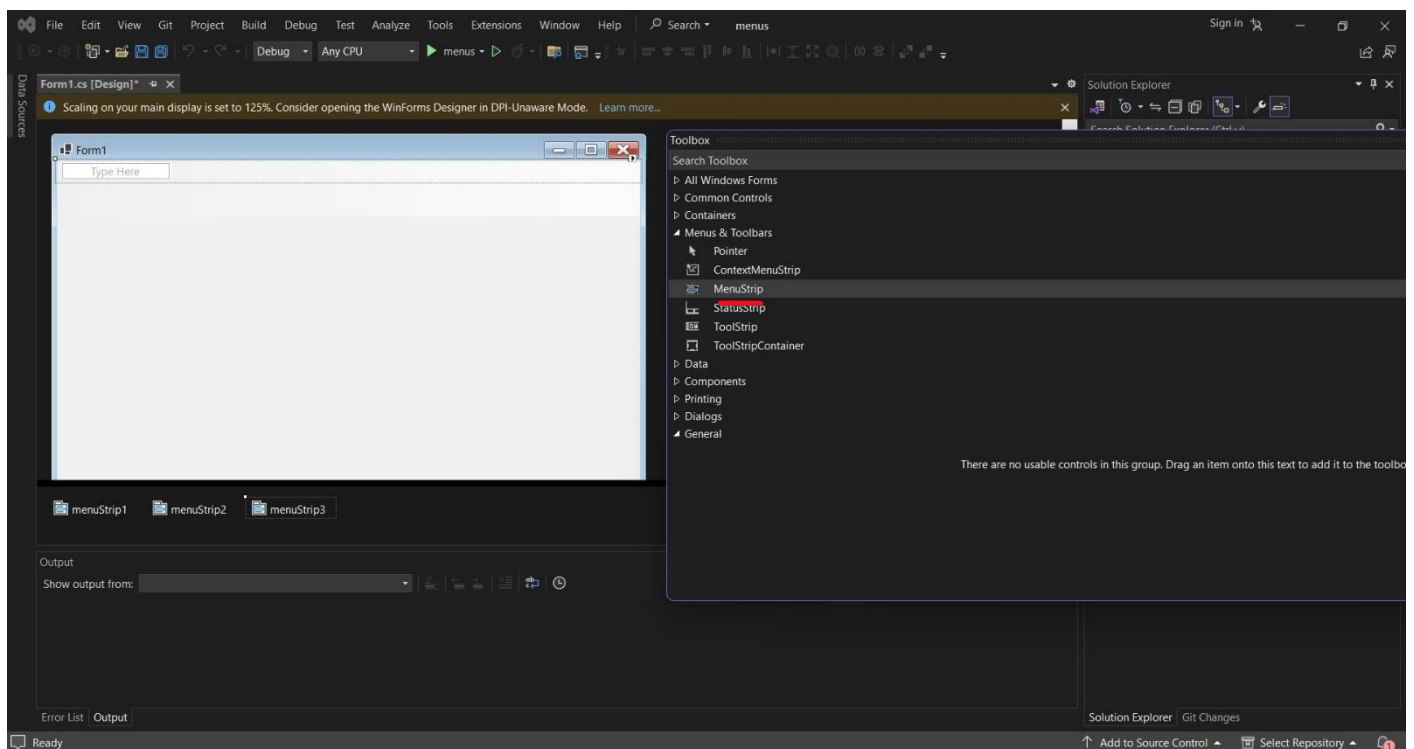


The screenshot shows the 'Additional information' window. It has the same 'Windows Forms App' header with 'C#', 'Windows', and 'Desktop' tabs. The 'Framework' dropdown menu is set to '.NET 8.0 (Long Term Support)'. At the bottom, the 'Create' button is highlighted with a red underline.

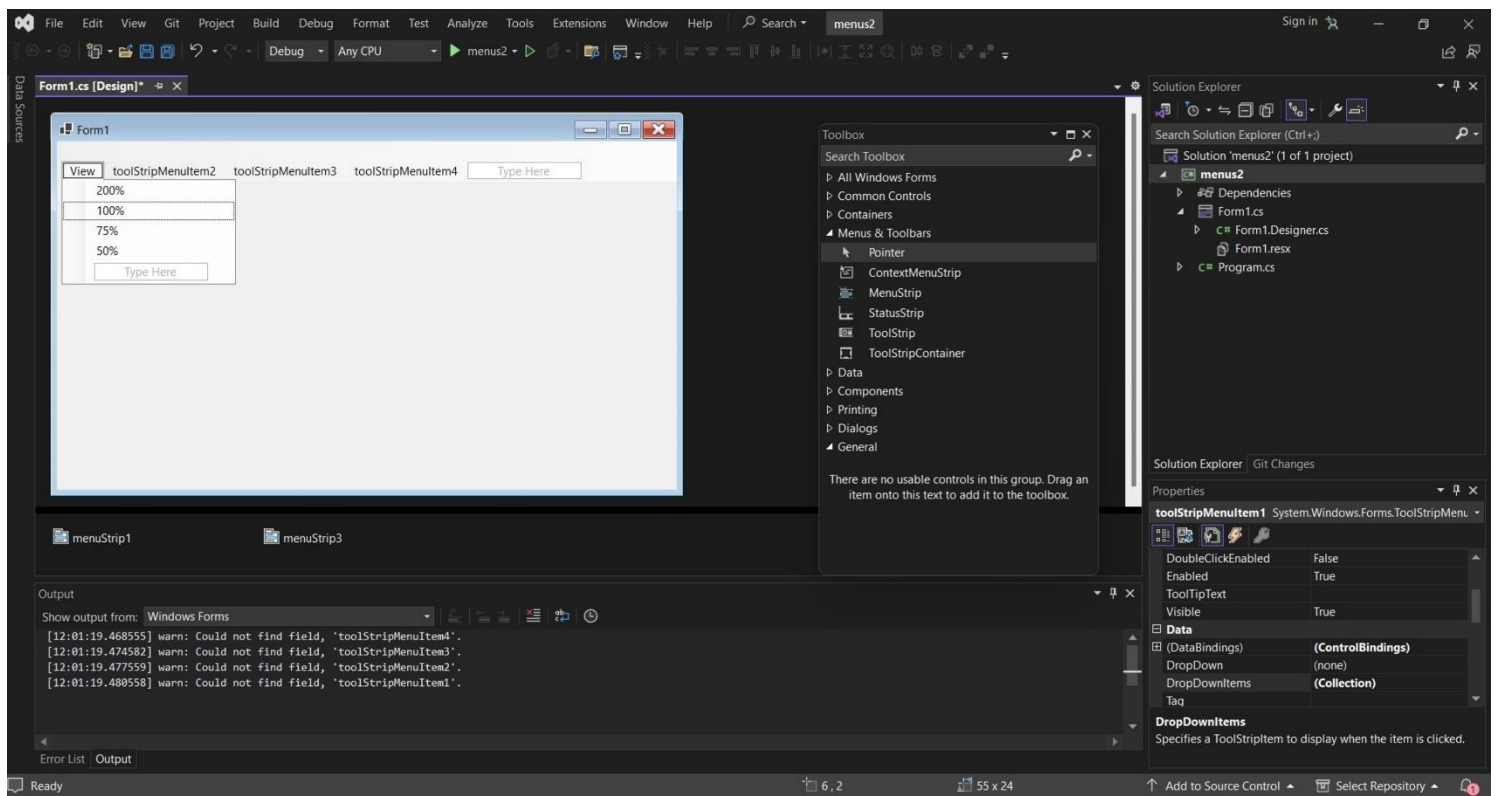
5. Form1 will open in the visual studio application. Click on “View” and From the Toolbox.



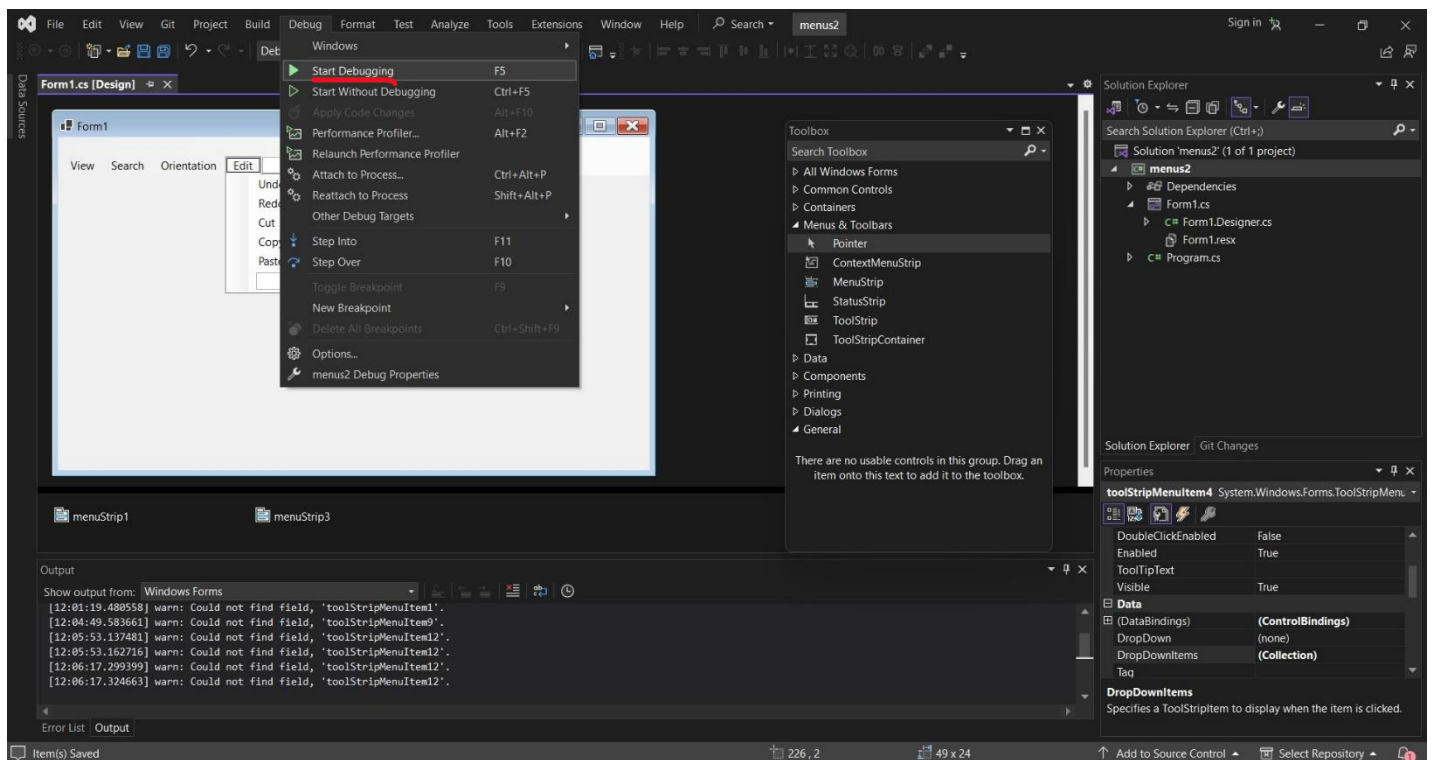
6. Drag a MenuStrip control onto the form.



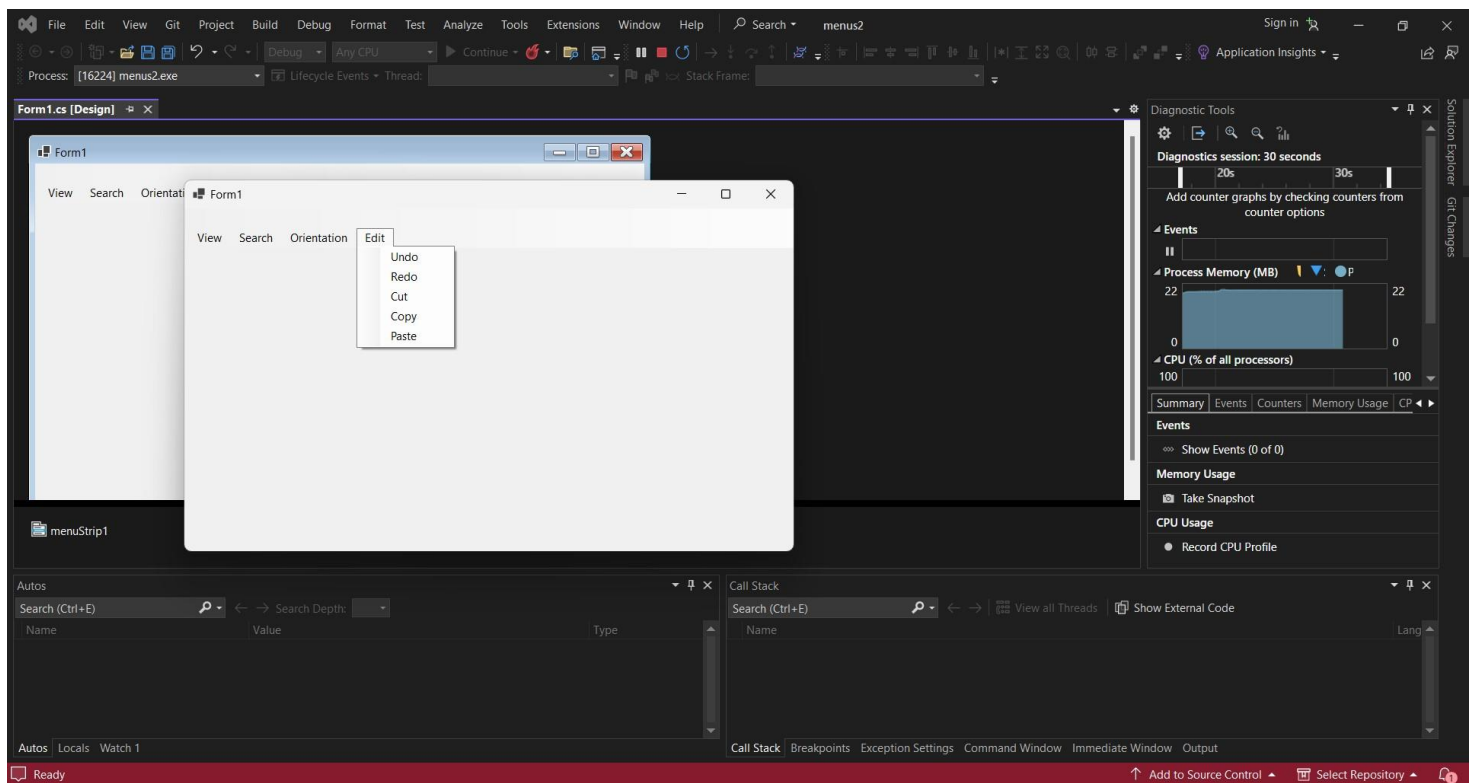
7. Click on the MenuStrip control to select it, and then click the "Edit Items" button. Alternatively, you can double-click the MenuStrip control to open the editor.



10.To run the program click on “Debug” and again click on “start debugging”.



11.The final output is:



Here is an example of creating a simple menu with two items:

- Drag a MenuStrip control onto the form.
- Select the MenuStrip control and click the "Edit Items" button.
- Click the "Add" button and select "ToolStripMenuItem" to add a new menu item.
- Set the "Text" property of the menu item to "File".
- Repeat steps 3 and 4 to add another menu item with the text "Exit".
- Double-click the "Exit" menu item to create a new event handler method.

Create a Tool Bar:

To create a toolbar in a Windows Forms application, you can use the ToolStrip control. Here are the steps to create a basic toolbar:

1. Open Visual Studio and "create a new project".
2. Select "Windows forms app" in the Create a new project tab.
3. Give the "project name" Ex:Toolbar. And then click on "Next" button
4. Click on "Create" button in the Additional information tab.
5. Form1 will open in the visual studio application. Click on "View" and From the Toolbox.
6. Drag a ToolStrip control onto the form.
7. By default, the ToolStrip will contain a few standard buttons. You can customize the buttons by selecting the ToolStrip and opening the Properties window (press F4).
8. In the Properties window, you can add new buttons by clicking the "..." button next to the "Items" property. This will open the ToolStripItems Collection Editor.

9. In the Collection Editor, you can add, remove, and modify the buttons on the toolbar. You can also change the order of the buttons by dragging them up or down in the list.

10. Once you have added all the buttons you need, you can customize their appearance and behavior by selecting them in the designer and changing their properties in the Properties window.

11. To run the program click on “Debug” and again click on “start debugging”.

12. Final output is:

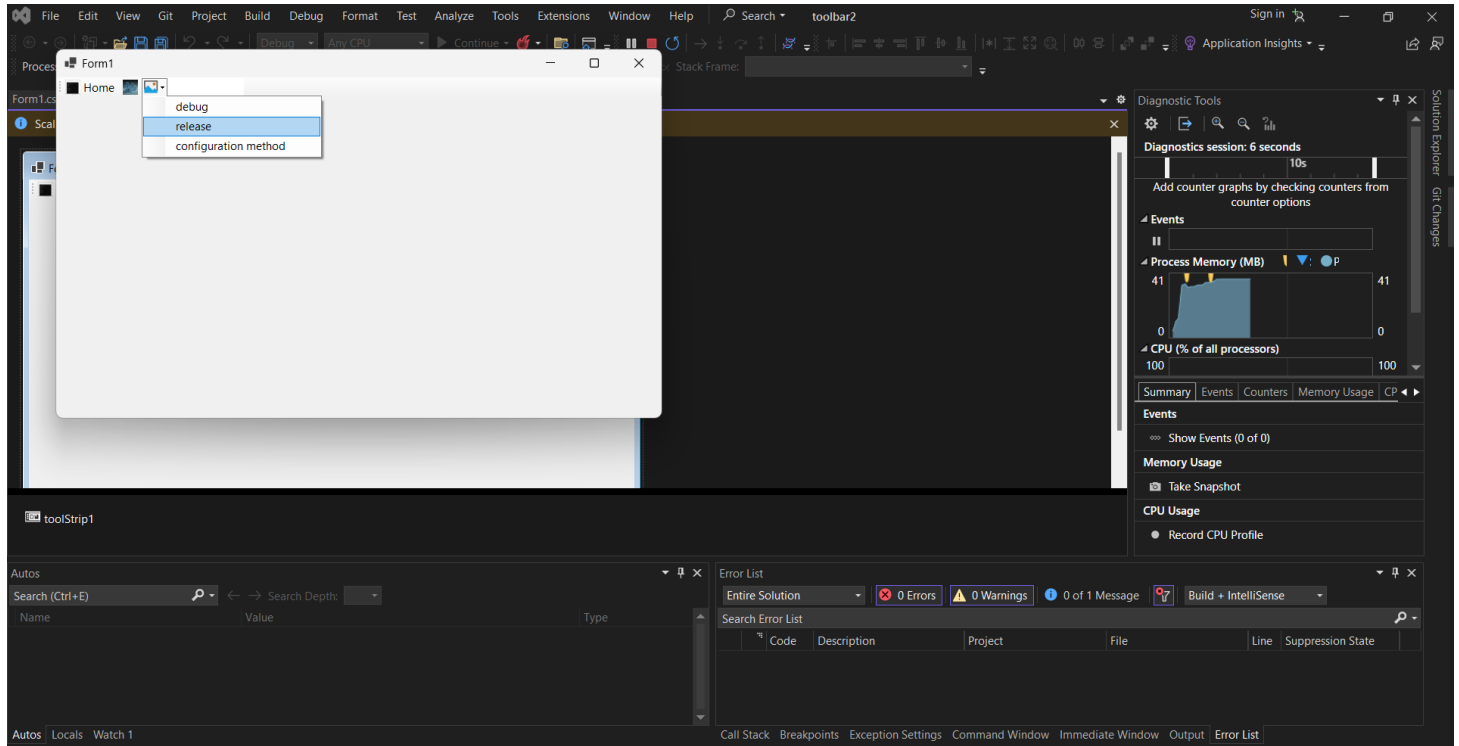


Image list:

In a Windows Forms application, you can use an ImageList control to store and manage a collection of images that can be used by other controls such as a ListView, aTreeView, or a ToolStrip. Here are the steps to create an ImageList control:

1. Open Visual Studio and “create a new project”.

2. Select “ Windows forms app” in the Create a new project tab.

3. Give the “project name” Ex:Imagelist. And then click on “Next” button

4. Click on “Create” button in the Additional information tab.

5. Form1 will open in the visual studio application. Click on “View” and From the Toolbox.

6. Drag a ImageList control onto the form.

7. In the Properties window (press F4), you can configure the ImageList control by setting the properties

such as ColorDepth, ImageSize, and TransparentColor.

8.To add images to the ImageList, click the "..." button next to the "Images"property in the Properties window. This will open the ImageCollectionEditor.

9.In the ImageCollectionEditor, you can add, remove, and modify the images inthe ImageList. You can also import images from a file by clicking the "Import" button.

10.Once you have added all the images you need, you can use them in other

11.controls that support the ImageList control. For example, you can set the ImageListproperty of a ListView control to the ImageList control you just created, and then

12.set the ImageIndex property of each item in the ListView to the index of the imageyou want to display.

13.To run the program click on “Debug” and again click on “start debugging”.

Example of code to set the ImageList property of a ListView control:

// Create an ImageList control

```
ImageList imageList1 = new ImageList(); imageList1.Images.Add(Image.FromFile("image1.png"));
```

```
imageList1.Images.Add(Image.FromFile("image2.png"));
```

// Set the ImageList property of the ListView control

```
myListView.SmallImageList = imageList1;
```

// Set the ImageIndex property of each item in the ListView

```
myListView.Items[0].ImageIndex = 0;
```

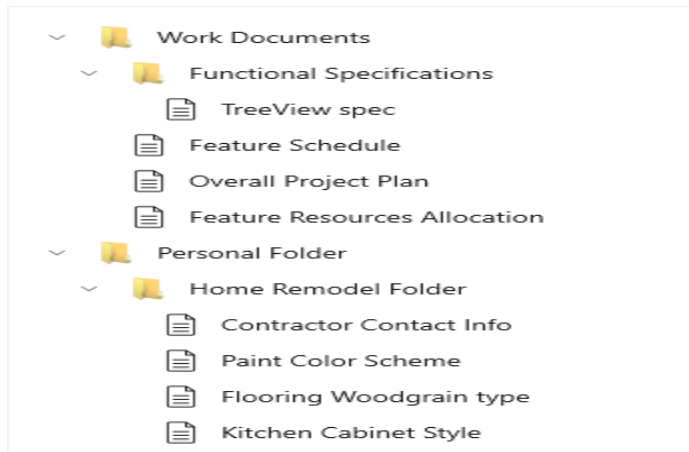
```
myListView.Items[1].ImageIndex = 1;
```

Tree view:

The tree view control enables a hierarchical list with expanding and collapsing nodes that contain nested items. It can be used to illustrate a folder structure or nested relationships in your UI.

The tree view uses a combination of indentation and icons to represent the nested relationship between parent nodes and child nodes. Collapsed nodes use a chevron pointing to the right, and expanded nodes use a chevron pointing down.

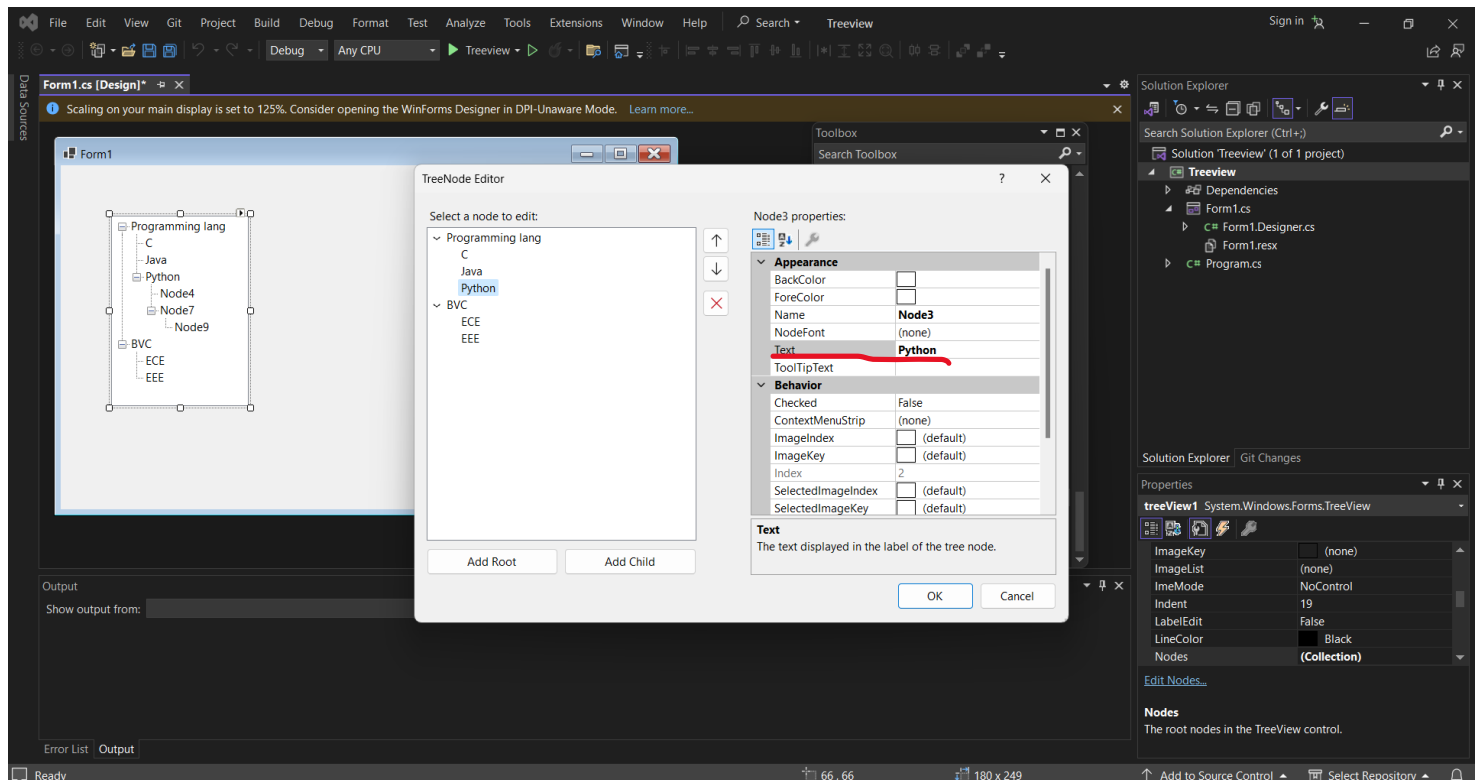
You can include an icon in the tree view item data template to represent nodes. For example, if you show a file system hierarchy, you could use folder icons for the parent notes and file icons for the leaf nodes.



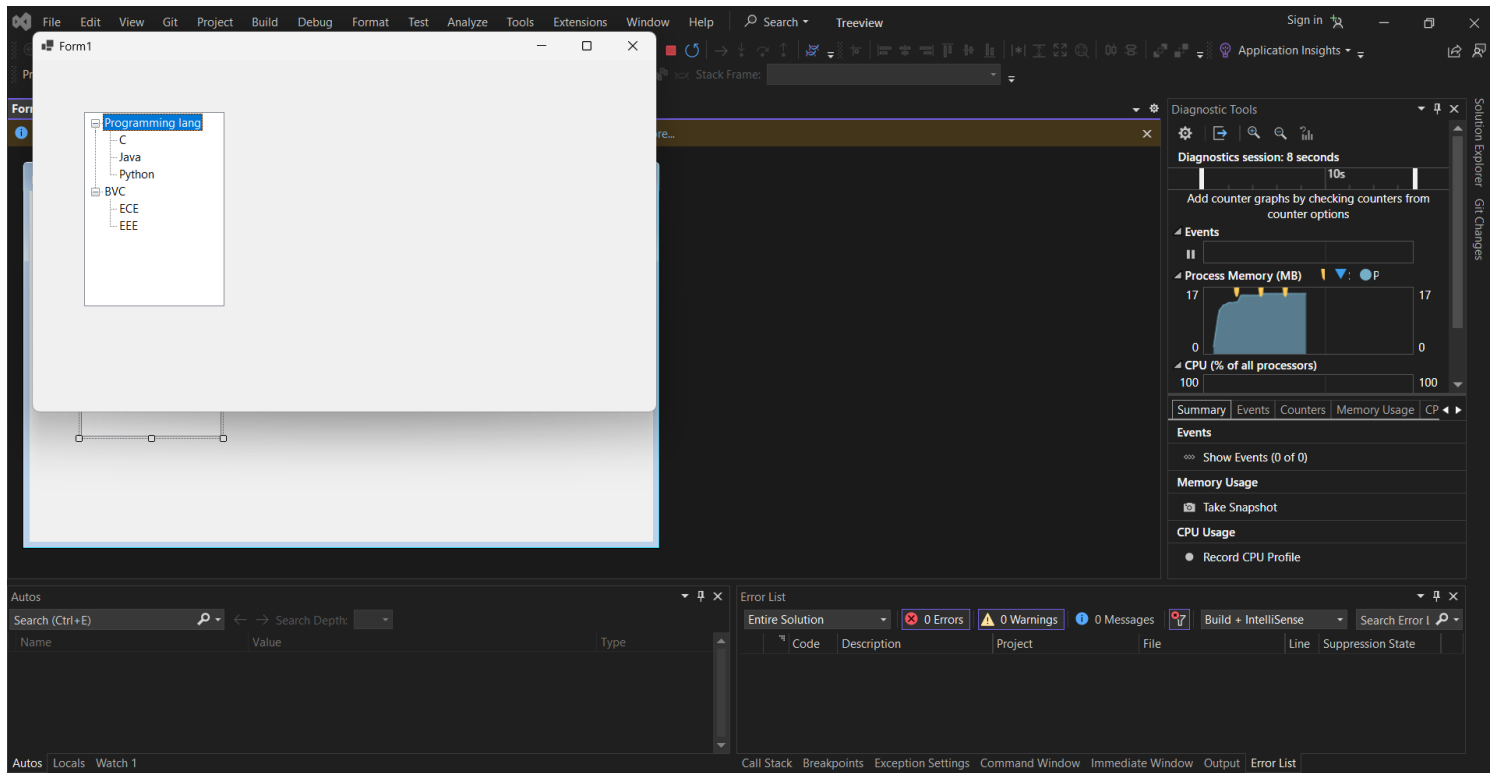
Creating tree view:

Steps to create tree view:

1. Open Visual Studio and “create a new project”.
2. Select “ Windows forms app” in the Create a new project tab.
3. Give the “project name” Ex:TreeView. And then click on “Next” button
4. Click on “Create” button in the Additional information tab.
5. Form1 will open in the visual studio application. Click on “View” and From the Toolbox.
6. Drag a TreeView control onto the form.
7. Right click on the TreeView box and click on edit items.
8. Click on “Add Root” button to add parent node and click on “Add Child” to add child node.
9. Click on next to “Text” to give name to parent and child roots.



10. To run the program click on “Debug” and again click on “start debugging”.
11. Final output is:



Here's an example of a simple tree view declared in XAML. You typically add the nodes in code, but we show the XAML hierarchy here because it can be helpful for visualizing how the hierarchy of nodes is created.

XAMLCopy

```
<muxc:TreeView>
  <muxc:TreeView.RootNodes>
    <muxc:TreeViewNode
      Content="Programming
      LAng" IsExpanded="True">
      <muxc:TreeViewNode.Children>
        <muxc:TreeViewNode Content="C" />
        <muxc:TreeViewNode Content="Java" />
        <muxc:TreeViewNode Content="Python" />
      </muxc:TreeViewNode.Children>
    </muxc:TreeViewNode>
  </muxc:TreeView.RootNodes>
</muxc:TreeView>
```

List View

The ListView control displays data stacked vertically in a single column. ListView works better for items that have text as a focal point, and for collections that are meant to be read top to bottom (for example, alphabetically ordered). A few common use cases for ListView include lists of messages and search results. If you need to display collections in multiple columns or in a table-like format, you should *not* use ListView.

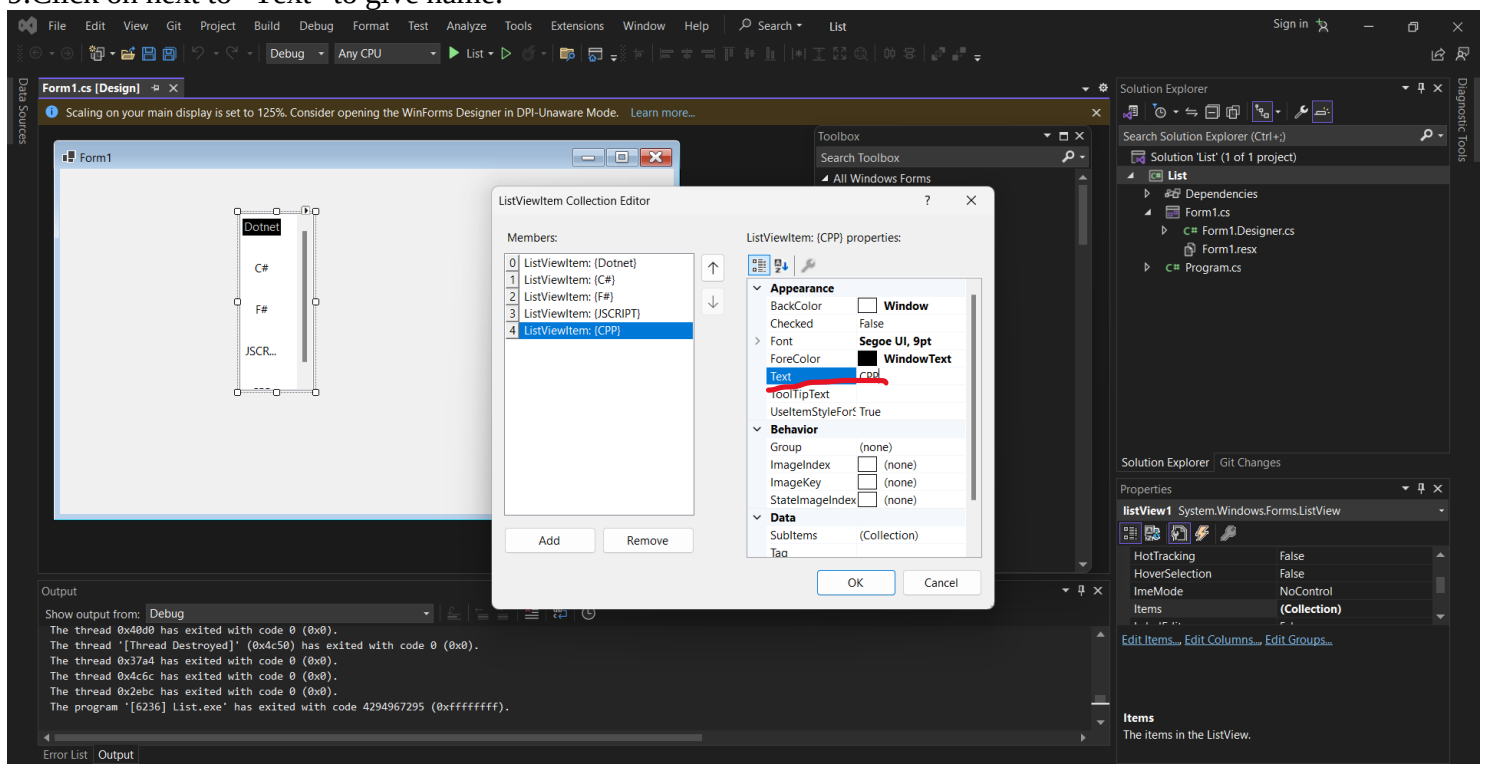
A

- Forging Specialties**
- Fausto Agramonte**
 Marriott Hotels Resorts Suites
- Ressie Auffrey**
 Faw
- Rhea Aredondo**
 Double B Foods Inc
- Alyce Arias**
 Fairbanks Scales

B

- James Buchanan**
 Benton

- 1.Open Visual Studio and “create a new project”.
- 2.Select “ Windows forms app” in the Create a new project tab.
- 3.Give the “project name” Ex:ListView. And then click on “Next” button
- 4.Click on “Create” button in the Additional information tab.
- 5.Form1 will open in the visual studio application. Click on “View” and From the Toolbox.
- 6.Drag a ListView control onto the form.
- 7.Right click on the ListView box and click on edit items then ListViewCollection Editor will open.
- 8.Click on “Add” button to add the members in the ListViewCollection Editor.
- 9.Click on next to “Text” to give name.



- 10.To run the program click on “Debug” and again click on “start debugging”.
- 11.Final output is:

FileEditViewGitProjectBuildDebugFormatTestAnalyzeToolsExtensionsWindowHelpSearchList

Process: [6236] List.exeLifecycle EventsThread:Stack Frame:

Form1.cs [Design]Form1Form1DotnetC#F#JSCR...C#

AutosSearch (Ctrl+E)Search Depth:NameValueType

Diagnostic ToolsDiagnostics session: 9 secondsAdd counter graphs by checking counters from counter optionsEventsProcess Memory (MB)19CPU (% of all processors)100SummaryEventsCountersMemory UsageCPU UsageShow Events (0 of 0)Take SnapshotRecord CPU Profile

Error ListEntire Solution0 Errors0 Warnings0 MessagesBuild + IntelliSenseSearch Error ICodeDescriptionProjectFileLineSuppression State

ADO.NET Introduction:

ADO(ActiveX Data Object) .NET is a module of .Net Framework which is used to establish connection between application and data sources. Data sources can be such as SQL Server and XML. ADO.NET consists of classes that can be used to connect, retrieve, insert and delete data.

All the ADO.NET classes are located into **System.Data.dll** and integrated with XML classes located into **System.Xml.dll**.

ADO.NET has two main components that are used for accessing and manipulating data are the .NET Framework data provider and the DataSet.

These are the components that are designed for data manipulation and fast access to data. It provides various objects such as **Connection, Command, DataReader and DataAdapter** that are used to perform database operations.

ADO.NET Framework Data Providers:

Data provider is used to connect to the database, execute commands and retrieve the record. It is lightweight component with better performance. It also allows us to place the data into DataSet to use it further in our application.

The .NET Framework provides the following data providers that we can use in our application.

.NET Framework data provider	Description
.NET Framework Data Provider for SQL Server	It provides data access for Microsoft SQL Server. It requires the System.Data.SqlClient namespace.
.NET Framework Data Provider for OLE DB	It is used to connect with OLE DB. It requires the System.Data.OleDb namespace.
.NET Framework Data Provider for ODBC	It is used to connect to data sources by using ODBC. It requires the System.Data.Odbc namespace.
.NET Framework Data Provider for Oracle	It is used for Oracle data sources. It uses the System.Data.OracleClient namespace.
EntityClient Provider	It provides data access for Entity Data Model applications. It requires the System.Data.EntityClient namespace.
.NET Framework Data Provider for SQL Server Compact 4.0.	It provides data access for Microsoft SQL Server Compact 4.0. It requires the System.Data.SqlServerCe namespace.

ADO.NET Connection

It is used to establish an open connection to the SQL Server database. It is a sealed class so that cannot be inherited. SqlConnection class uses SqlDataAdapter and SqlCommand classes together to increase performance when connecting to a Microsoft SQL Server database.

Connection does not close explicitly even it goes out of scope. Therefore, you must explicitly close the connection by calling Close() method.

SqlConnection Signature

public sealed class SqlConnection : System.Data.Common.DbConnection, ICloneable, IDisposable

SqlConnection Constructors

Constructors	Description
SqlConnection()	It is used to initializes a new instance of the SqlConnection class.

SqlConnection(String)0	It is used to initialize a new instance of the SqlConnection class and takes connection string as an argument.
SqlConnection(String, SqlConnectionCredential)	It is used to initialize a new instance of the SqlConnection class that takes two parameters. First is connection string and second is sql credentials.

SqlConnection Methods

Method	Description
BeginTransaction()	It is used to start a database transaction.
ChangeDatabase(String)	It is used to change the current database for an open SqlConnection.
ChangePassword(String, String)	It changes the SQL Server password for the user indicated in the connection string.
Close()	It is used to close the connection to the database.
CreateCommand()	It enlists in the specified transaction as a distributed transaction.
GetSchema()	It returns schema information for the data source of this SqlConnection.
Open()	It is used to open a database connection.
ResetStatistics()	It resets all values if statistics gathering is enabled.

SqlConnection Example

```
using (SqlConnection connection = new SqlConnection(connectionString))
{
    connection.Open();
}
```

ADO.NET Command

This class is used to store and execute SQL statement for SQL Server database. It is a sealed class so that cannot be inherited.

SqlCommand Signature

1. public sealed class SqlCommand : System.Data.Common.DbCommand, ICloneable, IDisposable

Constructors

This class provides the following constructors.

Constructor	Description
SqlCommand()	It is used to initialize a new instance of the SqlCommand class.
SqlCommand(String)	It is used to initialize a new instance of the SqlCommand class with a string parameter.
SqlCommand(String, SqlConnection)	It is used to initialize a new instance of the SqlCommand class. It takes two parameters, first is query string and second is connection string.
SqlCommand(String, SqlConnection, SqlTransaction)	It is used to initialize a new instance of the SqlCommand class. It takes three parameters query, connection and transaction string respectively.
SqlCommand(String, SqlConnection, SqlTransaction, SqlCommandColumnEncryptionSetting)	It Initializes a new instance of the SqlCommand class with specified command text, connection, transaction, and encryption setting.

Methods

Method	Description
BeginExecuteNonQuery()	It is used to Initiate the asynchronous execution of the SQL statement described by this SqlCommand.
Cancel()	It tries to cancel the execution of a SqlCommand.
Clone()	It creates a new SqlCommand object that is a copy of the current instance.
CreateParameter()	It creates a new instance of a SqlParameter object.
ExecuteReader()	It is used to send the CommandText to the Connection and builds a SqlDataReader.
ExecuteXmlReader()	It is used to send the CommandText to the Connection and builds an XmlReader object.
ExecuteScalar()	It executes the query and returns the first column of the first row in the result set. Additional columns or rows are ignored.
Prepare()	It is used to create a prepared version of the command by using the instance of SQL Server.
ResetCommandTimeout()	It is used to reset the CommandTimeout property to its default value.

ADO.NET Data Reader

This class is used to read data from SQL Server database. It reads data in forward-only stream of rows from a SQL Server database. it is sealed class so that cannot be inherited. It inherits DbDataReader class and implements IDisposable interface.

SqlDataReader Signature

```
public class SqlDataReader : System.Data.Common.DbDataReader, IDisposable
```

SqlDataReader Properties

Property	Description
Connection	It is used to get the SqlConnection associated with the SqlDataReader.
Depth	It is used to get a value that indicates the depth of nesting for the current row.
FieldCount	It is used to get the number of columns in the current row.
HasRows	It is used to get a value that indicates whether the SqlDataReader contains one or more rows.
IsClosed	It is used to retrieve a boolean value that indicates whether the specified SqlDataReader instance has been closed.
Item[String]	It is used to get the value of the specified column in its native format given the column name.
Item[Int32]	It is used to get the value of the specified column in its native format given the column ordinal.
RecordsAffected	It is used to get the number of rows changed, inserted or deleted by execution of the Transact-SQL statement.
VisibleFieldCount	It is used to get the number of fields in the SqlDataReader that are not hidden.

Methods

Method	Description
Close()	It is used to closes the SqlDataReader object.
GetBoolean(Int32)	It is used to get the value of the specified column as a Boolean.
GetByte(Int32)	It is used to get the value of the specified column as a byte.
GetChar(Int32)	It is used to get the value of the specified column as a single character.
GetDateTime(Int32)	It is used to get the value of the specified column as a DateTime object.
GetDecimal(Int32)	It is used to get the value of the specified column as a Decimal object.

GetDouble(Int32)	It is used to get the value of the specified column as a double-precision floating point number.
GetFloat(Int32)	It is used to get the value of the specified column as a single-precision floating point number.
GetName(Int32)	It is used to get the name of the specified column.
GetSchemaTable()	It is used to get a DataTable that describes the column metadata of the SqlDataReader.
GetValue(Int32)	It is used to get the value of the specified column in its native format.
GetValues(Object[])	It is used to populate an array of objects with the column values of the current row.
NextResult()	It is used to get the next result, when reading the results of SQL statements.
Read()	It is used to read record from the SQL Server database.

To create a SqlDataReader instance, we must call the ExecuteReader method of the SqlCommand object.

ADO.NET Data Adapter

The DataAdapter works as a bridge between a DataSet and a data source to retrieve data. DataAdapter is a class that represents a set of SQL commands and a database connection. It can be used to fill the DataSet and update the data source.

Data Adapter Class Signature

```
public class DataAdapter : System.ComponentModel.Component, System.Data.IDataAdapter
```

Data Adapter Constructors

Constructors	Description
DataAdapter()	It is used to initialize a new instance of a DataAdapter class.
DataAdapter(DataAdapter)	It is used to initializes a new instance of a DataAdapter class from an existing object of the same type.

Methods

Method	Description
CloneInternals()	It is used to create a copy of this instance of DataAdapter.
Dispose(Boolean)	It is used to release the unmanaged resources used by the DataAdapter.
Fill(DataSet)	It is used to add rows in the DataSet to match those in the data source.
FillSchema(DataSet, SchemaType, String, IDataReader)	It is used to add a DataTable to the specified DataSet.
GetFillParameters()	It is used to get the parameters set by the user when executing an SQL SELECT statement.
ResetFillLoadOption()	It is used to reset FillLoadOption to its default state.
ShouldSerializeAcceptChangesDuringFill()	It determines whether the AcceptChangesDuringFill property should be persisted or not.
ShouldSerializeFillLoadOption()	It determines whether the FillLoadOption property should be persisted or not.
ShouldSerializeTableMappings()	It determines whether one or more DataTableMapping objects exist or not.
Update(DataSet)	It is used to call the respective INSERT, UPDATE, or DELETE statements.

ADO.NET Data Set

It is a collection of data tables that contain the data. It is used to fetch data without interacting with a Data Source that's why, it also known as **disconnected** data access method. It is an in-memory data store that can hold more than one table at the same time. We can use DataRelation object to relate these tables. The DataSet can also be used to read and write data as XML document.

ADO.NET provides a DataSet class that can be used to create DataSet object. It contains constructors and methods to perform data related operations.

Data Set Class Signature

1. public class DataSet : System.ComponentModel.MarshalByValueComponent, System.ComponentModel.IListSource,
2. System.ComponentModel.ISupportInitializeNotification, System.Runtime.Serialization.ISerializable,
3. System.Xml.Serialization.IXmlSerializable

Data Set Constructors

Constructor	Description
DataSet()	It is used to initialize a new instance of the DataSet class.
DataSet(String)	It is used to initialize a new instance of a DataSet class with the given name.
DataSet(SerializationInfo, StreamingContext)	It is used to initialize a new instance of a DataSet class that has the given serialization information and context.
DataSet(SerializationInfo, StreamingContext, Boolean)	It is used to initialize a new instance of the DataSet class.

Data Set Properties

Properties	Description
CaseSensitive	It is used to check whether DataTable objects are case-sensitive or not.
DataSetName	It is used to get or set name of the current DataSet.
DefaultViewManager	It is used to get a custom view of the data contained in the DataSet to allow filtering and searching.
HasErrors	It is used to check whether there are errors in any of the DataTable objects within this DataSet.
IsInitialized	It is used to check whether the DataSet is initialized or not.
Locale	It is used to get or set the locale information used to compare strings within the table.
Namespace	It is used to get or set the namespace of the DataSet.
Site	It is used to get or set an ISite for the DataSet.
Tables	It is used to get the collection of tables contained in the DataSet.

Data Set Methods

The following table contains some commonly used methods of DataSet.

Method	Description
BeginInit()	It is used to begin the initialization of a DataSet that is used on a form.
Clear()	It is used to clear the DataSet of any data by removing all rows in all tables.
Clone()	It is used to copy the structure of the DataSet.
Copy()	It is used to copy both the structure and data for this DataSet.
CreateDataReader(DataTable[])	It returns a DataAdapter with one result set per DataTable.
CreateDataReader()	It returns a DataAdapter with one result set per DataTable.

Command Builder:

Sometimes creating SQL statements could be lengthy job when dealing with many columns in a table. A Command Builder object reduces the burden of creating SQL statements for you. In other words, the Command Builder helps you to generate update, delete, and insert commands on a single database table for a data adapter. Similar to other objects, each data provider has a command builder class. The OleDb Command Builder, SqlCommand Builder, and Odbc Command Builder classes represent the Common Builder object in the OleDb, Sql, and ODBC data providers. These classes also work in pretty similar fashion. Once you know how to use OleDb Command Builder, you can use SqlCommand Builder and Odbc Command Builder in a similar way. I'll use OleDb Command Builder class in this example.

Creating a Command Builder Object

Creating a Commoned Buider object is pretty simply. You pass a Data Adapter as an argument of the Command Builder constructor. For example:

```
// Create a command builder object
SqlCommandBuilder builder = new SqlCommandBuilder(adapter);
```

Sql Command Builder Members

The Data Adapter property of a Common Builder represents the Data Provider attached to a Common Builder object for which automatic SQL statements are generated. The Get Delete Common, Get Update Command, and Get Insert Command methods return the delete, update, and insert commands in the form of a Command object. The Refresh Schema method refreshes the database schema.

Data relations:

To provide data integrity and consistency, we should use relationships between two tables. We can achieve this relationship by defining a primary key in one table and using a foreign key in the other table. Say a college has multiple students; the student details table stores the student details, and the result table stores all the student results details. To avoid the redundancy of data, you define the primary key of the student table as a foreign key in the results table.

Create Database And Table

```
create database COLLEGE
```

```
use COLLEGE
```

```
create table Student_details
```

```
(
RollNo int constraint pk primary key,
F_Name varchar(30),
L_Name varchar(30),
City varchar(20)
);
```

```
create table Student_Results
```

```
(
RollNumber int constraint fk foreign key references AllStudent(RollNo),
Course varchar(20),
```

```
Result varchar(4)
);
```

ADO.NET and XML:

There are two approaches to work with XML and ADO. First, you can use ADO.NET to access XML documents. Second, we can use XML and ADO.NET to access XML. Additionally, you can access a relational database using ADO.NET and XML.NET.

Reading XML using Data Set

In ADO.NET, we can access the data using the Data Set class. The Data Set class implements methods and properties to work with XML documents. The following sections discuss methods that read XML data.

The Read xml Method

Read Xml is an overloaded method; we can use it to read a data stream, Text Reader, Xml Reader, or an XML file and to store into a Data Set object, which can later be used to display the data in a tabular format. The Read Xml method has eight overloaded forms. It can read a text, string, stream, Text Reader, Xml Reader, and their combination formats. In the following example, create a new Data Set object.

In the following example, create a new Data Set object and call the Data Set. Read Xml method to load the books.xml file in a Data Set object:

```
//Create a DataSet object
DataSet ds = new DataSet();
// Fill with the data
ds.ReadXml("books.xml");
```

The Read Xml Schema method

The Read XML Schema method reads an XML schema in a Data Set object. It has four overloaded forms. We can use a Text Reader, string, stream, and Xml Reader. The following example shows how to use a file as direct input and call the Read Xml Schema method to read the file:

```
DataSet ds = new DataSet();
ds.ReadSchema(@"c:\books.xml");
```

UNIT-IV

ASP.Net, Introduction to web applications, State management techniques, Session & ViewState, Response.Redirect & Server.Response, Query string, SQL Server, RDBMS concepts, Tables and data storage, Anomalies and Normalization techniques, 1NF, 2NF, 3NF, Relations, Primary key, Foreign key, Different data types in SQL server, Create, Read, Update & Delete data in tables (SQL queries), Group by, Order by on records, Aggregation operations, Numeric operations, String operations, Datetime operations, Exception handling, User defined data and table types, Temp tables, Triggers, stored procedures, Functions.

ASP.NET

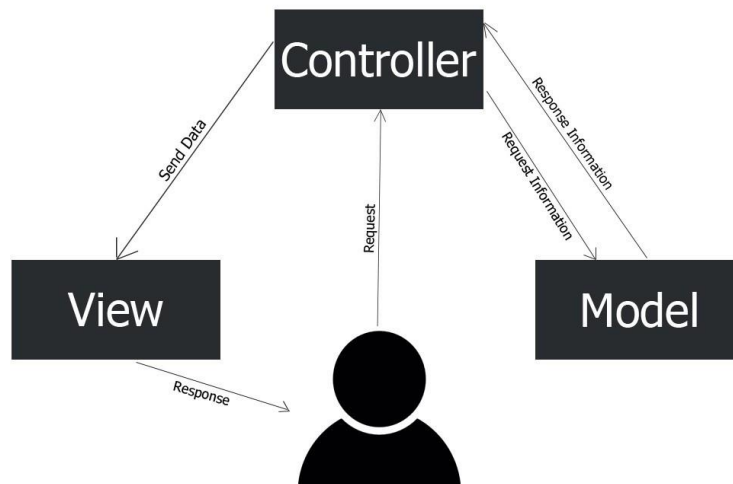
- ❖ The name stands for **Active Server Pages Network Enabled Technologies**. ASP.NET extends the .NET platform with tools and libraries.
- ❖ It was first released in January 2002 with version 1.0 of the .NET Framework. It was developed by Microsoft.
- ❖ It is an open-source, server-side web-application framework to produce dynamic web pages.
- ❖ ASP.NET pages have the extension **.aspx** and are normally written in C# (C sharp).
- ❖ ASP.NET 4.6 is the latest official version of ASP.NET.
- ❖ It is mainly used to build web pages, web applications, web sites and web services as it provides a good integration of HTML, CSS and JavaScript.
- ❖ .NET Framework is used to develop a variety of applications like mobile apps, desktop apps, microservices, web applications, games, cloud-based applications and ML based applications etc. But ASP.NET is only used to create web applications and web services. That's why we termed ASP.NET as a subset of the .NET Framework.
- ❖ ASP.NET works on top of the HTTP (Hypertext transport protocol) protocol and uses the HTTP commands and policies to set a browser-to-server bilateral communication and cooperation.
- ❖ The ASP.NET application codes can be written in any of the following languages:
 - C#
 - Visual Basic.Net
 - Jscript
 - J#
- ❖ It consists of a large number of controls such as text boxes, buttons, and labels for assembling, configuring, and manipulating code to create HTML pages.
- ❖ ASP.NET is built on the Common Language Runtime (CLR), allowing programmers to write ASP.NET code using any supported .NET language (C#, J#...etc).
- ❖ The main advantages are language independent, high performance, secure, globalization and localization and less time to execute.

Models of ASP.NET Versions:

Some of the models supported by ASP.NET are:

- ❖ **.NET Web Forms:** This is a framework for building modular web pages, UI (User Interface) for these pages is processed at the server-side.
- ❖ **.NET MVC:** Helps in building web pages using a model – view – controller design pattern.

Model-View-Controller



- ❖ **.NET Web Pages:** This is the framework that adds dynamic code and data access logic directly inside the HTML tags.
- ❖ **.NET Web API:** Introduced to build web API (Application Programming Interface) using the .NET framework.



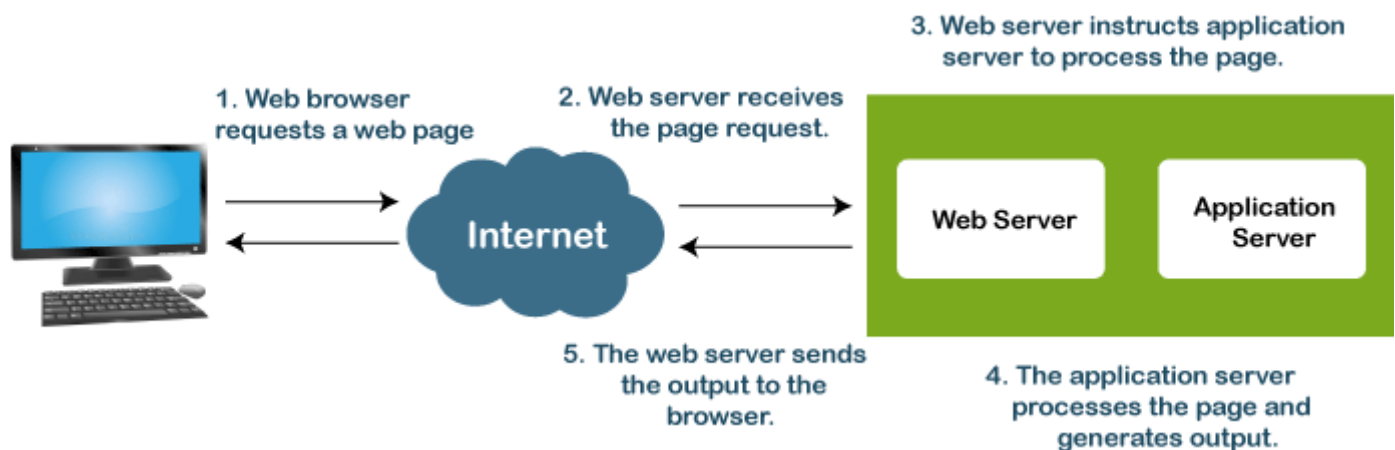
Introduction to web applications

- ❖ A web-application is an application program that is usually **stored on a remote server** and users can access it through the use of Software known as **web-browser**. Firefox, Chrome, Opera and Safari...etc are the examples of web browsers. To access web applications, we need authentication but for web sites we do not need authentication.

- ❖ Web-application does not require downloading them because the web application is a computer program that usually resides on the remote server. Any user can access it by using one of the standard web browsers such as Google Chrome, Safari, Microsoft Edge, etc., In general, a web application can contain online shops (or we can also tell them e-commerce shops), webmail's, social media platforms, etc.
- ❖ A web application usually uses a combination of the server-side scripts such as **PHP**(Personal Home Page), **ASP** (Active Server Pages) for handling the information/ data storage and retrieval of the data. Some of them also use the client-side scripts such as **HTML** (Hyper Text Markup Language), **CSS** (Cascading Style Sheets), **JavaScript** to represent the data/information in front of the users, and some of the web applications are also using both server-side and client-side at the same time.

The Flow of the Web Application

The flow of the typical web application looks like shown in below.



1. In general, a user sends a request to the web-server using web browsers such as **Google Chrome, Microsoft Edge, Firefox**, etc over the **internet**.
2. Then, the request is forwarded to the appropriate web **application server** by the **web-server**.
3. Web application server performs the requested operations/ tasks like **processing the database, querying the databases; produces** the result of the requested data.
4. The obtained result is sent to the web-server by the web application server along with the requested data/information or processed data.
5. The web server responds to the user with the requested or processed data/information and provides the result to the user's screen.

Types of web applications:

1.Static web application: A Static Web Application can be delivered directly to an end user's browser without any server-side alteration of the HTML, CSS, or JavaScript content. A static web app is one with stable content, where every user sees the exact same thing on each individual page. Wikipedia is the example of static web application.

2.Dynamic web application: A dynamic web application is a type of application that can change its content, appearance, and functionality in response to user input, system events, and information. Dynamic Web Applications can change their functionality without having to rewrite parts of their codebase. They are made up of three components: the backend component (the database), client-side components, and frontend components. These pieces work together to enable users to interact with a DWA, which can change over time just like it does in any other dynamic application. Gmail, Facebook and Instagram are the examples of dynamic web application.

3.Responsive web application: Responsive Web Design is about using HTML and CSS to automatically resize a website. Responsive Web Design is about making a website look good on all devices (desktops, tablets, and phones)

4.Single page web application: An SPA (Single-page application) is a web app implementation that loads only a single web document, and then updates the body content of that single document via JavaScript APIs. Gmail, Facebook, Google Maps, etc., all are Single Page Applications that offer an outstanding user experience in the browser with no page reloading.

5.Multi page web application: A multi-page application is a traditional app that, unlike an SPA, consists of a number of pages and reloads the entire web page every time a user forms a new request. Most apps you use on the web are multipage. E-commerce websites, Amazon, eBay...etc are the examples of multi-page web applications.

State management techniques:

ASP.NET browsers are generally stateless. Stateless means, whenever we visit a website, our browser communicates with the respective server using the HTTP or HTTPs protocol. But after that response, HTTP/HTTPs doesn't remember what website or URL we visited, or in other words we can say it doesn't hold the state of a previous website that we visited before closing our browser, that is called stateless.

Types of State Management in ASP.NET

1.Server-Side State Management in ASP.NET

- **Session:** By default, an ASP.NET session state is enabled for all ASP.NET applications. An important technique to maintain state. It is used to store **identity** and **information**. information is stored in the server using Session id. Every user has an individual session.
- **Application:** It is a server-side management state and is also known as the application-level state management. This is mainly used to **store user activity** in server memory and **application events**.
- **Cache:** This is stored on the server-side, and it is used to implement page caching and data caching. A cache is primarily used to **set expiration policies**.

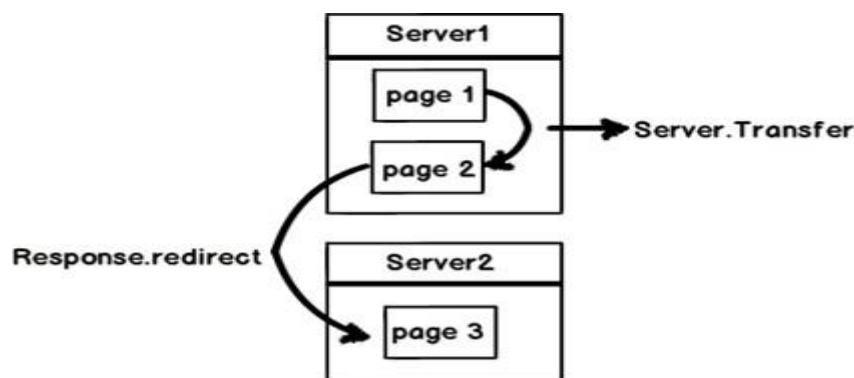
2.Client-Side State Management in ASP.NET

- **Cookies:** Cookies are small piece of text which is stored on the client's computer by the browser.it allows web applications to save user's information to **reuse**, if needed. The server can read these cookies from the client-side and perform data abstraction.

- **Viewstate:** It is used to manage page-level state and is used for **storing, sending, and receiving user's information**. We can store small values in Viewstate. It can able to store any type of data.
- **Control state:** It is a technique used to maintain data work in order, properly. We use Control State to use the view state without the possibility of it being disabled by the user. **We can't disable control state** but we can disable Viewstate.
- **Query String:** The query string is used to store the value in the URL.
- **Hidden Field:** This field is used to store values on the client-side. The hidden field works on request and is **not displayed on the browser**.

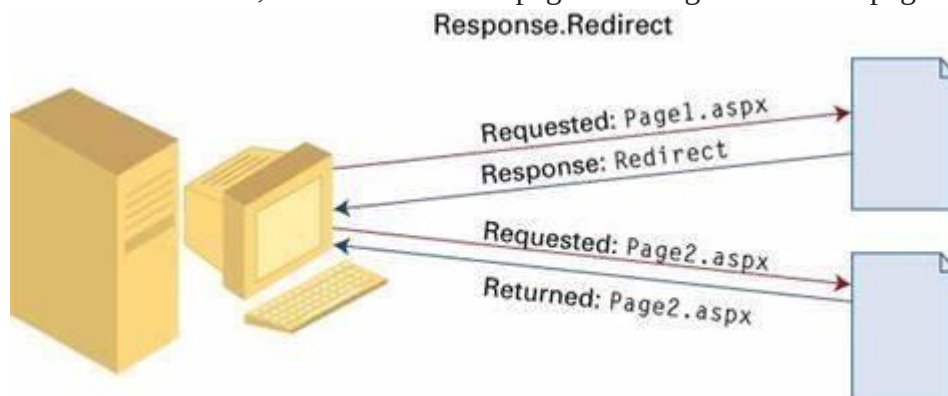
Response.Redirect and Server.Transfer

Response.Redirect and Server.Transfer both are used to navigate from one page to another page, but there are some differences between them depending on the pages that we want to navigate to.



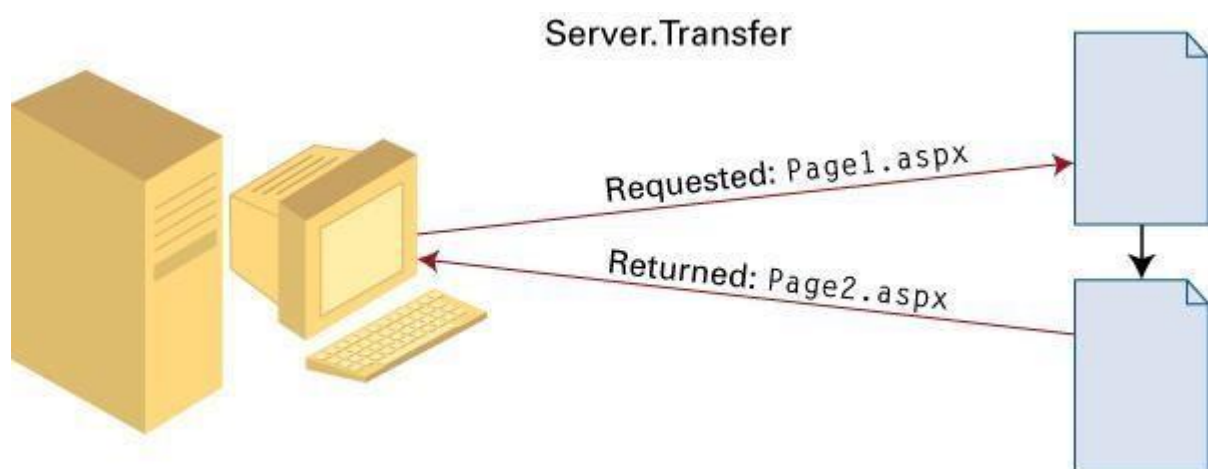
Response.Redirect

- Response.Redirect navigates the pages within the same application or within the same server and outside of the server also.
- The page is **redirected permanently**.
- Used to navigate within the same application as well as navigating from **one application to another application**.
- The URL is changed because the processing of the second page is **done on the second page**, not on the first page from where we redirected.
- We can **bookmark the page** because the full address is shown in a browser URL.
- An extra round trip happens to the server.
- It is used in HTML, ASP and ASP.Net pages to navigate from one page to another.



Server. Response

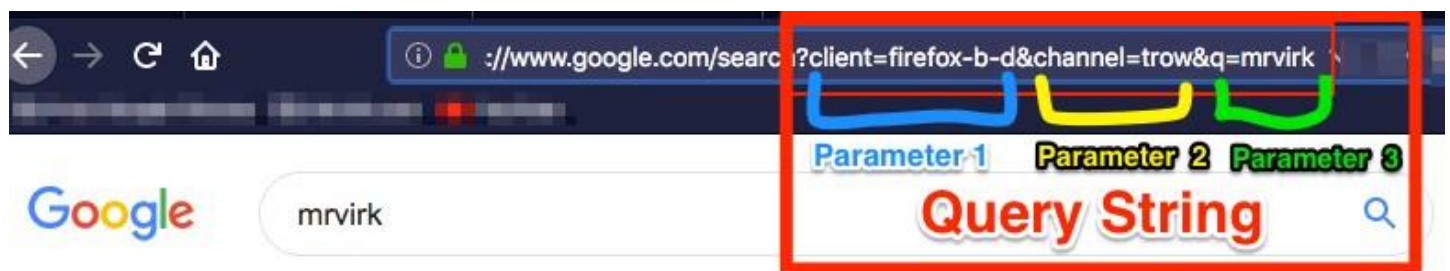
- Server. Response navigates the pages within the same application or within the same server, the page is still in memory that can read the values directly from page2 on page1.
- The Page is **not redirected permanently**.
- Used to navigate within the same application, **not outside of an application**.
- The URL is changed because the processing of the second page is **done on the same page**, without navigating on the second page.
- We **cannot bookmark the page** because the full address is not shown in a browser URL.
- An extra round trip does not happen to the server; because of this it saves server resources.
- It is only used to navigate within an ASP or ASP.Net page, not within HTML pages.



Query string

A **query string** is a part of a [uniform resource locator](#) (URL) that assigns values to specified parameters. A query string commonly includes fields added to a base URL by a Web browser or other client application.

Query string is string included in the url after a question mark (?). A Query String is the string passed to the url in the form of parameters, it can contain one or more parameters, each parameter is separated using the ampersand (&). Below is a real world use-case of query string and parameters.



In the above example, we search for a keyword “mrvirk” in Google search, and the page passes 3 parameters back to the Google server that helps G Search to provide you results for the keyword

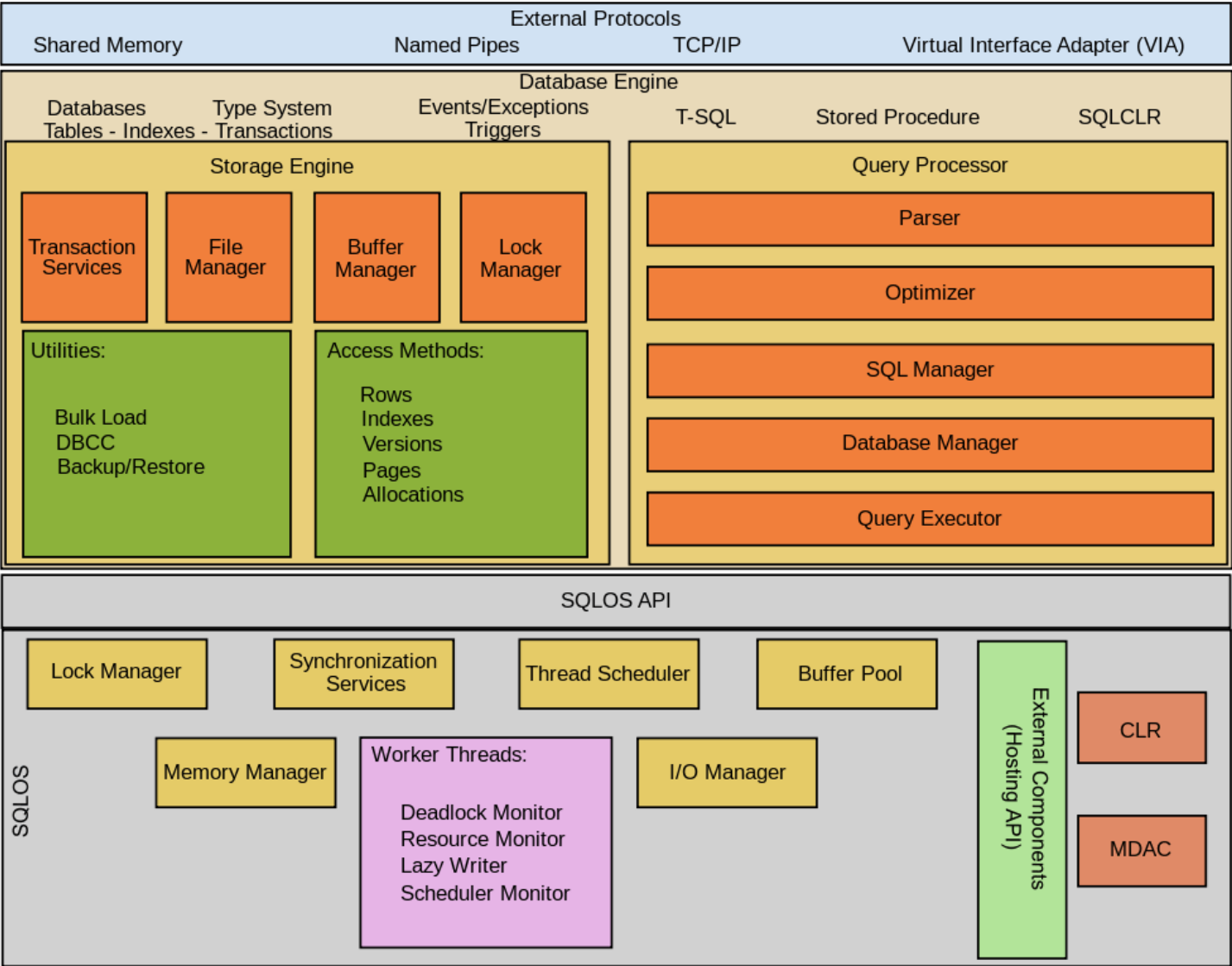
- Parameter 1 (client=firefox-b-d): tells google what browser user is on, in my case its Firefox
- Parameter 2 (channel=trow): tells google the channel
- Parameter 3 (q=mrvirk): tells google what the user searched for, in this case mrvirk

SQL Server

SQL Server is a relational database management system, or RDBMS, developed and marketed by Microsoft.

Similar to other RDBMS software, SQL Server is built on top of [SQL](#), a standard programming language for interacting with relational databases.

SQL Server works exclusively on the Windows environment for more than 20 years. In 2016, Microsoft made it available on Linux. SQL Server 2017 became generally available in October 2016 that ran on both Windows and Linux.



SQL Server consists of two main components:

- Database Engine
- SQLOS

1.Database Engine

The core component of the SQL Server is the Database Engine. The Database Engine consists of a relational engine that processes queries and a storage engine that manages database files, pages, indexes, etc. The database objects such as stored procedures, views, and triggers are also created and executed by the Database Engine.

query processor

The query processor contains the components that determine the best way to execute a query. The relational engine is also known as the Relational Engine.

The query processor requests data from the storage engine based on the input query and processed the results.

Storage Engine

The storage engine is in charge of storage and retrieval of data from the storage systems such as disks.

2.SQLOS (Structure Query Language Server Operating System)

Under the relational engine and storage engine is the SQL Server Operating System or SQLOS.

SQLOS provides many operating system services such as memory and I/O management. Other services include exception handling, memory management and synchronization services.

Database Management System (DBMS):

- Database management system is a software which is used to manage the database. For example: SQL, MySQL, Oracle etc are a very popular commercial database which is used in different applications.
- DBMS provides an interface to perform various operations like database creation, storing data in it, updating data, creating a table in the database and a lot more.
- It provides protection and security to the database. In the case of multiple users, it also maintains data consistency.

DBMS allows users to do the following tasks:

1. **Data Definition:** It is used for creation, modification, and removal the **structure of the tables** in the database.
2. **Data Updation:** It is used for the insertion, modification, and deletion of the **actual data** in the database.
3. **Data Retrieval:** It is used to retrieve the data from the database which can be used by applications for various purposes.
4. **User Administration:** It is used for **registering and monitoring users**, maintain data integrity, providing data security, performance and recovering information corrupted by unexpected failure.

SQL Table

- SQL Table is a collection of data which is organized in terms of rows and columns. In DBMS, the table is known as relation and row as a tuple.
- Table is a simple form of data storage. A table is also considered as a convenient representation of relations.

An example of the **EMPLOYEE** table:

EMP_ID	EMP_NAME	CITY	PHONE_NO
1	Kristen	Washington	7289201223
2	Anna	Franklin	9378282882
3	Jackson	Bristol	9264783838
4	Kellan	California	7254728346
5	Ashley	Hawaii	9638482678

In the above table, "EMPLOYEE" is the table name, "EMP_ID", "EMP_NAME", "CITY", "PHONE_NO" are the column names. The combination of data of multiple columns forms a row, e.g., 1, "Kristen", "Washington" and 7289201223 are the data of one row.

Operation on Table

1. **Create table:** SQL create table is used to create a table in the database.

Syntax:

```
create table "table_name"
(
"column1" "data type",
"column2" "data type",
"column3" "data type",
...
"columnN" "data type"
);
```

2. **Drop table:** A SQL drop table is used to delete a table definition and all the data from a table.

Syntax

```
DROP TABLE "table_name";
```

3. **Delete table:** In SQL, DELETE statement is used to delete rows from a table. We can use WHERE condition to delete a specific row from a table. If you want to delete all the records from the table, then you don't need to use the WHERE clause.

Syntax:

```
DELETE FROM table_name WHERE condition;
```

4. **Rename table:** We can change the name of the table.

Syntax:

```
RENAME TABLE EMPLOYEE TO EMP_INFO
```

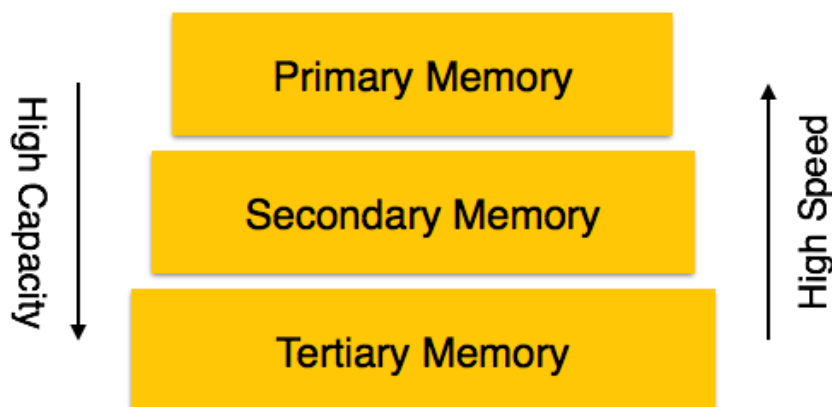

Data Storage in DBMS:

A database system provides an ultimate view of the stored data. However, data in the form of bits, bytes get stored in different storage devices.

Types of Data Storage

For storing the data, there are different types of storage options available. These storage types differ from one another as per the speed and accessibility. There are the following types of storage devices used for storing the data:

- Primary Storage
- Secondary Storage
- Tertiary Storage



1. **Primary Storage:** – CPU's internal memory (registers), fast memory (cache), and main memory (RAM) are directly accessible to the CPU, as they are **all placed on the motherboard or CPU** chipset. This storage is typically very small, ultra-fast, and volatile. Primary storage requires continuous power supply in order to maintain its state. In case of a power failure, all its data is lost.
2. **Secondary Storage** – Secondary storage devices are used to store data for future use or as backup. Secondary storage includes memory devices that are **not a part of the CPU** chipset or motherboard, for example, magnetic disks, optical disks (DVD, CD, etc.), hard disks, flash drives, and magnetic tapes.
3. **Tertiary Storage** – Tertiary storage is used to store huge volumes of data. Since such storage devices are **external to the computer system**, they are the slowest in speed. These storage devices are mostly used to take the back up of an entire system. Optical disks and magnetic tapes are widely used as tertiary storage.

Anomaly:

An anomaly is a fault that is present in the database which occurs because of the poor maintenance and poor storing of the data in the flat database. Normalization is the process of removing anomalies from the database. There are different types of anomalies that can occur in referencing and referenced relations which can be discussed as:

These anomalies can be categorized into three types:

1. Insertion Anomalies

2. Deletion Anomalies
3. Update Anomalies

1. **Insertion Anomaly:** Insertion Anomaly refers to when one cannot insert a new tuple into a relationship due to lack of data.
2. **Deletion Anomaly:** The delete anomaly refers to the situation where the deletion of data results in the unintended loss of some other important data.
3. **Updation Anomaly:** The update anomaly is when an update of a single data value requires multiple rows of data to be updated.

Normalization:

- Normalization is the process of organizing the data in the database.
- The main reason for normalizing the relations is removing Insertion, Update, and Deletion Anomalies.
- Normalization divides the larger table into smaller and links them using relationships.
- The normal form is used to reduce redundancy from the database table.

First Normal Form (1NF)

- A relation will be 1NF if it contains an atomic value.
- It states that an attribute of a table cannot hold multiple values. It must hold only single-valued attribute.
- First normal form disallows the multi-valued attribute, composite attribute, and their combinations.

Example: Relation EMPLOYEE is not in 1NF because of multi-valued attribute EMP_PHONE.

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	7272826385, 9064738238	UP
20	Harry	8574783832	Bihar
12	Sam	7390372389, 8589830302	Punjab

The decomposition of the EMPLOYEE table into 1NF has been shown below:

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	7272826385	UP
14	John	9064738238	UP
20	Harry	8574783832	Bihar
12	Sam	7390372389	Punjab
12	Sam	8589830302	Punjab

Second Normal Form (2NF)

- In the 2NF, relational must be in 1NF.

- In the second normal form, all non-key attributes are fully functional dependent on the primary key

Example: Let's assume, a school can store the data of teachers and the subjects they teach. In a school, a teacher can teach more than one subject.

TEACHER table

TEACHER_ID	SUBJECT	TEACHER_AGE
25	Chemistry	30
25	Biology	30
47	English	35
83	Math	38
83	Computer	38

In the given table, non-prime attribute TEACHER_AGE is dependent on TEACHER_ID which is a proper subset of a candidate key. That's why it violates the rule for 2NF.

To convert the given table into 2NF, we decompose it into two tables:

TEACHER_DETAIL table:

TEACHER_ID	TEACHER_AGE
25	30
47	35
83	38

TEACHER_SUBJECT table:

TEACHER_ID	SUBJECT
25	Chemistry
25	Biology
47	English
83	Math
83	Computer

Third Normal Form (3NF)

- A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.
- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

A relation is in third normal form if it holds atleast one of the following conditions for every non-trivial function dependency $X \rightarrow Y$.

1. X is a super key.
2. Y is a prime attribute, i.e., each element of Y is part of some candidate key.

Example:

EMPLOYEE_DETAIL table:

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harry	201010	UP	Noida
333	Stephan	02228	US	Boston
444	Lan	60007	US	Chicago
555	Katharine	06389	UK	Norwich
666	John	462007	MP	Bhopal

Super key in the table above:

1. {EMP_ID}, {EMP_ID, EMP_NAME}, {EMP_ID, EMP_NAME, EMP_ZIP}. so on

Candidate key: {EMP_ID}

Non-prime attributes: In the given table, all attributes except EMP_ID are non-prime.

Here, EMP_STATE & EMP_CITY dependent on EMP_ZIP and EMP_ZIP dependent on EMP_ID. The non-prime attributes (EMP_STATE, EMP_CITY) transitively dependent on super key(EMP_ID). It violates the rule of third normal form.

That's why we need to move the EMP_CITY and EMP_STATE to the new <EMPLOYEE_ZIP> table, with EMP_ZIP as a Primary key.

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_ZIP
222	Harry	201010
333	Stephan	02228
444	Lan	60007
555	Katharine	06389
666	John	462007

EMPLOYEE_ZIP table:

EMP_ZIP	EMP_STATE	EMP_CITY
201010	UP	Noida
02228	US	Boston
60007	US	Chicago
06389	UK	Norwich
462007	MP	Bhopal

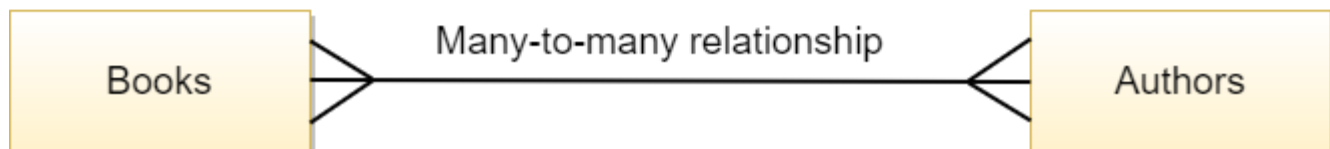
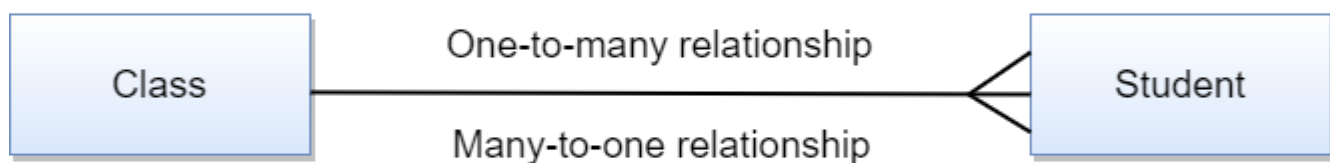
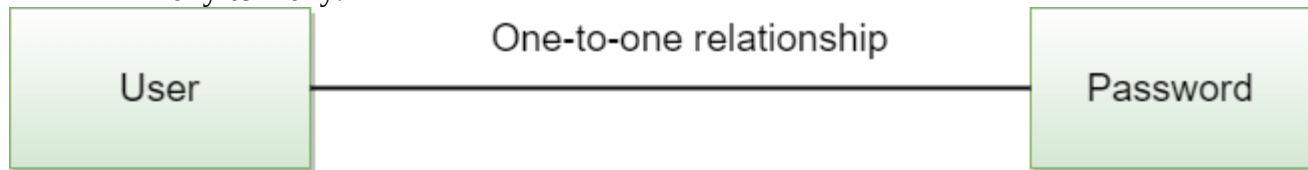
Relations:

In relational databases, a relationship exists between two tables when one of them has a foreign key that references the primary key of the other table. It allows relational databases to split and store data in different tables.

Types of relationships in a database

There are 3 main types of relationship in a database:

- one-to-one
- one-to-many
- many-to-many.



Primary Key:

The primary key is a **unique or non-null key** that uniquely identifies every record in a table or relation. Each database needs a unique identifier for every row of a table, and the primary key plays a vital role in identifying rows in the table uniquely. The primary key column can't store **duplicate values**. It is also called a **minimal super key**; therefore, we cannot specify more than one primary key in any relationship.

For example, we have a table named **customer** with attributes such as ID, Name, and City. Only the **ID column** can never contain duplicate and NULL values because each customer has a unique identification number. This feature helps to identify each record in the database uniquely. Hence, we can make the ID attribute a primary key.

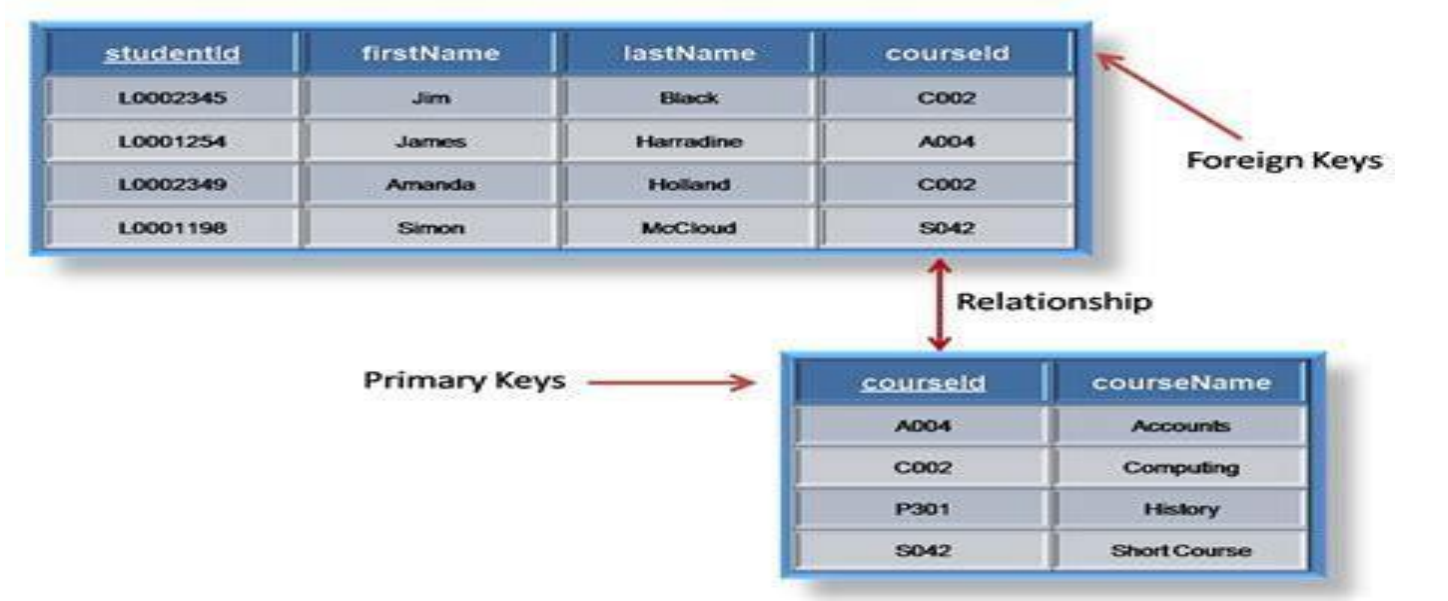
```
MySQL 8.0 Command Line Client

mysql> SELECT * FROM customer;
+-----+-----+-----+
| ID | Name | City |
+-----+-----+-----+
| 1 | Joseph | California |
| 2 | Mary | NewYork |
| 3 | John | Alaska |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

Foreign Key:

A foreign key is a column or group of columns in a relational database table that provides a link between data in two tables. It is also known as the **referencing key that establishes a relationship** between two different tables in a database. A FOREIGN KEY is a field (or collection of fields) in one table, that refers to the PRIMARY KEY in another table. A foreign key is beneficial in relational database normalization, especially when we need to access records from other tables.

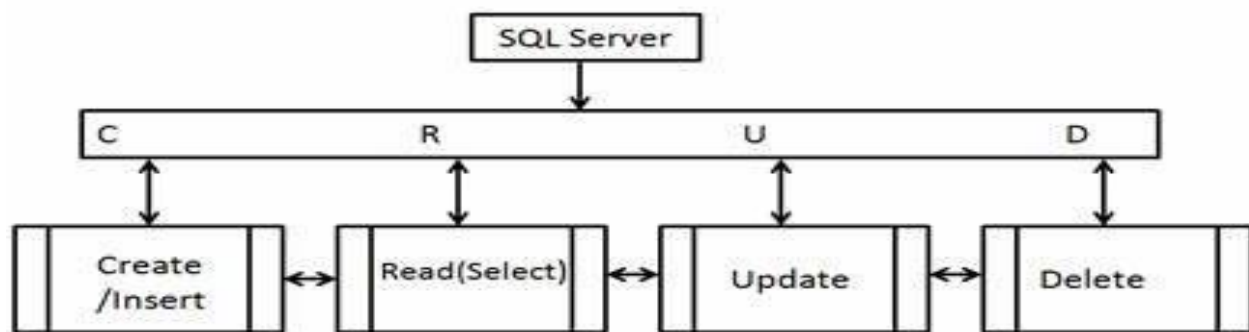
Example: Refer the figure –
STUD_NO in STUDENT_COURSE is a foreign key to STUD_NO in STUDENT relation.



SQL Data types

S.NO	DATA TYPE	DESCRIPTION
1.	BOOLEAN	TRUE or FALSE
2.	CHAR	Fixed length string EX: CHAR(40)
3.	VARCHAR	Variable length string EX: VARCHAR(40)
4.	BIT	Bit string EX: BIT(4) '1010'
5.	NUMERIC	Exact numeric data type EX: NUMERIC(7,2) 23233.45
6.	INTEGER	Integer (numbers) data only EX: 12
7.	FLOAT	Approximate numeric data type EX: 12.54
8.	DATE	Stores date "Year-Month-Day" EX:"2001-10-21"
9.	TIME	Stores time "Hour-Minutes-Seconds" EX:"05-30-50"
10.	TIMESTAMP	Stores both date and time.

CRUD operations:



CREATE:

In CRUD operations, 'C' is an acronym for **create**, which *means to add or insert data into the SQL table*. So, firstly we will create a table using CREATE command and then we will use the INSERT INTO command to insert rows in the created table. By using CREATE command we can create database and tables.

1. **Create Database** Students;

2. **CREATE TABLE** Stu_details

```
(
Roll_No Int,
Name Varchar(20),
Age Int,
DOB Date,
Pn_num Char(14)
);
```

OUTPUT:

Roll_No	Name	Age	DOB	Pn_num

3. **CREATE INDEX** Stu_Name_DOB **ON** Stu_details (Name,DOB);

INSERT:

1. **INSERT INTO** Stu_details **VALUES** (1,"Laxmi",20,"2000-08-28","+91-9012312233");
2. **INSERT INTO** Stu_details (Roll_No,Name,age) **VALUES** (2,"Ramu",21);

3. **INSERT INTO** Stu_details **VALUES** (3,"Ramya",19,"2001-04-25"," +91-9034512553"),
(4,"Santoshi",20,"2000-01-28"," +91-6312312239"), (5,"Pavani",18,"2002-05-23"," + 91-9149812237");

OUTPUT:

Roll_No	Name	Age	DOB	Pn_num
1	Laxmi	20	2000-08-28	91-9012312233
2	Ramu	21	NULL	NULL
3	Ramya	19	2001-04-25	+91-9034512553
4	Santoshi	20	2000-01-28	+91-6312312239
5	Pavani	18	2002-05-23	+91-9149812237

READ/SELECT:

In CRUD operations, 'R' is an acronym for **read**, which means *retrieving or fetching the data from the SQL table*. So, we will use the SELECT command to fetch the inserted records from the SQL table. We can retrieve all the records from a table using an asterisk (*) in a SELECT query. There is also an option of retrieving only those records which satisfy a particular condition by using the WHERE clause in a SELECT query.

1. **SELECT * FROM** Stu_details;

OUTPUT:

Roll_No	Name	Age	DOB	Pn_num
1	Laxmi	20	2000-08-28	91-9012312233
2	Ramu	21	NULL	NULL
3	Ramya	19	2001-04-25	+91-9034512553
4	Santoshi	20	2000-01-28	+91-6312312239
5	Pavani	18	2002-05-23	+91-9149812237

2. **SELECT** Name,Pn_num **FROM** Stu_details;

OUTPUT:

Name	Pn_num
Laxmi	91-9012312233
Ramu	NULL
Ramya	+91-9034512553
Santoshi	+91-6312312239

Pavani	+91-9149812237
--------	----------------

3. **SELECT** Name, Age, DOB **FROM** Stu_details **WHERE** age < 20;

OUTPUT:

Name	Age	DOB
Ramya	19	2001-04-25
Pavani	18	2002-05-23

4. **SELECT** Name, DOB **FROM** Stu_details **WHERE** age < 20;

OUTPUT:

Name	DOB
Ramya	2001-04-25
Pavani	2002-05-23

UPDATE:

In CRUD operations, 'U' is an acronym for the **update**, which *means making updates to the records present in the SQL tables*. So, we will use the UPDATE command to make changes in the data present in tables.

1. **UPDATE** Stu_details **SET** Name="Kaveri" **WHERE** Roll_No=4 ;

OUTPUT:

Roll_No	Name	Age	DOB	Pn_num
1	Laxmi	20	2000-08-28	91-9012312233
2	Ramu	21	NULL	NULL
3	Ramya	19	2001-04-25	+91-9034512553
4	Kaveri	20	2000-01-28	+91-6312312239
5	Pavani	18	2002-05-23	+91-9149812237

2. **UPDATE** Stu_details **SET** age=18, DOB="2002-04-15" **WHERE** Roll_No=2;

OUTPUT:

Roll_No	Name	Age	DOB	Pn_num
1	Laxmi	20	2000-08-28	91-9012312233

2	Ramu	21	NULL	NULL
3	Ramya	19	2001-04-25	+91-9034512553
4	Kaveri	20	2000-01-28	+91-6312312239
5	Pavani	18	2002-05-23	+91-9149812237

DELETE:

In CRUD operations, 'D' is an acronym for **delete**, which means *removing or deleting the records from the SQL tables*. We can delete all the rows from the SQL tables using the DELETE query. There is also an option to remove only the specific records that satisfy a particular condition by using the WHERE clause in a DELETE query.

1. **DELETE FROM** Stu_details **WHERE** Roll_No=2; (Complete row will be deleted)

OUTPUT:

Roll_No	Name	Age	DOB	Pn_num
1	Laxmi	20	2000-08-28	91-9012312233
3	Ramya	19	2001-04-25	+91-9034512553
4	Kaveri	20	2000-01-28	+91-6312312239
5	Pavani	18	2002-05-23	+91-9149812237

2. **DELETE FROM** Stu_details **WHERE** age>19 ;

OUTPUT:

Roll_No	Name	Age	DOB	Pn_num
3	Ramya	19	2001-04-25	+91-9034512553
5	Pavani	18	2002-05-23	+91-9149812237

Aggregate operations

- SQL aggregation operations is used to perform the calculations on multiple rows of a single column of a table. It returns a single value.
- It is also used to summarize the data.

Types of SQL Aggregation Operations

1. COUNT FUNCTION

- COUNT function is used to Count the number of rows in a database table. It can work on both numeric and non-numeric data types.

- COUNT function uses the COUNT(*) that returns the count of all the rows in a specified table. COUNT(*) considers duplicate and Null.

Example database: Stu_list

S_NO	Name	Branch	Marks
1	Devi	ECE	85
2	Chandu	Civil	91
3	Ramya	ECE	97
4	Anusha	CSE	93
5	Ramu	Mech	88
6	Anitha	Civil	90
7	Kaveri	EEE	92

SELECT COUNT(S_NO) **FROM** Stu_list;

COUNT(S_NO)
7

2. SUM FUNCTION

Sum function is used to calculate the sum of all selected columns. It works on numeric fields only.

SELECT SUM(Marks) **FROM** Stu_list;

SUM(Marks)
636

3. AVG FUNCTION

The AVG function is used to calculate the average value of the numeric type. AVG function returns the average of all non-Null values.

SELECT AVG(Marks) **FROM** Stu_list;

AVG(Marks)
90

4. MAX FUNCTION

MAX function is used to find the maximum value of a certain column. This function determines the largest value of all selected values of a column.

SELECT Name,Branch,MAX(Marks) **FROM** Stu_list;

Name	Branch	MAX(Marks)
Ramya	ECE	97

4.MIN FUNCTION

MIN function is used to find the minimum value of a certain column. This function determines the smallest value of all selected values of a column.

SELECT Name,Branch,MIN(Marks) **FROM** Stu_list;

Name	Branch	MIN(Marks)
Devi	ECE	85

SQL GROUP BY Functions:

Group by clause in SQL used to arrange logically related data into groups with help of some functions.

Example database: Stu_list

S_NO	Name	Branch	Marks
1	Devi	ECE	85
2	Chandu	Civil	91
3	Ramya	ECE	97
4	Anusha	CSE	93
5	Ramu	Mech	92
6	Anitha	Civil	90
7	Kaveri	EEE	92

Group by with COUNT():

SELECT COUNT(S_NO),Branch **FROM** Stu_list **GROUP BY** Branch;

COUNT(S_NO)	Branch
2	ECE
2	CIVIL
1	CSE
1	Mech

1	EEE
---	-----

Group by with SUM():

SELECT SUM(Marks),Branch **FROM** Stu_list **GROUP BY** Branch;

SUM(Marks)	Branch
182	ECE
181	CIVIL
93	CSE
92	Mech
92	EEE

Group by with AVG():

SELECT AVG(Marks),Branch **FROM** Stu_list **GROUP BY** Branch;

AVG(Marks)	Branch
91	ECE
90.5	CIVIL
93	CSE
92	Mech
92	EEE

Group by with MAX():

SELECT MAX(Marks),Branch,Name **FROM** Stu_list **GROUP BY** Branch;

MAX(Marks)	Branch	Name
97	ECE	Ramya
91	CIVIL	Chandu
93	CSE	Anusha
92	Mech	Ramu
92	EEE	Kaveri

Group by with MIN():

SELECT MIN(Marks),Branch,Name **FROM** Stu_list **GROUP BY** Branch;

MIN(Marks)	Branch	Name
85	ECE	Devi
90	CIVIL	Anitha

93	CSE	Anusha
92	Mech	Ramu
92	EEE	Kaveri

SQL ORDER BY Functions:

The ORDER BY statement in SQL is used to sort the fetched data in either ascending or descending according to one or more columns.

- By default ORDER BY sorts the data in ascending order.
- We can use the keyword DESC to sort the data in descending order and the keyword ASC to sort in ascending order.

Order by with ASC:

SELECT * FROM Stu_list **ORDER BY** Marks;

(OR)

SELECT * FROM Stu_list **ORDER BY** Marks **ASC**;

S_NO	Name	Branch	Marks
1	Devi	ECE	85
6	Anitha	Civil	90
2	Chandu	Civil	91
5	Ramu	Mech	92
7	Kaveri	EEE	92
4	Anusha	CSE	93
3	Ramya	ECE	97

Order by with DESC:

SELECT * FROM Stu_list **ORDER BY** Marks **DESC**;

S_NO	Name	Branch	Marks
3	Ramya	ECE	97
4	Anusha	CSE	93
7	Kaveri	EEE	92
5	Ramu	Mech	92
2	Chandu	Civil	91
6	Anitha	Civil	90
1	Devi	ECE	85

Order by with ASC & DESC:

SELECT * FROM Stu_list **ORDER BY** Marks **ASC,S_NO DESC**;

S_NO	Name	Branch	Marks
1	Devi	ECE	85
6	Anitha	Civil	90
2	Chandu	Civil	91
7	Kaveri	EEE	92
5	Ramu	Mech	92
4	Anusha	CSE	93
3	Ramya	ECE	97

SQL Date Time Operations

Following are some of the important date time functions which are in-built in SQL.

1. NOW()
2. CURDATE()
3. CURTIME()
4. DATE()
5. EXTRACT()
6. DATE_ADD()
7. DATE_SUB()
8. DATEDIFF()
9. DATE_FORMAT()

1. NOW():

NOW () function in SQL will give the current system's date and time.

Syntax:

SELECT NOW ();

Query:

SELECT NOW () **AS** Current_Date_Time;

Current_Date_Time
2023-04-10 18:28:44

2. CURDATE():

CURDATE () function in SQL will give the current system's date.

Syntax:

SELECT CURDATE ();

Query:

SELECT CURDATE () **AS** Current_Date;

Current_Date
2023-04-10

3. CURTIME():

CURTIME () function in SQL will give the current system time.

Syntax:

SELECT CURTIME ();

Query:

SELECT CURTIME () **AS** Current_Time;

Current_Time
18:28:44

4. DATE():

Using the DATE () function in SQL, you can specifically extract the date from the DATETIME datatype column.

Syntax:

SELECT DATE (DateTimeValue);

Query:

SELECT DATE ("2021-10-24 18:28:44") **AS** SHOW_DATE;

SHOW_DATE
2023-04-10

5. EXTRACT():

Using the EXTRACT() function in SQL, we can extract a specific part of date and time according to our requirements: day, month, year, day, hour, minute, etc.

Syntax:

SELECT EXTRACT (PART **FROM** DATE / TIME);

Query:

1. **SELECT** EXTRACT (YEAR **FROM** "2021-10-24") **AS** SHOW_YEAR;

SHOW_YEAR
2021

2. **SELECT** EXTRACT (MONTH **FROM** "2021-10-24") **AS** SHOW_MONTH;

SHOW_MONTH
10

3. **SELECT** EXTRACT (DAY **FROM** "2021-10-24") **AS** SHOW_DAY;

SHOW_DAY
24

4. **SELECT** EXTRACT (HOUR **FROM** "19:10:43") **AS** SHOW_HOUR;

SHOW_HOUR
19

5. **SELECT** EXTRACT (MINUTE **FROM** "19:10:43") **AS** SHOW_MINUTE;

SHOW_MINUTE
10

6. **SELECT** EXTRACT (SECOND **FROM** "19:10:43") **AS** SHOW_SECOND;

SHOW_SECOND
43

6. DATE_ADD():

Using the DATE_ADD () function in SQL, we can add a specific time interval to the given date.

Syntax:

SELECT DATE_ADD (**DATE**, INTERVAL VALUE Unit_to_be_added);

Query:

1. **SELECT** DATE_ADD ("2021-10-24", INTERVAL 15 **DAY**) **AS** NEW_DATE;

NEW_DATE
2021-11-08

2. **SELECT** DATE_ADD ("2021-10-24", INTERVAL 5 **MONTH**) **AS** NEW_DATE;

NEW_DATE
2022-03-24

3. **SELECT** DATE_ADD ("2021-10-24", INTERVAL 25 **YEAR**) **AS** NEW_DATE;

NEW_DATE
2046-10-24

7. DATE_SUB():

Using the DATE_SUB () function in SQL, we can remove a specific time interval from the given date.

Syntax:

SELECT DATE_SUB (**DATE**, INTERVAL VALUE Unit_to_be_subtracted);

Query:

1. **SELECT** DATE_SUB("2021-10-24", INTERVAL 25 **YEAR**) **AS** NEW_DATE;

NEW_DATE
1996-10-24

2. **SELECT** DATE_SUB("2021-10-24", INTERVAL 5 **MONTH**) **AS** NEW_DATE;

NEW_DATE
2021-05-24

3. **SELECT** DATE_SUB("2021-10-24", INTERVAL 15 **DAY**) **AS** NEW_DATE;

NEW_DATE
2021-10-09

8. DATEDIFF():

Using the DATEDIFF() function in SQL will give us the number of days that fall between the two given dates.

Syntax:

```
SELECT DATEDIFF(Date1, Date2);
```

Query:

```
SELECT DATEDIFF("2021-10-24", "2021-10-09") AS Number_of_Days;
```

Number_of_Days
15

9. DATE_FORMAT()

Using the DATE_FORMAT () function in SQL, we can display the date or time-related information in a well-formatted manner.

Syntax of using a DATE_FORMAT function on a table's column:

```
SELECT DATE_FORMAT (ColumnName, Expression) FROM Table Name;
```

Query:

```
SELECT DATE_FORMAT ("2021-10-24 22:30:17", "%W %D %M %Y %r") AS Formatted_DateTime;
```

Formatted_DateTime
Sunday 24th October 2021 10:30:17 PM

String functions

are used to perform an operation on input string and return an output string.

Following are the string functions defined in SQL:

1. ASCII(): This function is used to find the ASCII value of a character.

Syntax: SELECT ascii('t');

Output: 116

2.LCASE(): This function is used to convert the given string into l+ower case.

Syntax: LCASE ("DotNet");

Output: dotnet

3.UCASE(): This function is used to make the string in upper case.

Syntax: UCASE ("DotNet");

Output:DOTNET

4.RIGHT(): This function is used to SELECT a sub string from the right end of the given size.

Syntax: SELECT RIGHT('DotNet', 3);

Output: 'Net'

5.LEFT(): This function is used to SELECT a sub string from the left of given size or characters.

Syntax: SELECT LEFT('DotNet', 3);

Output: Dot

6.LENGTH(): This function is used to find the length of a word.

Syntax: LENGTH('DotNet');

Output: 6

7.LOCATE(): This function is used to find the nth position of the given word in a string.

Syntax: SELECT LOCATE('Net', 'DotNet', 1);

Output: 4

8.LPAD(): This function is used to make the given string of the given size by adding the given symbol.

Syntax: LPAD('DotNet', 9, '0');

Output:000DotNet

9.RPAD(): This function is used to make the given string as long as the given size by adding the given symbol on the right.

Syntax: RPAD('DotNet', 9, '0')

Output: 'DotNet000'

10.TRIM(): This function is used to cut the given symbol from the string.

Syntax: TRIM(LEADING '0' FROM '000123');

Output: 123

11.LTRIM(): This function is used to cut the given sub string from the original string.

Syntax: LTRIM('123123DotNet', '123');

Output: DotNet

12.RTRIM(): This function is used to cut the given sub string from the original string.

Syntax: RTRIM('DotNet123123', '123');

Output: 'geeks'

13.REPEAT(): This function is used to write the given string again and again till the number of times mentioned.

Syntax: SELECT REPEAT('DotNet', 2);

Output: DotNetDotNet

14.REVERSE(): This function is used to reverse a string.

Syntax: SELECT REVERSE('geeksforgeeks.org');

Output: 'gro.skeegrofskeeg'

15.SPACE(): This function is used to write the given number of spaces.

Syntax: SELECT SPACE(7);

Output: ‘ ‘

Numeric Functions

Function	Description										
ABS	Returns the absolute value of a number. 1.SELECT ABS(2);2.SELECT ABS(-2); <table><tr><td>ABS(2)</td><td></td><td>ABS(2)</td></tr><tr><td>2</td><td></td><td>2</td></tr></table>	ABS(2)		ABS(2)	2		2				
ABS(2)		ABS(2)									
2		2									
CEILING	Returns the nearest integer value which is larger than or equal to the specified decimal value. SELECT CEILING(2.74); <table><tr><td>CEILING(2.74)</td></tr><tr><td>3</td></tr></table>	CEILING(2.74)	3								
CEILING(2.74)											
3											
COUNT	Returns the number of records in the SELECT query. SELECT COUNT(*); <table><tr><td>COUNT(*)</td></tr><tr><td>10</td></tr></table>	COUNT(*)	10								
COUNT(*)											
10											
FLOOR	Returns the largest integer value that is less than or equal to a number. The return value is of the same data type as the input parameter. SELECT FLOOR(2.74); <table><tr><td>FLOOR(2.74)</td></tr><tr><td>2</td></tr></table>	FLOOR(2.74)	2								
FLOOR(2.74)											
2											
MAX	Returns the maximum value in an expression.										
MIN	Returns the minimum value in an expression.										
POWER(X,Y)	These two functions return the value of X raised to the power of Y. SELECT POWER(3,3); <table><tr><td>POWER(3,3)</td></tr><tr><td>27</td></tr></table>	POWER(3,3)	27								
POWER(3,3)											
27											
ROUND	Returns a numeric expression rounded to a specified number of places right of the decimal point. SELECT ROUND(2.7434567,3); <table><tr><td>ROUND(2.7434567,3)</td></tr><tr><td>2.743</td></tr></table>	ROUND(2.7434567,3)	2.743								
ROUND(2.7434567,3)											
2.743											
SIGN	Returns an indicator of the sign of the input integer expression. 1.SELECT SIGN(-3);2.SELECT SIGN(3);3.SELECT SIGN(0); <table><tr><td>SIGN(-3)</td><td></td><td>SIGN(3)</td><td></td><td>SIGN(0)</td></tr><tr><td>-1</td><td></td><td>1</td><td></td><td>0</td></tr></table>	SIGN(-3)		SIGN(3)		SIGN(0)	-1		1		0
SIGN(-3)		SIGN(3)		SIGN(0)							
-1		1		0							
SUM	Returns the sum of all the values or only the distinct values, in the expression. NULL values are ignored.										

Function	Description		
	SELECT SUM(Marks); <table><tr><th>SUM(Marks)</th></tr><tr><td>98</td></tr></table>	SUM(Marks)	98
SUM(Marks)			
98			
+	The Addition operator is used to perform additional operations on the operands.		
-	The Subtraction operator performs subtraction operations on the operands (to get the difference between operands).		
*	The Multiplication operator performs multiplication operations on the operands.		
/	The Division operator performs division operations on the operands.		
%	The Modulus operator is used to perform the modulus operation on the operands (to get the remainder when one operand is divided by the other).		

Exception handling:

An exception is an error which disrupts the normal flow of program instructions. PL/SQL provides us the exception block which raises the exception thus helping the programmer to find out the fault and resolve it.

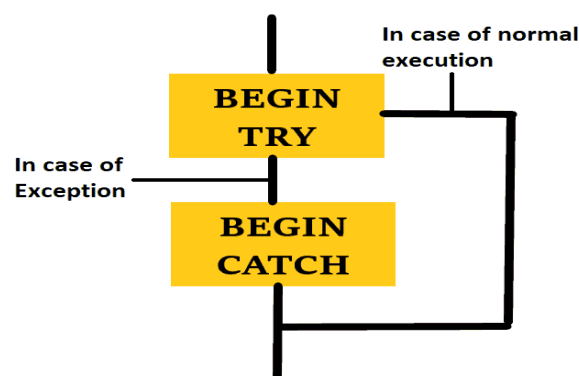
There are two types of exceptions defined in PL/SQL

1.System Defined Exceptions: These are the exceptions that are generated by the system during the program execution. The error messages are predefined in the system and the final user will see the predefined message on the console.

2.User-Defined Exceptions: These are the exceptions that are thrown explicitly by the programmer. The programmer can raise exceptions at any part of the SQL code according to the logic he has implemented in the program.

In SQL Server we use **BEGIN TRY** and **BEGIN CATCH** blocks to handle exceptions.

- ❖ We put the SQL statements that may cause an exception in the **BEGIN TRY** block.
- ❖ If any exception arises, the control is immediately transferred to the **BEGIN CATCH** block.
- ❖ In the **BEGIN CATCH** block, we write the statements that should be executed if an exception is occurred in the **BEGIN TRY** block.
- ❖ If no exception arises i.e. all the SQL statements inside the **BEGIN TRY** block are executed, the control is not transferred to the **BEGIN CATCH** block.



general syntax for **BEGIN TRY** and **BEGIN CATCH** block:

BEGIN TRY

--SQL Statements where exception may encounter

END TRY

BEGIN CATCH

--SQL statements to handle exceptions

END CATCH

EX:

USE [master]

GO

DECLARE @Number1 int,
 @Number2 int,
 @Result real

SET @Number1=10

SET @Number2=0

BEGIN TRY

SET @Result=@Number1/@Number2

PRINT(@Result)

END TRY

BEGIN CATCH

PRINT('You cannot divide a number by zero')

END CATCH

OUTPUT:

You cannot divide a number by zero

Trigger:

Trigger is a statement that a system executes automatically when there is any modification to the database. In a trigger, we first specify when the trigger is to be executed and then the action to be performed when the trigger executes.

Example –

Suppose, we are adding a tuple to the 'Donors' table that is some person has donated blood. So, we can design a trigger that will automatically add the value of donated blood to the 'Blood_record' table.

Types of Triggers –

We can define 6 types of triggers for each table:

1. **AFTER INSERT** activated after data is inserted into the table.
2. **AFTER UPDATE:** activated after data in the table is modified.
3. **AFTER DELETE:** activated after data is deleted/removed from the table.
4. **BEFORE INSERT:** activated before data is inserted into the table.
5. **BEFORE UPDATE:** activated before data in the table is modified.
6. **BEFORE DELETE:** activated before data is deleted/removed from the table.

Temporary tables:

Temporary Tables are most likely as Permanent Tables. Temporary Tables are Created in TempDB and are automatically deleted as soon as the last connection is terminated. Temporary Tables helps us to store and process intermediate results. Temporary tables are very useful when we need to store temporary data. The Syntax to create a Temporary Table is given below:

To Create Temporary Table:

```
CREATE TABLE #EmpDetails (id INT, name VARCHAR(25))
```

To Insert Values Into Temporary Table:

```
INSERT INTO #EmpDetails VALUES (01, 'Laxmi'), (02, 'Puja')
```

To Select Values from Temporary Table:

```
SELECT * FROM #EmpDetails
```

Result:

id	name
1	Laxmi
2	Puja

There are 2 types of Temporary Tables: Local Temporary Table, and Global Temporary Table. These are explained as following below.

1. Local Temporary Table:

A Local Temp Table is available only for the session that has created it. It is automatically dropped (deleted) when the connection is closed. To create Local Temporary Table Single “#” is used as the prefix of a table name.

Example: CREATE TABLE #EmpDetails (id INT, name VARCHAR(25))

2. Global Temporary Table:

Global Temporary Tables are available to all connections and Dropped when the last connection referencing the table is closed. Global Table Name must have an Unique Table Name. There will be no random Numbers suffixed at the end of the Table Name.

To create a Global Temporary Table, add the “##” symbol before the table name.

Example: CREATE TABLE ##EmpDetails (id INT, name VARCHAR(25))

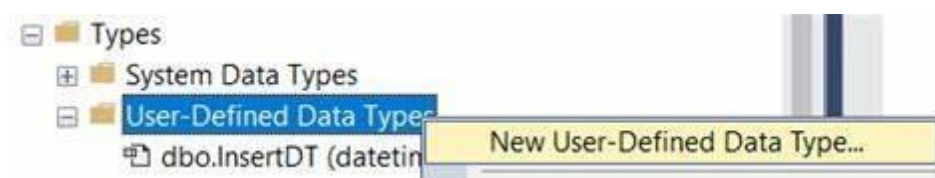
User defined data types:

When there is a requirement to create a column with a datatype in the database and creating the same data type columns multiple times by multiple people. Here there is no guarantee that all developers will create the same type. In this situation we can make use of User-Defined data types. So, we create at one time and using the type wherever you need.

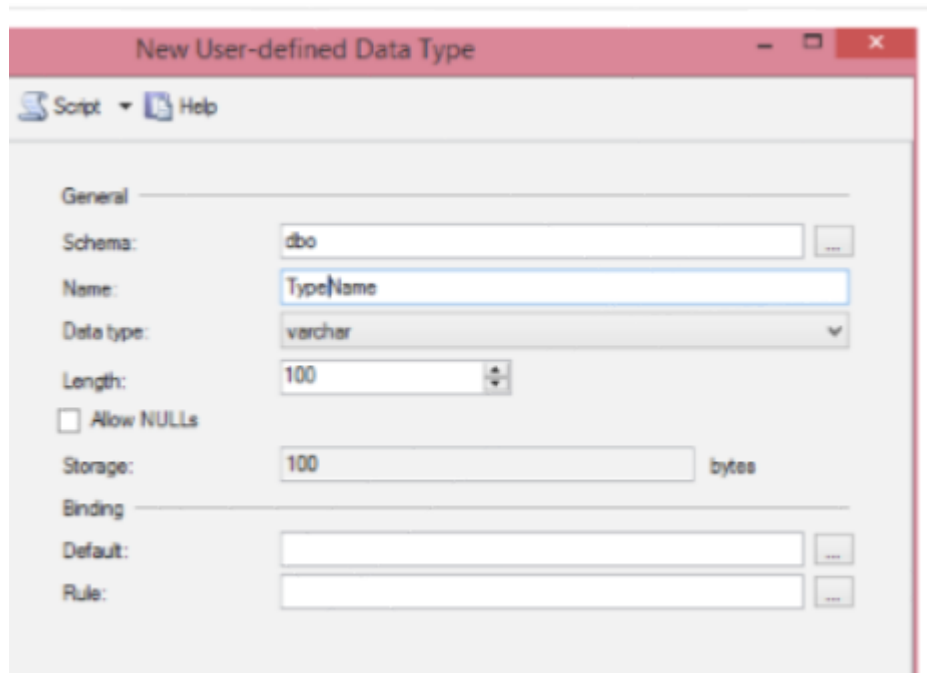
There are 2 ways to create it. The first one and the easiest method is from UI and the second one is using script.

Using UI steps to create User-defined Data types:

1. Expand the data base where you want to create data types -> Programmability -> Types.



2.Right click on the User-Defined Data Types and click on New User-Defined Data Type... . A window will pop up to create the type as shown in the below figure.



3.Enter the Name of the type in the Name textbox and select the base type from Data Type dropdown. Length input will be enabled based on the data type selection. And click on OK button of the popup.

Using Script to create User-defined Data types:

creating 3 user defined data types for Name, Age and Location.

```
CREATE TYPE TypeName FROM VARCHAR(100)
```

```
CREATE TYPE TypeAge FROM INT
```

```
CREATE TYPE Location FROM VARCHAR(500)
```

User defined table types:

Generally, we declare tables to store temp data and use it in the next lines of the script.

```
DECLARE @empTempTab Table (name varchar(100), age int, location VARCHAR(100))
```

Suppose we need to create the same kind of temp table in multiple SPs. In this case, instead of creating temp tables multiple times we can create required table type once and use it where ever we require. We don't have UI to create the User-Defined Table. We can create User-Defined Table types using script method.

Using Script to create User-defined Data types:

Let's take the above temp table structure.

```
CREATE TYPE empTempTabtype AS Table (name varchar(100), age int, location VARCHAR(100))
```

With the above statement we can create the table type.

Now let's use the above table type to create the temp table.

```
DECLARE @empTempTab empTempTabType
```

Stored procedure:

In SQL, stored procedure is a set of statement(s) that perform some defined actions. We make stored procedures so that we can reuse statements that are used frequently. Stored procedures are similar to functions in programming. They can accept parameters, and perform operations when we call them.

Creating a Procedure:

We create stored procedures using the `CREATE PROCEDURE` command followed by SQL commands. For example

```
CREATE PROCEDURE us_customers AS
SELECT customer_id, first_name
FROM Customers
WHERE Country = 'USA';
```

Executing Stored Procedure:

Now whenever we want to fetch all customers who live in the **USA**, we can simply call the procedure mentioned above. For example,

```
EXEC us_customers;
```

Functions in SQL:

SQL Server has many built-in functions including aggregate functions, date functions, string functions, control flow functions, window functions, and math functions.

SQL Aggregate Functions

In this tutorial, you will learn about the SQL aggregate functions including `AVG()`, `COUNT()`, `MIN()`, `MAX()`, and `SUM()`.

SQL Comparison Functions

This section provides you with the SQL comparison functions including `COALESCE`, `DECODE`, and `NULLIF`.

SQL String Functions

This section provides you with some handy SQL string functions that allow you to manipulate string data more effectively.

SQL Math Functions

SQL has many mathematical functions that allow you to perform business and engineering calculations.

SQL Date Functions

This section covers the most important SQL date functions that allow you to manipulate date and time data effectively.

SQL Window Functions

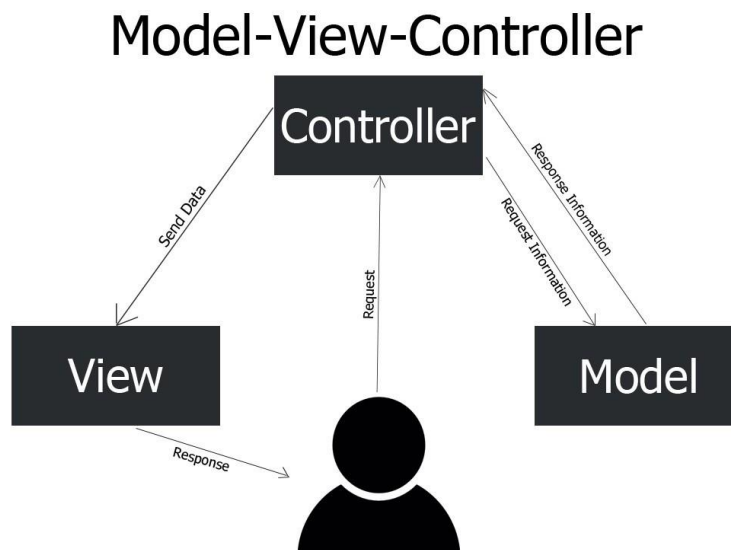
This tutorial shows you how to use the SQL window functions to solve complex query challenges in easy ways.

UNIT-V

Introduction to ASP.NET MVC, The role of the Model, View, and Controller, Key benefits of ASP.NET MVC Getting Started with ASP.NET MVC, ASP.NET MVC project templates, Understanding the structure of an ASP.NET MVC project, Naming conventions, Creating views, Defining controllers, Defining a data model, Creating a ASP.NET MVC 4 Application, Creating strongly-typed views, Understanding URLs and action methods, Using HTML helpers, Handling form post-backs, Data validation Using the Razor View Engine, Getting started with Razor, Razor design goals, Implementing a Razor view, Razor syntax, Accessing Model Data in Razor Views, Deployment, Server Requirements, Configuration Options, Preparing an application for deployment, Deploying to IIS and Windows Azure.

Introduction to ASP.NET MVC:

.NET MVC Helps in building web pages using a model – view – controller design pattern.



The MVC (Model-View-Controller) is an application development pattern or design pattern which separates an application into three main components:

1. Model
2. View
3. Controller

Model: Model is a part of the application which implements the logic for the data domain of the application. It is used to retrieve and store model state in a database such as SQL Server database. It also used for business logic separation from the data in the application.

View: View is a component that forms the application's user interface. It is used to create web pages for the application. An example would be an edit view of a Products table that displays text boxes, drop-down lists and check boxes based on the current state of a Product object.

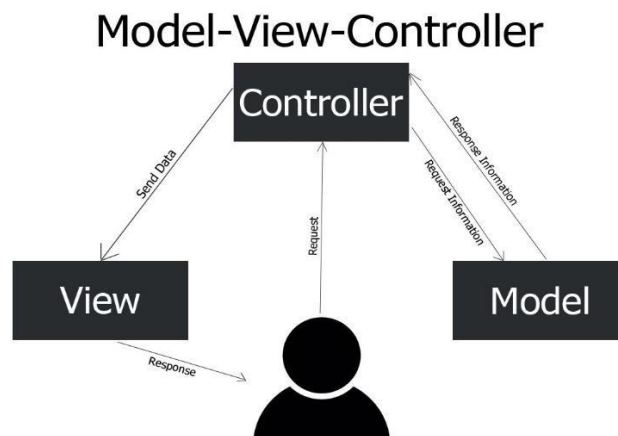
Controller: Controller is the component which handles user interaction. It works with the model and selects the view to render the web page. In an MVC application, the view only displays information whereas the controller handles and responds to the user input and requests.

The role of the Model, View, and Controller

Model: Model is a part of the application which implements the **logic for the data domain** of the application. It is used to **retrieve and store model state** in a database such as SQL Server database. It also used for business logic separation from the data in the application.

View: View is a component that forms the application's **user interface**. It is used to create web pages for the application. An example would be an edit view of a Products table that displays text boxes, drop-down lists and check boxes based on the current state of a Product object.

Controller: Controller is the component which **handles user interaction**. It works with the model and selects the view to render the web page. In an MVC application, the view only displays information whereas the controller handles and responds to the user input and requests.



Key benefits of ASP.NET MVC:

This approach provides the following advantages.

- It manages application complexity by dividing an application into the model, view and controller.
- It is suitable for large scale developer team and web applications.
- It provides high degree of control to the developer over the application behavior.
- It makes it very easy to manage complexity by dividing an application into the Model, the view and the Controller.
- It does not use view state or server-based forms.
- Full control over HTML, JavaScript and CSS.
- It provides better support for Test-Driven Development (TDD).
- It works well for Web applications that are supported by large teams of developers and for webdesigners who need a high degree of control over the application behaviour.
- By default support of **Facebook** and **google** authentication.
- It is easy to manage a large application by dividing it into multiple **Areas**.

Getting started with ASP.NET:

Create a web app

Steps:

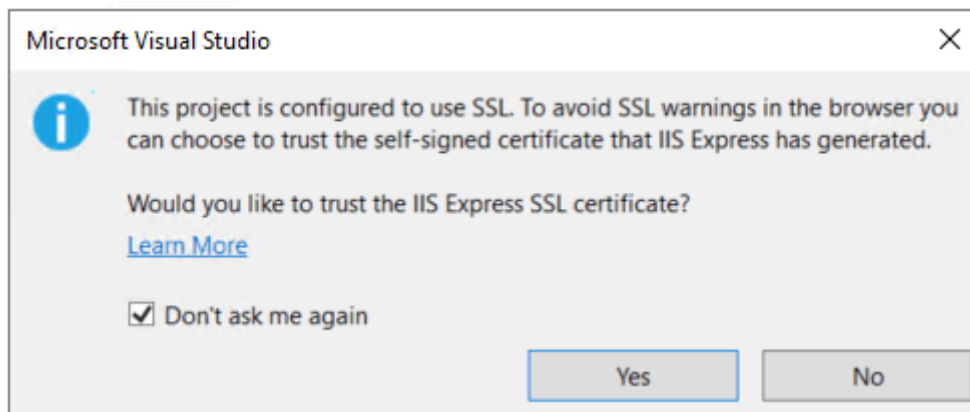
- Start Visual Studio and select **Create a new project**.
- In the **Create a new project** dialog, select **ASP.NET Core Web App (Model-View-Controller)** > **Next**.
- In the **Configure your new project** dialog, enter `MvcMovie` for **Project name**. It's important to name the project `MvcMovie`. Capitalization needs to match each namespace when code is copied.
- Select **Next**.
- In the **Additional information** dialog:
 - Select **.NET 8.0 (Long Term Support)**.
 - Verify that **Do not use top-level statements** is unchecked.
- Select **Create**.

The screenshot shows the 'Additional information' dialog box in Visual Studio. The title bar reads 'Additional information'. Below the title bar, the project type is 'ASP.NET Core Web App (Model-View-Controller)' with tabs for C#, Linux, macOS, Windows, Cloud, Service, and Web. The 'Framework' dropdown is set to '.NET 8.0 (Long Term Support)'. The 'Authentication type' dropdown is set to 'None'. The 'Configure for HTTPS' checkbox is checked. The 'Enable Docker' checkbox is unchecked. The 'Docker OS' dropdown is set to 'Linux'. The 'Do not use top-level statements' checkbox is unchecked. At the bottom right, there are 'Back' and 'Create' buttons. The 'Create' button is highlighted with a red border.

Run the app

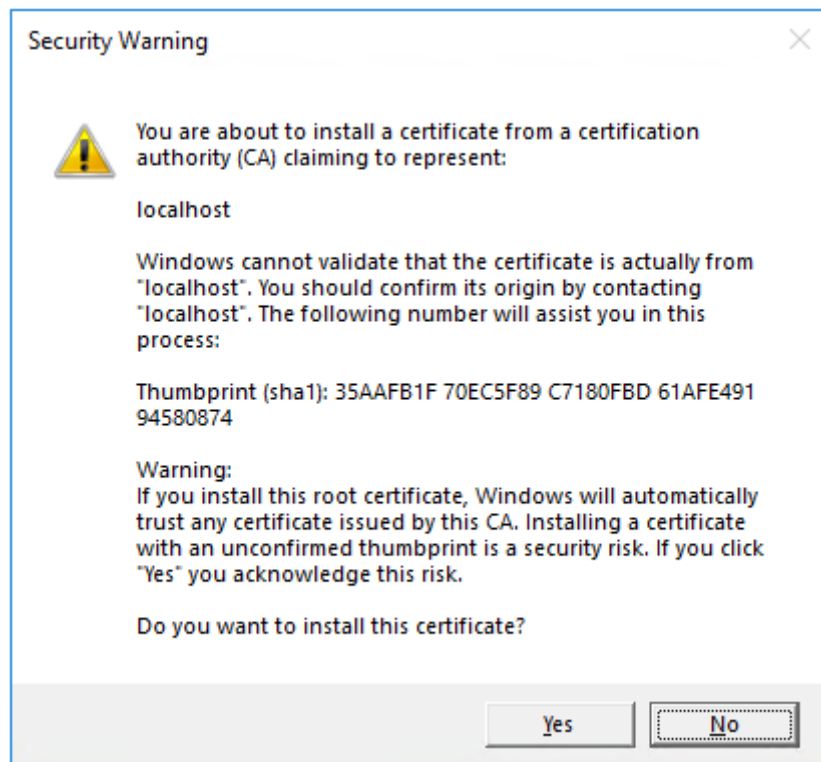
Press `Ctrl+F5` to run the app without the debugger.

Visual Studio displays the following dialog when a project is not yet configured to use SSL:



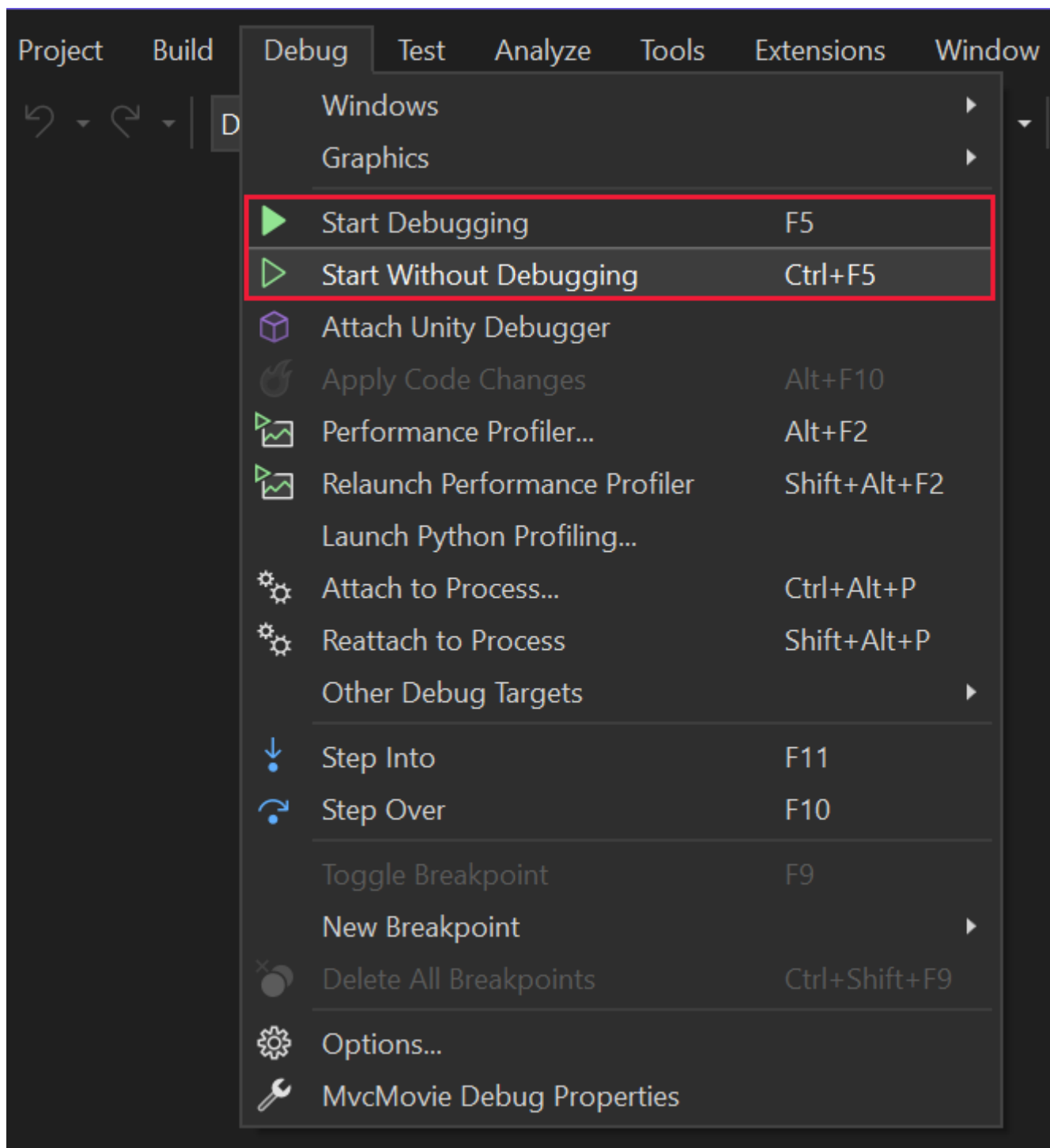
Select **Yes** if you trust the IIS Express SSL certificate.

The following dialog is displayed:

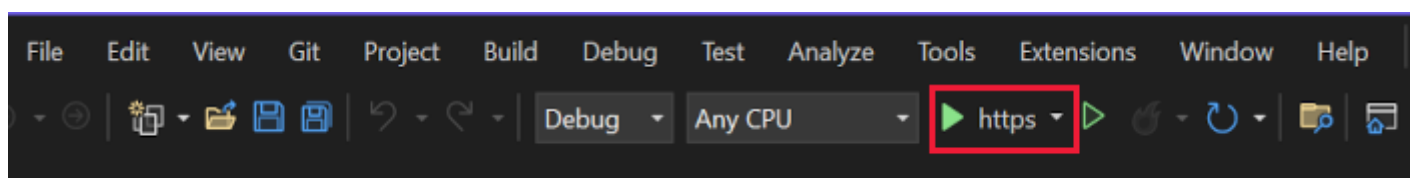


Select **Yes** if you agree to trust the development certificate.

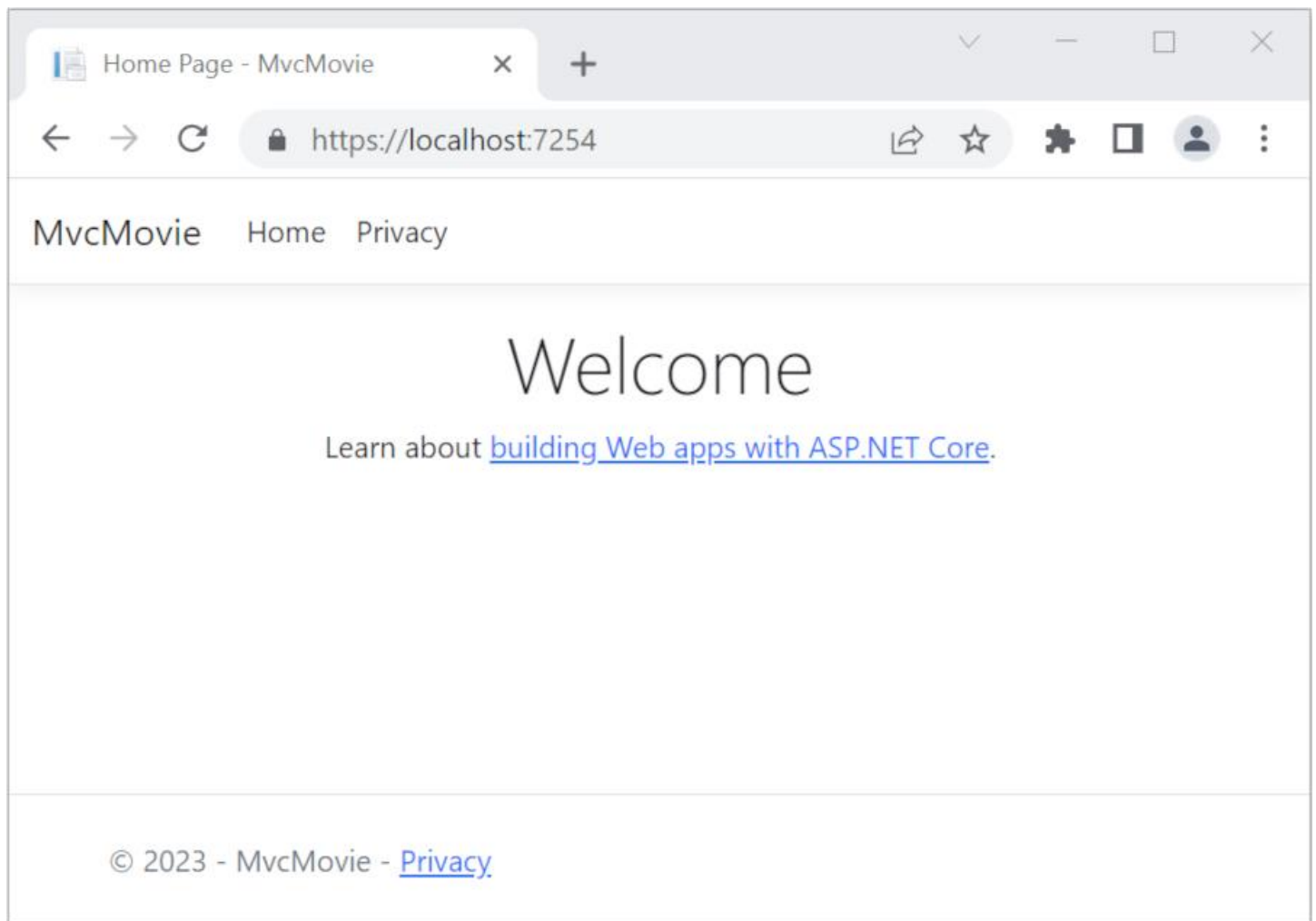
You can launch the app in debug or non-debug mode from the **Debug** menu:



You can debug the app by selecting the **https** button in the toolbar:



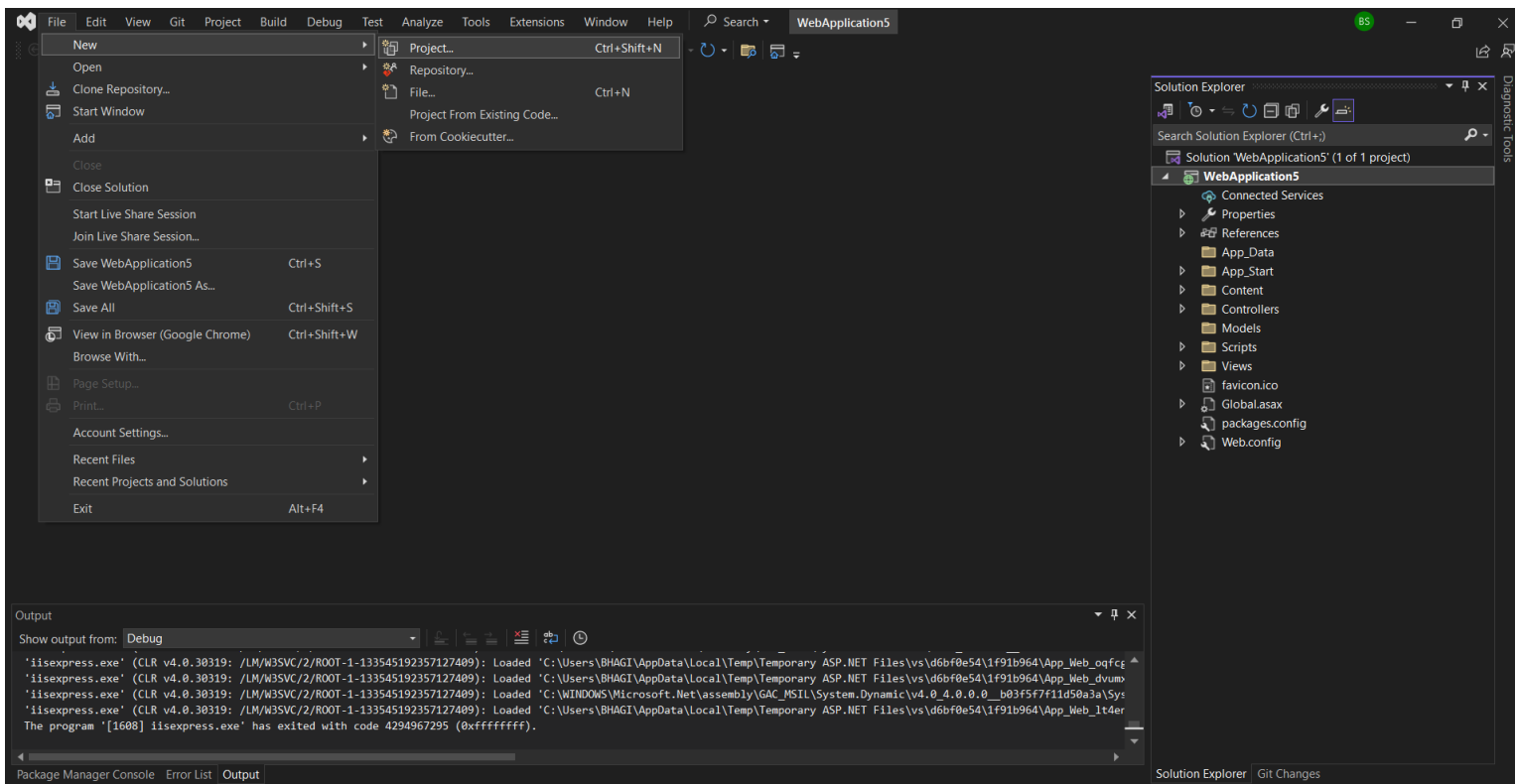
The following image shows the app:



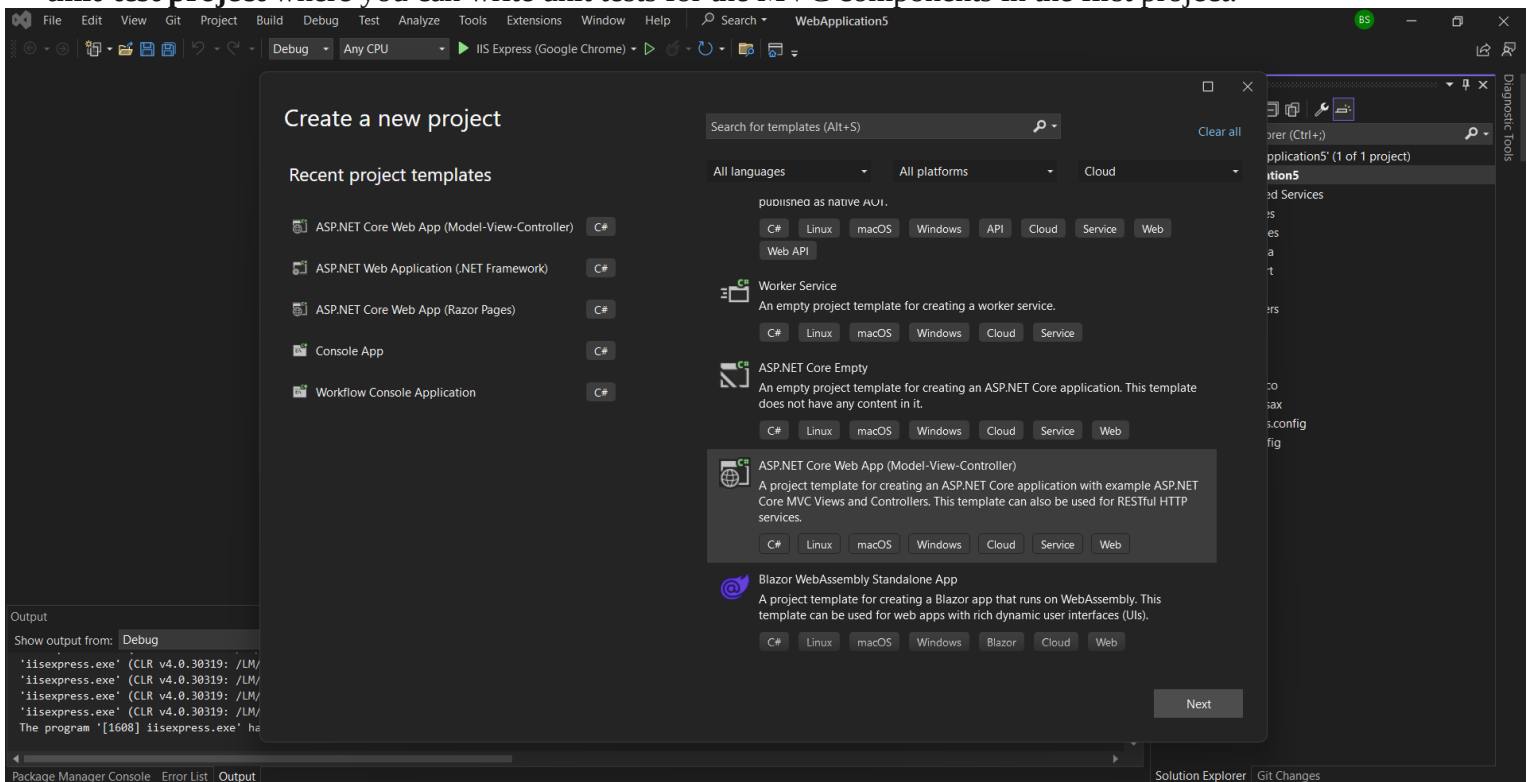
ASP.NET MVC project templates:

The ASP.NET MVC framework includes a Visual Studio project template that helps you create Web applications that are structured to support the MVC pattern. This template creates a new MVC Web application that have the required folders, item templates, and configuration-file entries.

The ASP.NET MVC Web Application project templates are based on the ASP.NET Web Application project template. You select a new ASP.NET MVC project by selecting **New Project** from the **File** menu instead of by selecting **New Web Site**.



When you create a new MVC Web application, Visual Studio gives you the option to create **two projects** at the same time. The first project is a **Web project** where you implement your application. The second project is a **unit-test project** where you can write unit tests for the MVC components in the first project.

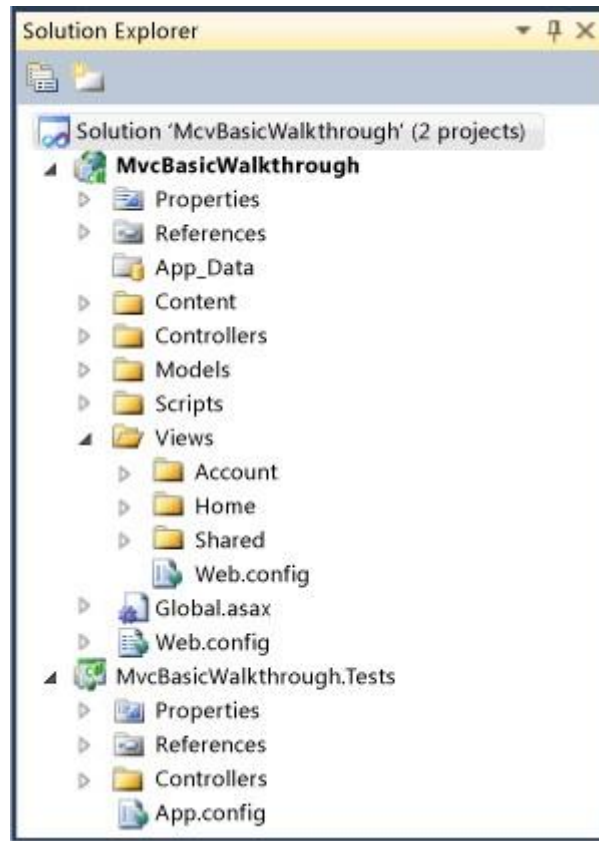


Microsoft Visual Studio Standard Edition and Microsoft Visual Web Developer Express do not support creating unit test projects. Therefore, they do not offer the option to create a test project when you create an MVC application.

You can use any unit-testing framework that is compatible with the .NET Framework in order to test ASP.NET MVC applications.

Understanding the structure of an ASP.NET MVC project:

When you create an ASP.NET MVC Web application project, MVC components are separated based on the project folders shown in the following illustration:



By default, MVC projects include the following folders:

- ❖ App_Data, which is the **physical store** for data. This folder has the same role as it does in ASP.NET Web sites that use Web Forms pages.
- ❖ Content, which is the recommended location to add content files such as **cascading style sheet files, images, and so on**. In general, the Content folder is for static files.
- ❖ Controller is the component which **handles user interaction**. It works with the model and selects the view to render the web page. In an MVC application, the view only displays information whereas the controller handles and responds to the user input and requests. The MVC framework requires the names of all controllers to end with "Controller", such as HomeController, LoginController, or ProductController.

- ❖ Model is a part of the application which implements the logic for the data domain of the application. It is used to retrieve and store model state in a database such as SQL Server database. It is also used for business logic separation from the data in the application.
- ❖ Scripts, which is the recommended location for script files that support the application. By default, this folder contains ASP.NET AJAX foundation files and the jQuery library.
- ❖ The MVC View is a **standard HTML page** that may contain script. It is used to **create web pages** for the application. View is a component that forms the application's user interface. It is used to create web pages for the application. An example would be an edit view of a Products table that displays text boxes, drop-down lists and check boxes based on the current state of a Product object.

Naming conventions:

Mainly three naming convention should be followed in ASP.NET MVC application

Controller: Controller is the component which **handles user interaction**. It works with the model and selects the view to render the web page. In an MVC application, the view only displays information whereas the controller handles and responds to the user input and requests. The MVC framework requires the names of all controllers to end with "Controller", such as HomeController, LoginController, or ProductController.

Model: Model is a part of the application which implements the logic for the data domain of the application. It is used to retrieve and store model state in a database such as SQL Server database. It is also used for business logic separation from the data in the application.

- Model name must be in Singular name. For example, if the database table name is "PersonalDetails" (Plural), the model name should be "PersonalDetail" (Singular).

View: View is a component that forms the application's user interface. It is used to create web pages for the application. An example would be an edit view of a Products table that displays text boxes, drop-down lists and check boxes based on the current state of a Product object.

There should be a view folder inside ~/Views folder corresponding to each controller. Like for **PersonalDetailsController**, there should be a folder **~/Views/PersonalDetails**, similarly for **EmployeesController**, there should be **~/Views/Employees** folder.

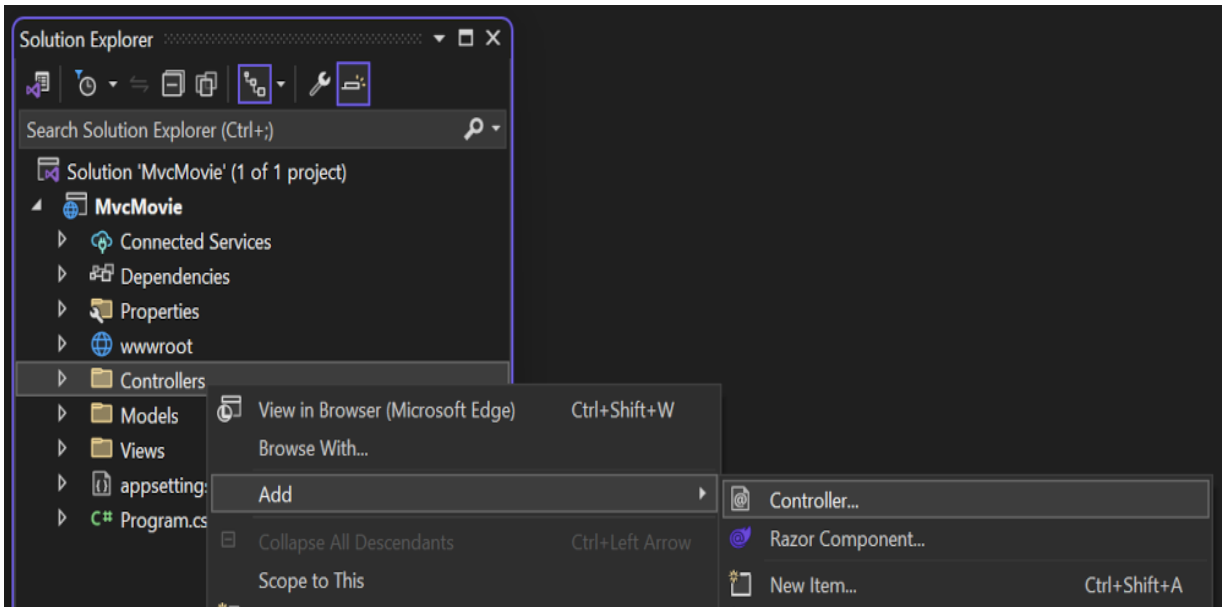
Create Controllers with ASP.NET core MVC/ Creating a ASP.NET MVC 4 Application:

- **Controllers:** Classes that:
 - Handle browser requests.
 - Retrieve model data.
 - Call view templates that return a response.

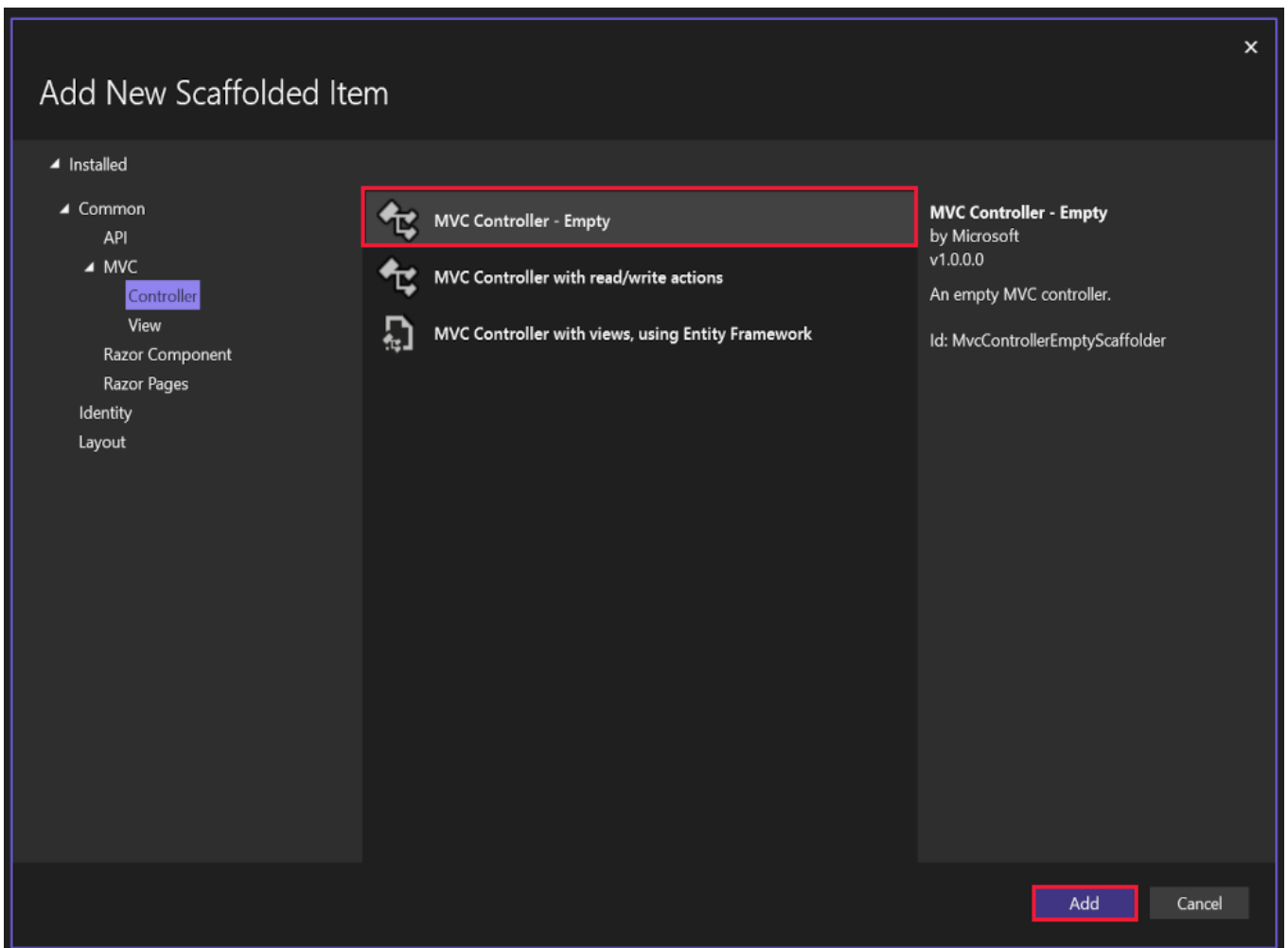
Steps:

- Start Visual Studio and select **Create a new project**.
- In the **Create a new project** dialog, select **ASP.NET Core Web App (Model-View-Controller) > Next**.

- In the **Configure your new project** dialog, enter `MvcMovie` for **Project name**. It's important to name the project `MvcMovie`. Capitalization needs to match each namespace when code is copied.
- Select **Next**.
- In the **Additional information** dialog:
 - Select **.NET 8.0 (Long Term Support)**.
 - Verify that **Do not use top-level statements** is unchecked.
- Select **Create**.
- Click on **View> Solution Explorer**, right-click **Controllers > Add > Controller**.



- In the **Add New Scaffolded Item** dialog box, select **MVC Controller - Empty > Add**.



- In the **Add New Item - MvcMovie** dialog, enter *HelloWorldController.cs* and select **Add**.

Replace the contents of `Controllers/HelloWorldController.cs` with the following code:

```
using Microsoft.AspNetCore.Mvc;
using System.Text.Encodings.Web;

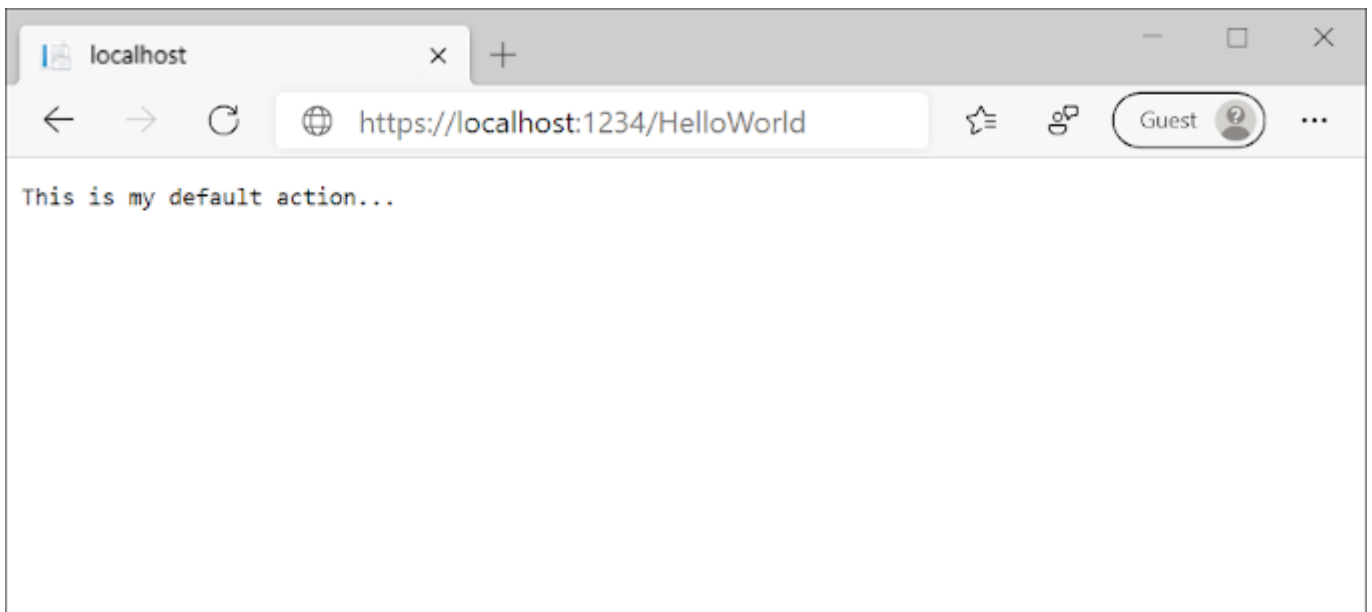
namespace MvcMovie.Controllers;

public class HelloWorldController : Controller
{
    //
    // GET: /HelloWorld/
    public string Index()
    {
        return "This is my default action...";
    }
    //
    // GET: /HelloWorld/Welcome/
    public string Welcome()
    {
        return "This is the Welcome action method...";
    }
}
```

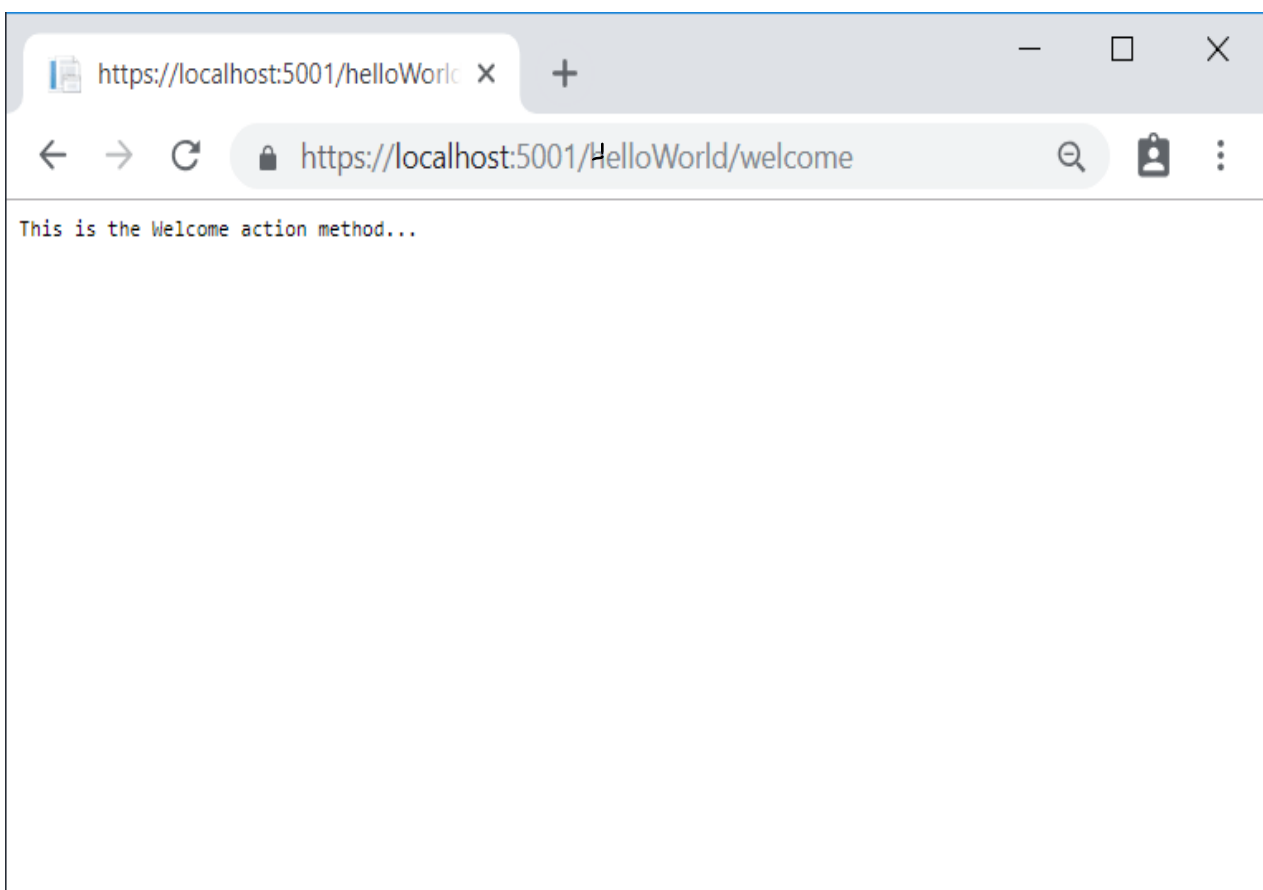
Every public method in a controller is callable as an HTTP endpoint. In the sample above, both methods return a string. Note the comments preceding each method.

An HTTP endpoint:

- Is a targetable URL in the web application, such as `https://localhost:5001/HelloWorld`.
- Combines:
 - The protocol used: HTTPS.
 - The network location of the web server, including the TCP port: `localhost:5001`.
 - The target URI: `HelloWorld`.
- The first comment states this is an [HTTP GET](#) method that's invoked by appending `/HelloWorld/` to the base URL.
- The second comment specifies an [HTTP GET](#) method that's invoked by appending `/HelloWorld/Welcome/` to the URL. Later on in the tutorial, the scaffolding engine is used to generate HTTP POST methods, which update data.
- Run the app without the debugger by pressing `Ctrl+F5` (Windows) or `⌘+F5` (macOS).
- Append `/HelloWorld` to the path in the address bar. The `Index` method returns a string.



- Browse to: `https://localhost:{PORT}/HelloWorld/Welcome`. Replace `{PORT}` with your port number.
- The `Welcome` method runs and returns the string `This is the Welcome action method...`. For this URL, the controller is `HelloWorld` and `Welcome` is the action method. You haven't used the `[Parameters]` part of the URL yet.



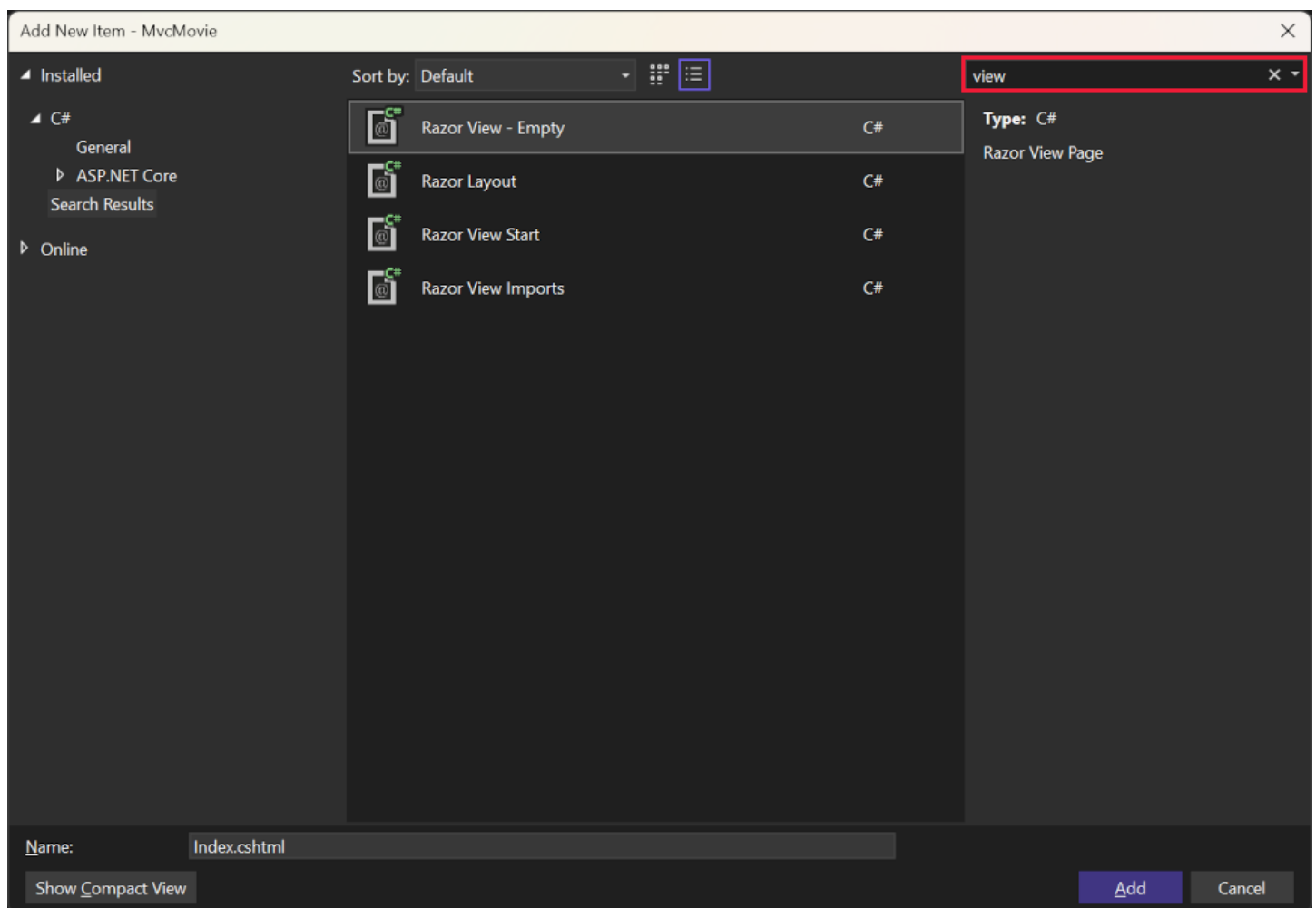
Creating views with ASP.NET Core MVC app / Implementing a razor view:

This is the continuation process of creating a controller. After creating the controller follow the below steps.

Currently, the `Index` method returns a string with a message in the controller class. In the `HelloWorldController` class, replace the `Index` method with the following code:

```
public IActionResult Index()
{
    return View();
}
```

- Click on **View> Solution Explorer**.
- Right-click on the *Views* folder, and then **Add > New Folder** and name the folder *HelloWorld*.
- Right-click on the *Views/HelloWorld* folder, and then **Add > New Item**.
- In the **Add New Item** dialog select **Show All Templates**.
- In the **Add New Item - MvcMovie** dialog:
 - In the search box in the upper-right, enter *view*
 - Select **Razor View - Empty**
 - Keep the **Name** box value, *Index.cshtml*.
 - Select **Add**



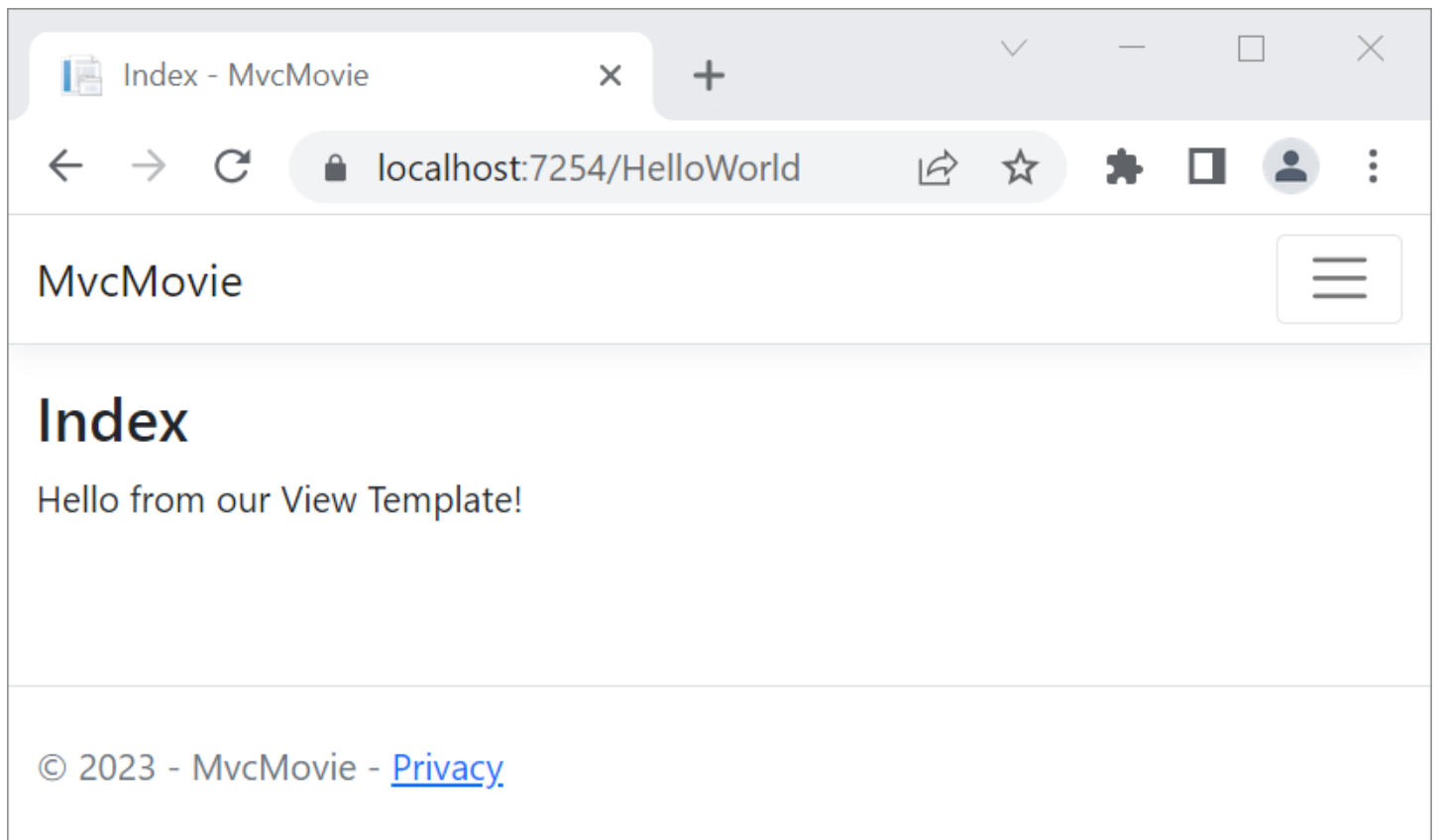
Replace the contents of the *Views/HelloWorld/Index.cshtml* Razor view file with the following:

```
@{
    ViewData["Title"] = "Index";
}

<h2>Index</h2>

<p>Hello from our View Template!</p>
```

- Navigate to <https://localhost:{PORT}/HelloWorld>:
- The following image shows the string "Hello from our View Template!" hard-coded in the view:



Creating Models with ASP.NET Core MVC/ Accessing Model Data in Razor Views:

- This is the continuation process of creating a View. After creating the View follow the below steps.
- Click on **View> Solution Explorer**.
- Right-click the *Models* folder > **Add** > **Class**. Name the file *Movie.cs*.

Add a data model class

- Update the *Models/Movie.cs* file with the following code:

```
using System.ComponentModel.DataAnnotations;

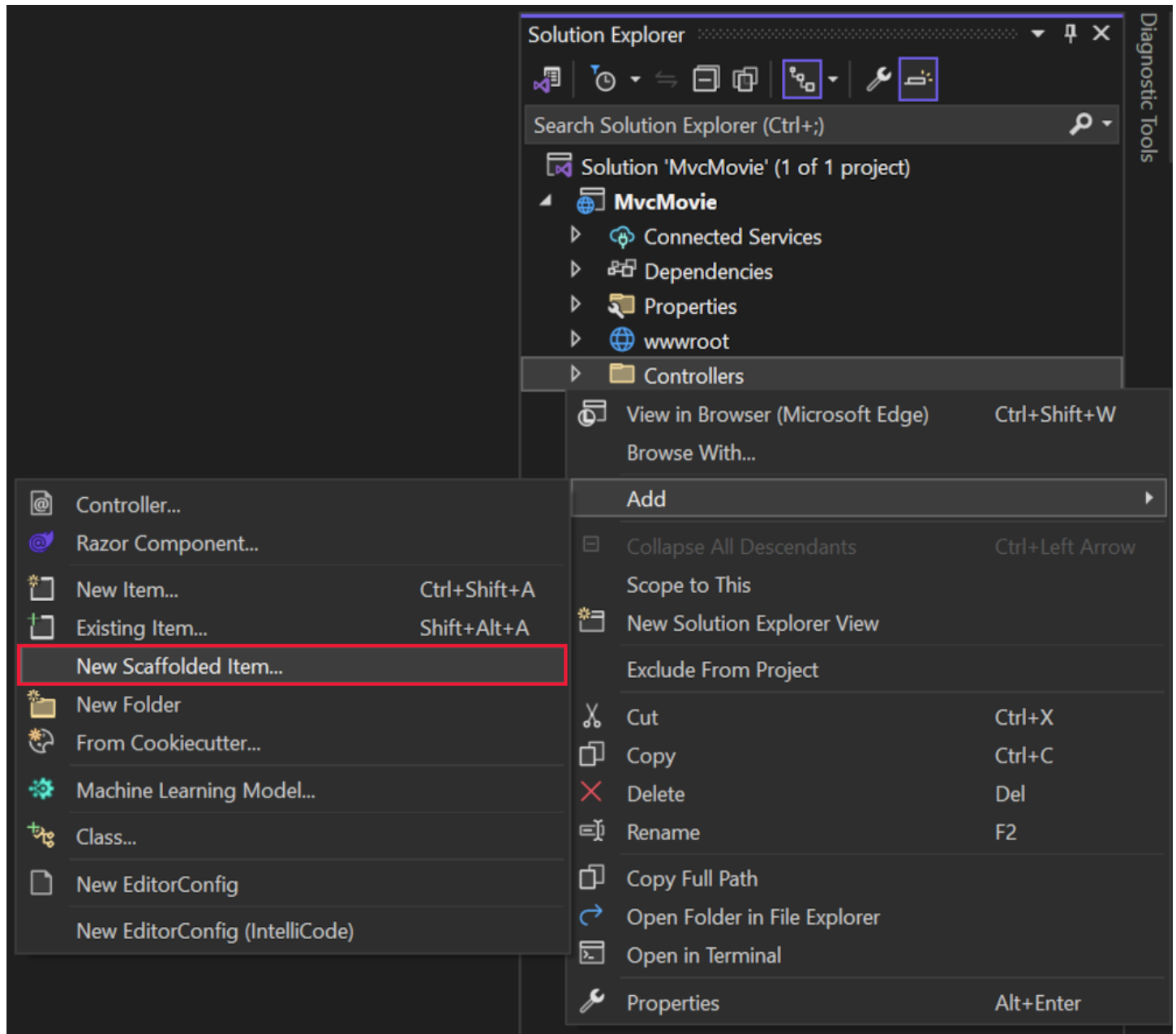
namespace MvcMovie.Models;

public class Movie
{
    public int Id { get; set; }
    public string? Title { get; set; }
    [DataType(DataType.Date)]
    public DateTime ReleaseDate { get; set; }
    public string? Genre { get; set; }
    public decimal Price { get; set; }
}
```

Scaffold movie pages

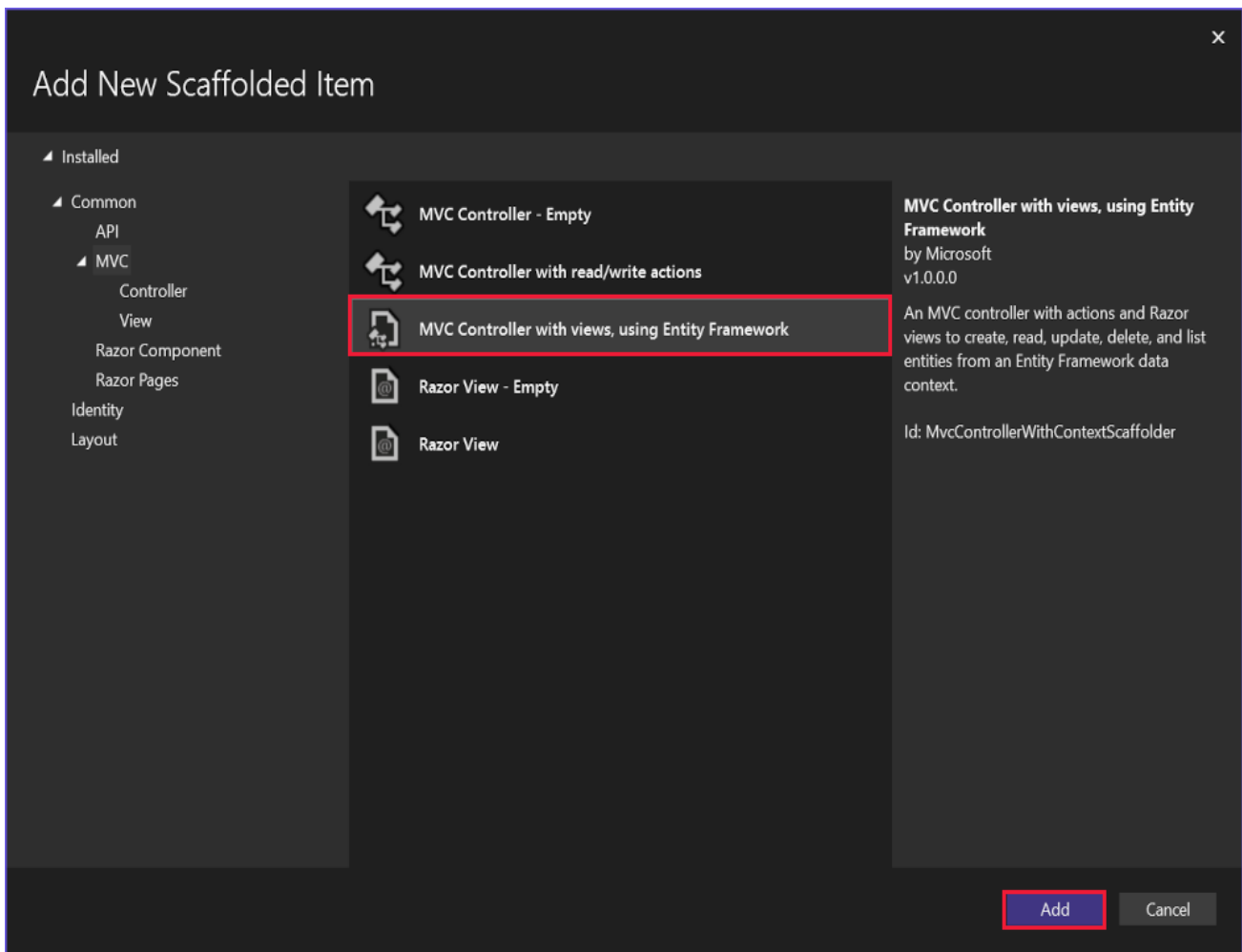
Use the scaffolding tool to produce Create, Read, Update, and Delete (CRUD) pages for the movie model.

In **Solution Explorer**, right-click the *Controllers* folder and select **Add > New Scaffolded Item**.



In the **Add New Scaffolded Item** dialog:

- In the left pane, select **Installed > Common > MVC**.
- Select **MVC Controller with views, using Entity Framework**.
- Select **Add**.



Complete the **Add MVC Controller with views, using Entity Framework** dialog:

- In the **Model class** drop down, select **Movie (MvcMovie.Models)**.
- In the **Data context class** row, select the + (plus) sign.
 - In the **Add Data Context** dialog, the class name *MvcMovie.Data.MvcMovieContext* is generated.
 - Select **Add**.
- In the **Database provider** drop down, select **SQL Server**.
- **Views** and **Controller name**: Keep the default.
- Select **Add**.

×

Add MVC Controller with views, using Entity Framework

Model class

Movie (MvcMovie.Models)

DbContext class

MvcMovie.Data.MvcMovieContext

+

Database provider

SQL Server

Views

☒ Generate views

☒ Reference script libraries

☒ Use a layout page

...

(Leave empty if it is set in a Razor `_viewstart` file)

Controller name

MoviesController

Add

Cancel

Initial migration:

From the **Tools** menu, select **NuGet Package Manager** > **Package Manager Console** .

In the Package Manager Console (PMC), enter the following commands:

Add-Migration InitialCreate

Update-Database

Test the app

Run the app and select the Movie App link.

Create - MvcMovies

localhost:7057/Movies/Create

MvcMoviesHomePrivacy

Create

Movie

Title

ReleaseDate

dd-mm-yyyy

The ReleaseDate field is required.

Genre

Price

The Price field is required.

Create

[Back to List](#)

© 2024 - MvcMovies - [Privacy](#)

After entering the details:

Index - MvcMovies

localhost:7057/Movies

MvcMoviesHomePrivacy

Index

[Create New](#)

Title	ReleaseDate	Genre	Price	
abc	07-03-2024	xy	1000.00	Edit Details Delete

Creating strongly-typed views:

To create "strongly typed" view, trigger the create a view dialog and choose any template other than "Empty (without model)". You should now be able to choose your Model from the "Model Class" dropdown. Doing this will create the equivalent of a strongly typed view.

In order to create a strongly typed view in ASP.NET MVC application, we need to pass the model object as a parameter to the View() extension method. The Controller base class provide us the following four overloaded versions of View() extension method which we can use to pass the model data from the controller action method to a view.

```
protected internal ViewResult View(object model);  
protected internal virtual ViewResult View(IView view, object model);  
protected internal ViewResult View(string viewName, object model);  
protected internal virtual ViewResult View(string viewName, string masterName, object model);
```

Understanding URLs:

Every document on the Web has a unique address. This address is known as **Uniform Resource Locator (URL)**.

Several HTML/XHTML tags include a URL attribute value, including hyperlinks, inline images, and forms. All of them use the same syntax to specify the location of a web resource. That's why it is known as a Uniform Resource Locator.

A URL is composed of different parts, some mandatory and others optional. The most important parts are highlighted on the URL below (details are provided in the following sections):

The diagram shows the URL `http://www.example.com:80/path/to/myfile.html?key1=value1&key2=value2#SomewhereInTheDocument` with its components highlighted in different colors and labeled with arrows:

- Authority**: A bracket above `www.example.com:80`.
- Scheme**: `http://` (green box).
- Domain Name**: `www.example.com` (teal box).
- Port**: `:80` (yellow box).
- Path to the file**: `/path/to/myfile.html` (orange box).
- Parameters**: `?key1=value1&key2=value2` (blue box).
- Anchor**: `#SomewhereInTheDocument` (purple box).

- ❖ The first part of the URL is the *scheme*, which indicates the protocol that the browser must use to request the resource .

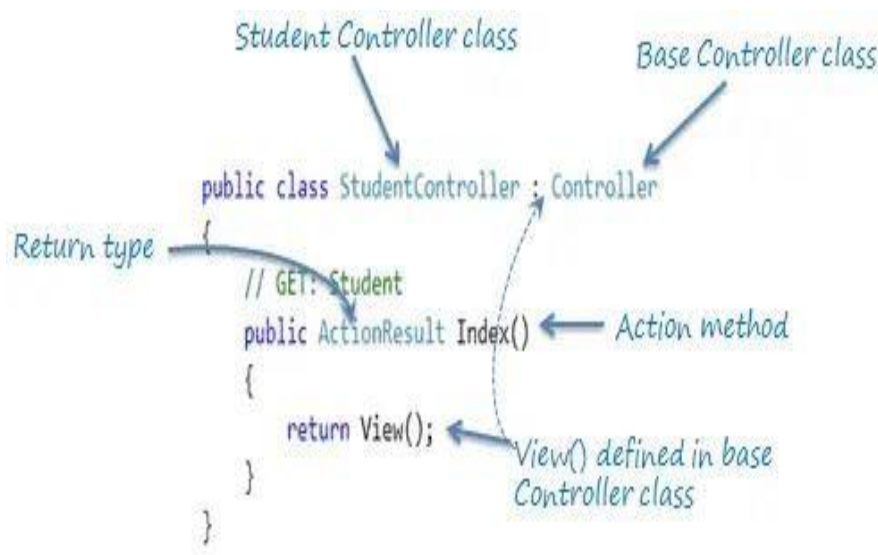
- ❖ Next follows the *authority*, which is separated from the scheme by the character pattern `://`. If present the authority includes both the *domain* (e.g. `www.example.com`) and the *port* (80), separated by a colon:
- ❖ `/path/to/myfile.html` is the path to the resource on the Web server.
- ❖ `?key1=value1&key2=value2` are extra parameters provided to the Web server. Those parameters are a list of key/value pairs separated with the `&` symbol.
- ❖ An anchor represents a sort of "bookmark" inside the resource, giving the browser the directions to show the content located at that "bookmarked" spot.

Action methods:

All the public methods of the `Controller` class are called `Action` methods. They are like any other normal methods with the following restrictions:

1. Action method must be public. It cannot be private or protected
2. Action method cannot be overloaded
3. Action method cannot be a static method.

The following illustrates the `Index()` action method in the `StudentController` class.



As you can see in the above figure, the `Index()` method is public, and it returns the `ActionResult` using the `View()` method. The `View()` method is defined in the `Controller` base class, which returns the appropriate `ActionResult`.

Using HTML helpers:

`HtmlHelper` is a class which is introduced in MVC 2. It is used to create HTML controls programmatically. It provides built-in methods to generate controls on the view page.

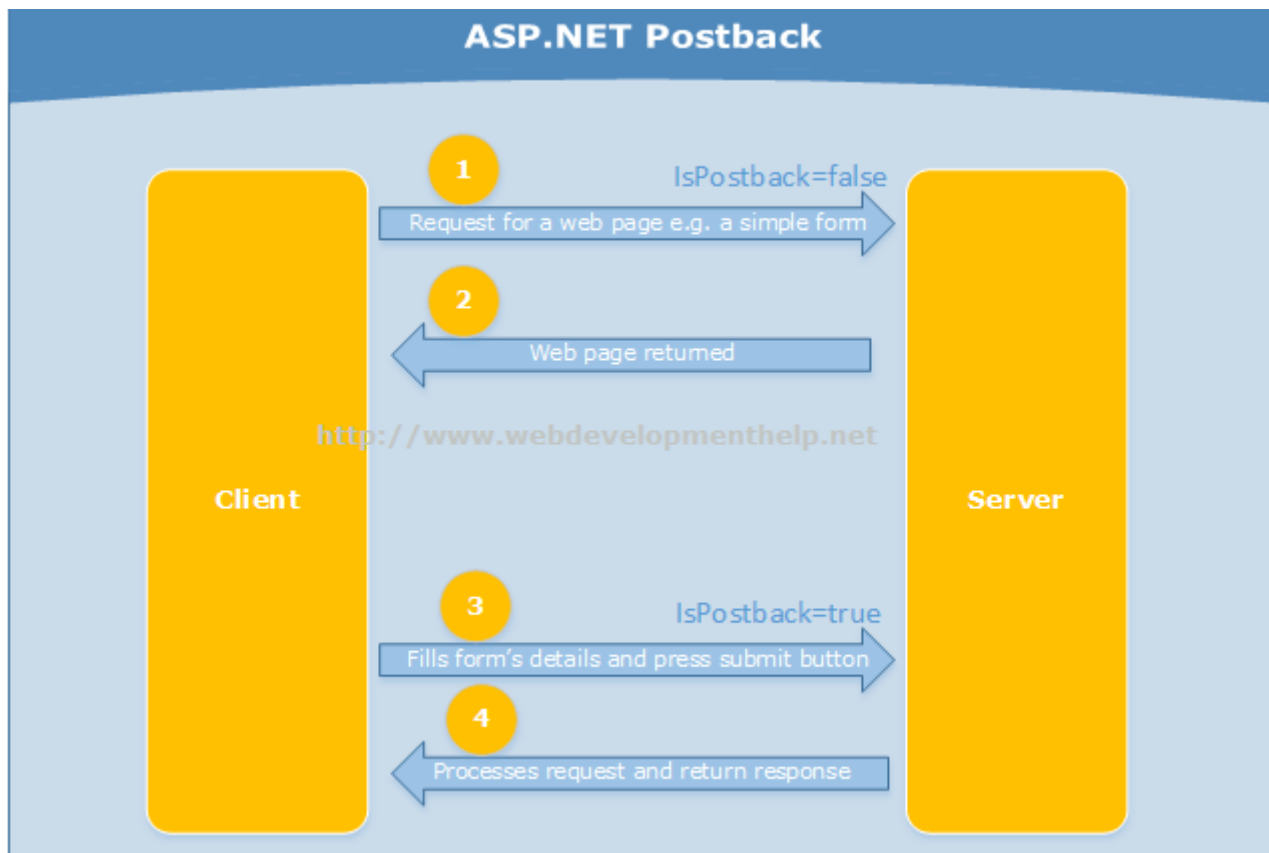
The following table lists the `HtmlHelper` methods and HTML control each method renders.

Extension Method	Strongly Typed Method	Html Control
<code>Html.ActionLink()</code>	NA	<code><a></code>
<code>Html.TextBox()</code>	<code>Html.TextBoxFor()</code>	<code><input type="text"></code>
<code>Html.TextArea()</code>	<code>Html.TextAreaFor()</code>	<code><input type="text"></code>
<code>Html.CheckBox()</code>	<code>Html.CheckBoxFor()</code>	<code><input type="checkbox"></code>

Html.RadioButton()	Html.RadioButtonFor()	<input type="radio">
Html.DropDownList()	Html.DropDownListFor()	<select> <option> </select>
Html.ListBox()	Html.ListBoxFor()	multi-select list box: <select>
Html.Hidden()	Html.HiddenFor()	<input type="hidden">
Html.Password()	Html.PasswordFor()	<input type="password">
Html.Display()	Html.DisplayFor()	HTML text: ""
Html.Label()	Html.LabelFor()	<label>
Html.Editor()	Html.EditorFor()	Generates Html controls based on data type of specified model property e.g. textbox for string property, numeric field for int, double or other numeric type.

Handling form post-backs:

PostBack is the name given to the process of submitting an ASP.NET page to the server for processing. PostBack is done if certain credentials of the page are to be checked against some sources (such as verification of username and password using database). This is something that a client machine is not able to accomplish and thus these details have to be 'posted back' to the server.



ASP.NET also adds two additional hidden input fields that are used to pass information back to the server. This information consists of ID of the Control that raised the event and any additional information if needed. These fields will empty initially as shown below,

```
1. <input type="hidden" name="__EVENTTARGET" id="__EVENTTARGET" value="" />
2. <input type="hidden" name="__EVENTARGUMENT" id="__EVENTARGUMENT" value="" />
```

The `_doPostBack()` function has the responsibility for setting these values with the appropriate information about the event and the submitting the form. The `_doPostBack()` function is shown below:

```
1. <script language="text/javascript">
2.
3.     function __doPostBack(eventTarget, eventArgument) {
4.         if (!theForm.onsubmit || (theForm.onsubmit() != false)) {
5.             theForm.__EVENTTARGET.value = eventTarget;
6.             theForm.__EVENTARGUMENT.value = eventArgument;
7.             theForm.submit();
8.         }
9.     }
10. </script>
```

ASP.NET generates the `_doPostBack()` function automatically, provided at least one control on the page uses automatic postbacks.

Data validation Using the Razor View Engine:

Open the *Movie.cs* file. Notice the [System.ComponentModel.DataAnnotations](#) namespace does not contain `System.Web`. `DataAnnotations` provides a built-in set of validation attributes that you can apply declaratively to any class or property. (It also contains formatting attributes like [DataType](#) that help with formatting and don't provide any validation.)

Now update the `Movie` class to take advantage of the built-in [Required](#), [StringLength](#), [RegularExpression](#), and [Range](#) validation attributes. Replace the `Movie` class with the following:

```
using System;

using System.ComponentModel.DataAnnotations;

using System.ComponentModel.DataAnnotations.Schema;

namespace MvcMovie.Models;

public class Movie
{
    public int Id { get; set; }

    [StringLength(10, MinimumLength = 3)]
    [Required]
    public string? Title { get; set; }

    [Display(Name = "Release Date")]
    [DataType(DataType.Date)]
    public DateTime ReleaseDate { get; set; }

    [Range(150, 1000)]
    [DataType(DataType.Currency)]
    [Column(TypeName = "decimal(16, 2)")]
    public decimal Price { get; set; }

    [RegularExpression(@"^[A-Z]+[a-zA-Z\s]*$")]
    [Required]
    [StringLength(30)]
    public string? Genre { get; set; }
```



```
[RegularExpression(@"^[1-5\s-]*$")]
```

```
[StringLength(1)]
```

```
[Required]
```

```
public string? Rating { get; set; }
```

```
}
```

After debugging the code the output looks like the below:

MVCMovie Home Privacy

Create Movie

Title

The Title field is required.

Release Date

dd-mm-yyyy

The Release Date field is required.

Price

10000

The field Price must be between 1 and 100.

Genre

1

The field Genre must match the regular expression '[A-Z]+[a-zA-Z\s]*'.

Rating

a

The field Rating must match the regular expression '^[1-5\s-]*\$'.

Create

[Back to List](#)

© 2024 - MVCMovie - [Privacy](#)

Getting started with Razor:

- Start Visual Studio and select **New project**.
- In the **Create a new project** dialog, select **ASP.NET Core Web App (Razor Pages)** > **Next**.
- In the **Configure your new project** dialog, enter RazorPagesMovie for **Project name**. It's important to name the project **RazorPagesMovie**, including matching the capitalization, so the namespaces will match when you copy and paste example code.
- Select **Next**.
- In the **Additional information** dialog:
 - Select **.NET 8.0 (Long Term Support)**.
 - Verify: **Do not use top-level statements** is unchecked.
- Select **Create**.

Additional information

ASP.NET Core Web App (Razor Pages)

C#LinuxmacOSWindowsCloudServiceWeb

Framework ⓘ

.NET 8.0 (Long Term Support)

Authentication type ⓘ

None

☒ Configure for HTTPS ⓘ☐ Enable Docker ⓘ

Docker OS ⓘ

Linux

☐ Do not use top-level statements ⓘ

Back

Create

The following starter project is created:

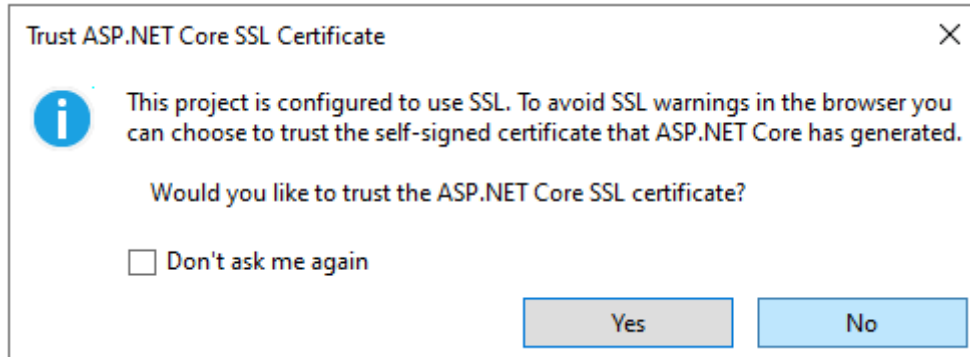
The screenshot shows the Visual Studio Solution Explorer for a project named 'RazorPagesMovie'. The project is expanded, revealing the following structure:

- Connected Services**
- Dependencies**
- Properties**
- wwwroot**
- Pages**
 - Shared**
 - _ViewImports.cshtml**
 - _ViewStart.cshtml**
 - Error.cshtml**
 - Index.cshtml**
 - Privacy.cshtml**
- appsettings.json**
- Program.cs**

Run the app

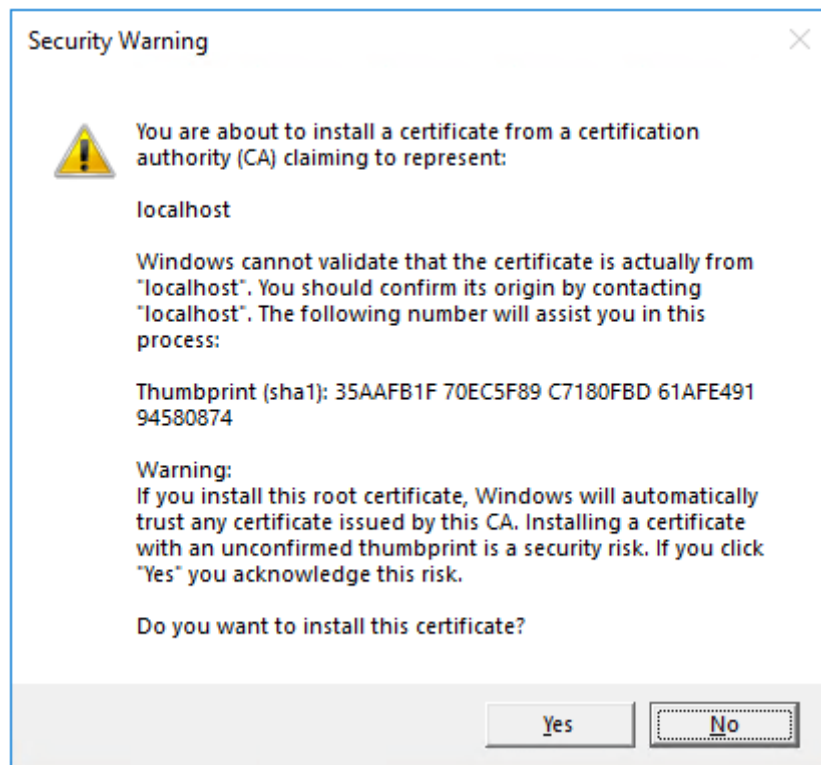
Select **RazorPagesMovie** in **Solution Explorer**, and then press **Ctrl+F5** to run the app without the debugger.

Visual Studio displays the following dialog when a project is not yet configured to use SSL:



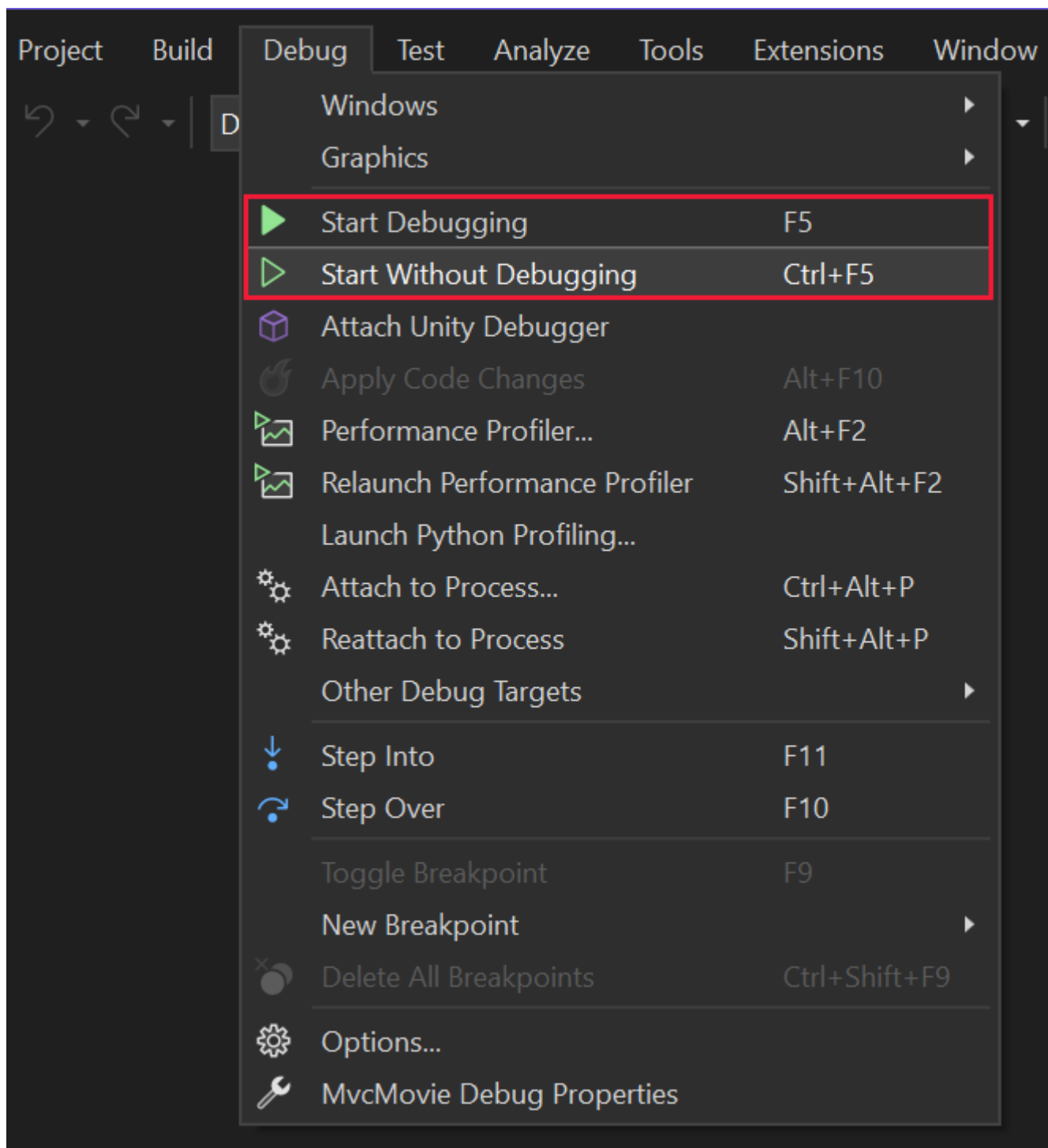
Select **Yes** if you trust the IIS Express SSL certificate.

The following dialog is displayed:

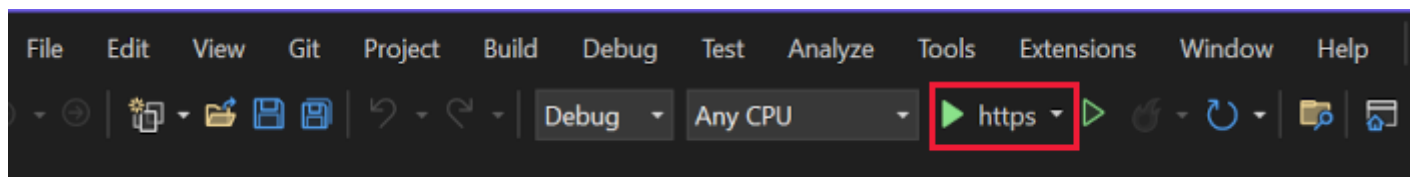


Select **Yes** if you agree to trust the development certificate.

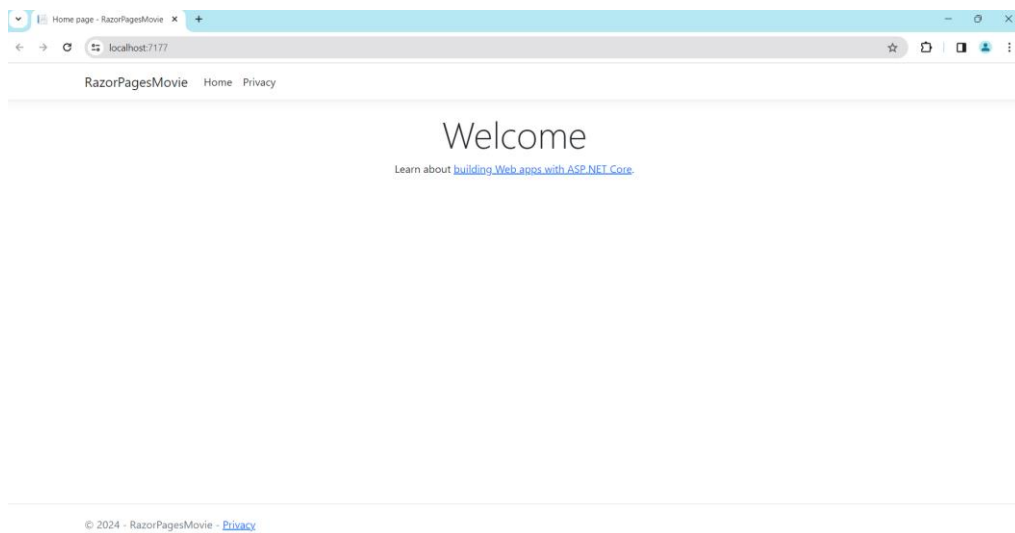
You can launch the app in debug or non-debug mode from the **Debug** menu:



You can debug the app by selecting the **https** button in the toolbar:



The following image shows the app:



Razor design goals:

- **Compact, Expressive, and Fluid:** Razor minimizes the number of characters and keystrokes required in a file, and enables a fast, fluid coding workflow. Unlike most template syntaxes, you do not need to interrupt your coding to explicitly denote server blocks within your HTML. The parser is smart enough to infer this from your code. This enables a really compact and expressive syntax which is clean, fast and fun to type.
- **Easy to Learn:** Razor is easy to learn and enables you to quickly be productive with a minimum of concepts. You use all your existing language and HTML skills.
- **Is not a new language:** We consciously chose not to create a new imperative language with Razor. Instead we wanted to enable developers to use their existing C#/VB (or other) language

skills with Razor, and deliver a template markup syntax that enables an awesome HTML construction workflow with your language of choice.

- **Works with any Text Editor:** Razor doesn't require a specific tool and enables you to be productive in any plain old text editor (notepad works great).
- **Has great Intellisense:** While Razor has been designed to not require a specific tool or code editor, it will have awesome statement completion support within Visual Studio. We'll be updating Visual Studio 2010 and Visual Web Developer 2010 to have full editor intellisense for it.
- **Unit Testable:** The new view engine implementation will support the ability to unit test views (without requiring a controller or web-server, and can be hosted in any unit test project – no special app-domain required)

Razor syntax:

Razor is one of the view engines supported in ASP.NET MVC. Razor allows you to write a mix of HTML and server-side code using C# or Visual Basic. Razor view with visual basic syntax has `.vbhtml` file extension and C# syntax has `.cshtml` file extension.

Razor syntax has the following Characteristics:

- **Compact:** Razor syntax is compact, enabling you to minimize the number of characters and keystrokes required to write code.
- **Easy to Learn:** Razor syntax is easy to learn where you can use your familiar language C# or Visual Basic.
- **Intellisense:** Razor syntax supports statement completion within Visual Studio.

Start with `@` symbol to write server-side C# or VB code with HTML code. For example, write `@Variable_Name` to display the value of a server-side variable, e.g., `DateTime.Now` returns the current date and time. So, write `@DateTime.Now` to display the current date and time, as shown below. A single line expression does not require a semicolon at the end of the expression.

C# Razor Syntax

```
<h1>Razor syntax demo</h1>  
<h2>@DateTime.Now.ToShortDateString()</h2>
```

Output:

Razor syntax demo

18-04-2023

Deployment:

There are two categories of ASP.NET deployment:

- **Local deployment :** In this case, the entire application is contained within a virtual directory and all the contents and assemblies are contained within it and available to the application.
- **Global deployment :** In this case, assemblies are available to every application running on the server.

There are different techniques used for deployment, however, we will discuss the following most common and easiest ways of deployment:

- XCOPY deployment
- Copying a Website
- Creating a set up project

XCOPY Deployment

XCOPY deployment simply copies the application file to the production server and sets a virtual directory there. You need to set a virtual directory using the Internet Information Manager Microsoft Management Console

Copying a Website

The Copy Web Site option is available in Visual Studio. It is available from the Website -> Copy Web Site menu option. This menu item allows copying the current web site to another local or remote location. It is a sort of integrated FTP tool.

Creating a Setup Project

In this method, you use Windows Installer and package your web applications so it is ready to deploy on the production server. Visual Studio allows you to build deployment packages. Let us test this on one of our existing project, say the data binding project.

Server requirements:

Minimum Server Requirements (Database & Web Server)	
Item	Specification
Processor	Intel Xenon or comparable multi-core CPU
Memory	16GB RAM (recommended 4 GB free)
Storage	10Gb disk space for CMS Online application (including website) 10Gb disk space for SQL Server databases
Operating System	The most recent version of Windows Server as possible.
Web Server	IIS 7.0 or higher .NET framework 4.6 asp.net MVC 5 enabled aspPDF (PDF printing client provided by vendor)
Database Server	MS SQL Server 2017 or higher

Configuration option:

ASP.NET Core supports many methods of configuration. In ASP.NET Core application, the configuration is stored in name-value pairs and it can be read at runtime from various parts of

the application. The name–value pairs may be grouped into a multi–level hierarchy. The application configuration data may come from

- File, such as JSON, XML, INI
- Environment variables
- Command Line arguments
- An in–memory collection
- Custom providers

Configuration System in ASP.NET Core is restructured from the older version of ASP.NET. The older version uses "System.Configuration" namespace and is able to read XML configuration files such as web.config. The new configuration model can be accessed to the key/value–based settings and it can retrieve various sources, such as JSON, XML, and INI.

Generally, the configuration remains in simple key/value structure or might in hierarchical structure when using external files. Consider the following appsetting.json file for this example.

appSetting.json

```
1. {
2.   "status" : "This Status read from appSettings.json file",
3.   "ConnectionStrings": {
4.     "DefaultConnection": "Server=.\sqlexpress;Database=Test;Trusted
   _Connection=True;MultipleActiveResultSets=true"
5.   }
6. }
```

To read the JSON configuration file, we need to add "Microsoft.Extensions.Configuration" and "Microsoft.Extensions.Configuration.Json" libraries as a dependency. Following two lines need to be added in package.json file in dependency section.

```
1. "Microsoft.Extensions.Configuration": "1.0.0",
2. "Microsoft.Extensions.Configuration.Json": "1.0.0"
```

Preparing an application for deployment:

1. Coordinate between development and operations teams

Communication between the development and operations teams is key to a smooth–running deployment plan. Everyone needs to know what everyone else is working on. This allows them to anticipate potential problems and have troubleshooting methods ready if needed.

2. Conduct regular testing

Regular testing is the best way to ensure new software works well on different computer platforms and operating systems. Have assigned employees test the product at planned intervals in the deployment plan. Just before launch, conduct another thorough test to help ensure implementation occurs with no issues.

3. Determine the right strategy for the launch

Before launch, determine whether software updates for users outside of the company will happen automatically or if they will need to update their systems manually. You may also want

to decide whether you are planning to release the software in stages or all at once. Detailing this information in the deployment plan can help your team fully prepare for the launch.

4. Monitor performance

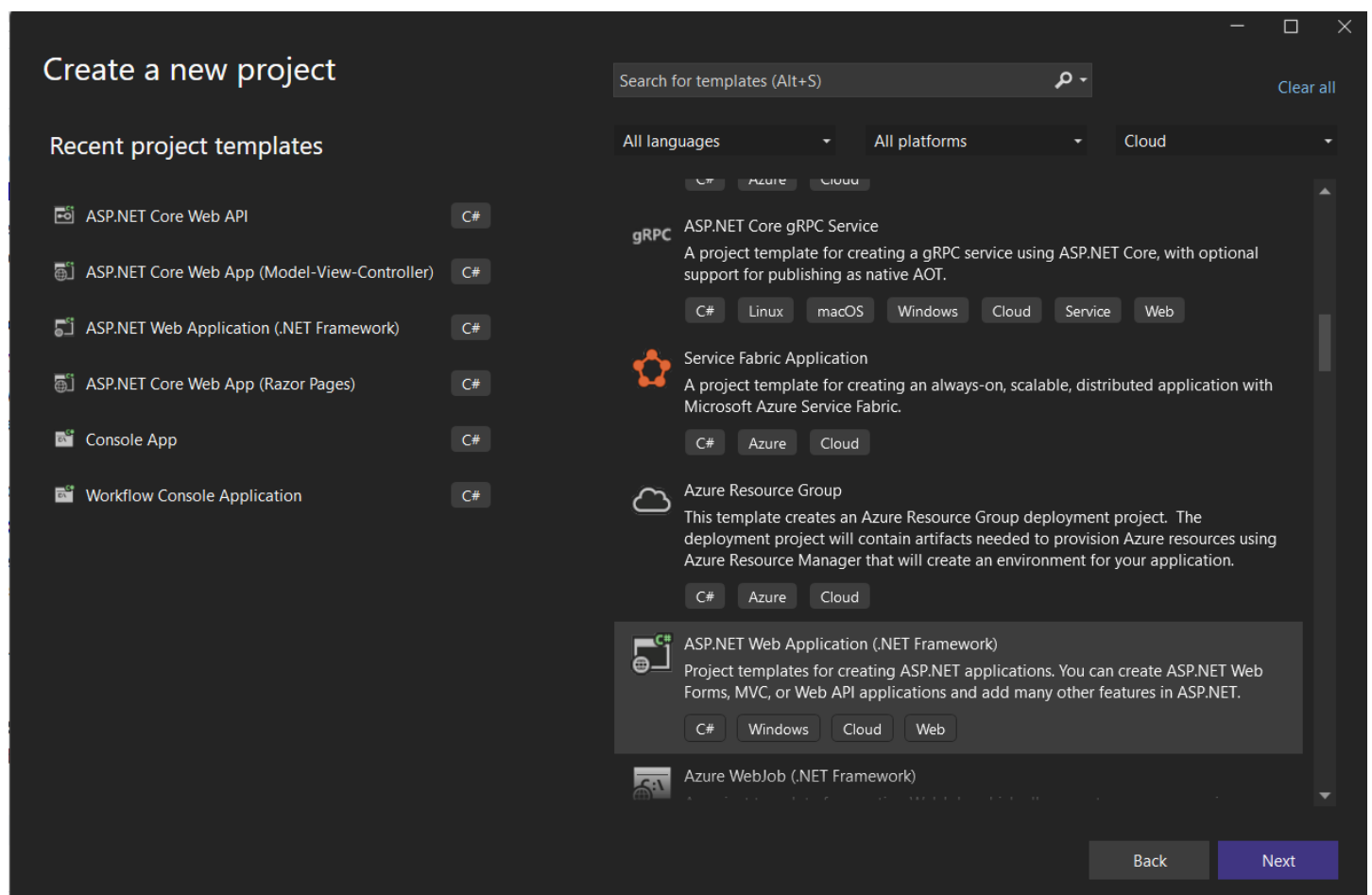
Monitoring the software's performance using Key Performance Indicators (KPIs) like page loading time is one way to ensure the new product works effectively. You can customize your KPIs to determine how efficient the software is after deployment compared to how it was before.

5. Raise awareness of new changes

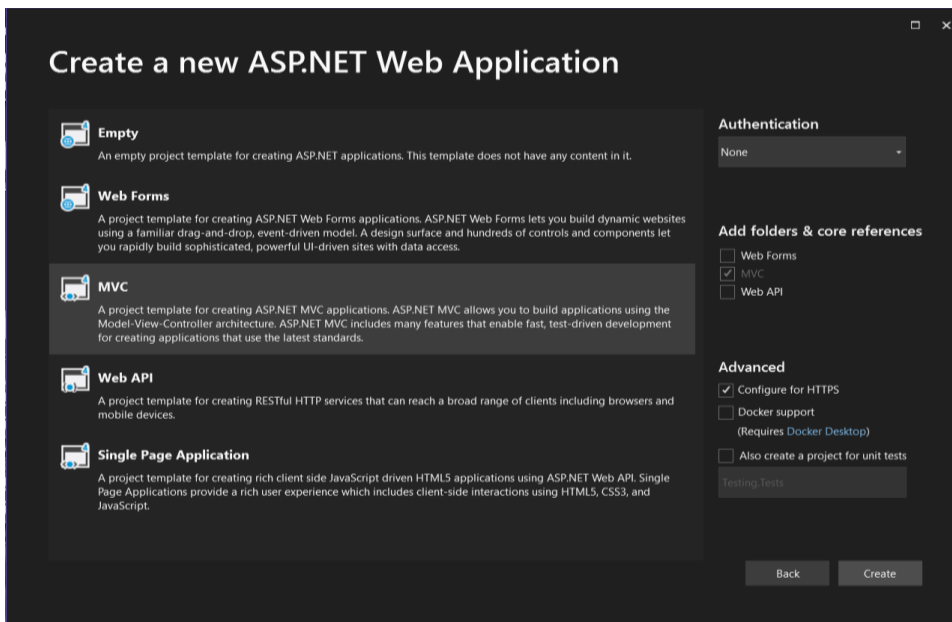
Conducting a successful deployment plan requires communication between teams and the rest of the organization. Before launching a change, let other team members know to alert them of expected alterations.

Deploy an application in IIS:

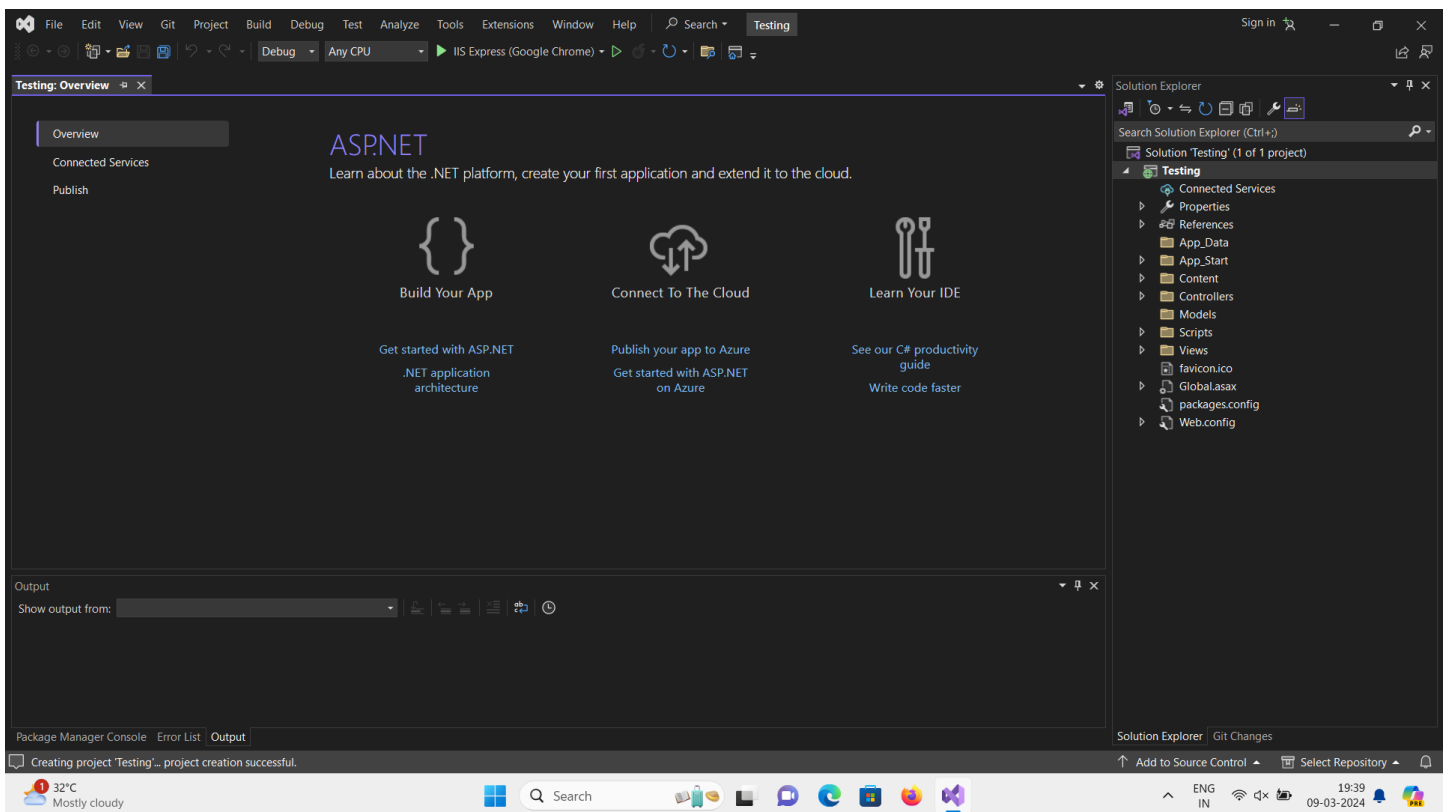
- Start Visual Studio and select **New project**.
- In the **Create a new project** dialog, select **ASP.NET Web Application (.NET Framework)** > **Next**.



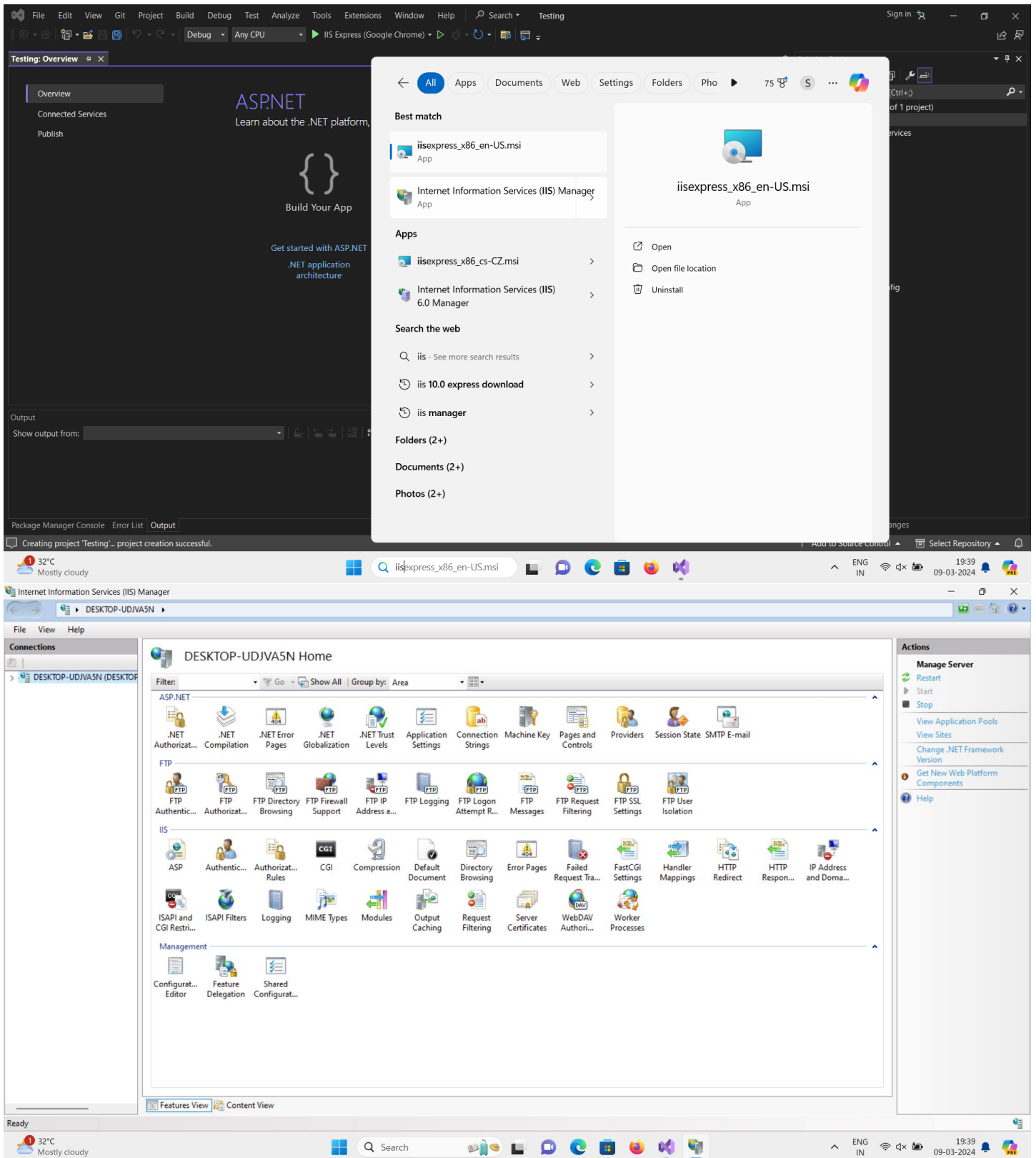
- In the **Configure your new project** dialog, enter **Project name Ex:Testing**.
- In the create a new ASP.NET Web application dialogue box select **MVC** and click on **create**.



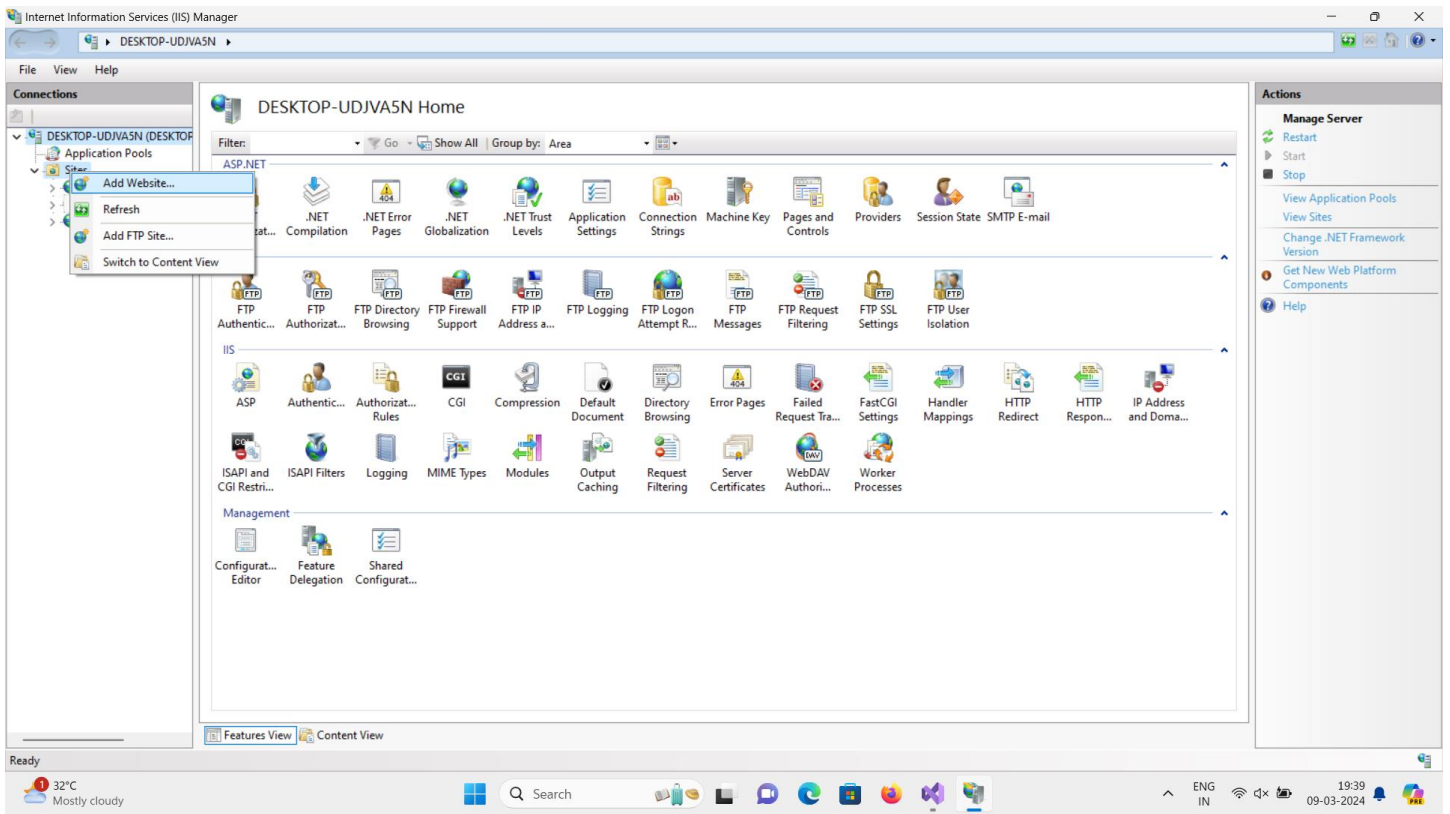
- You can see below project in your device screen.



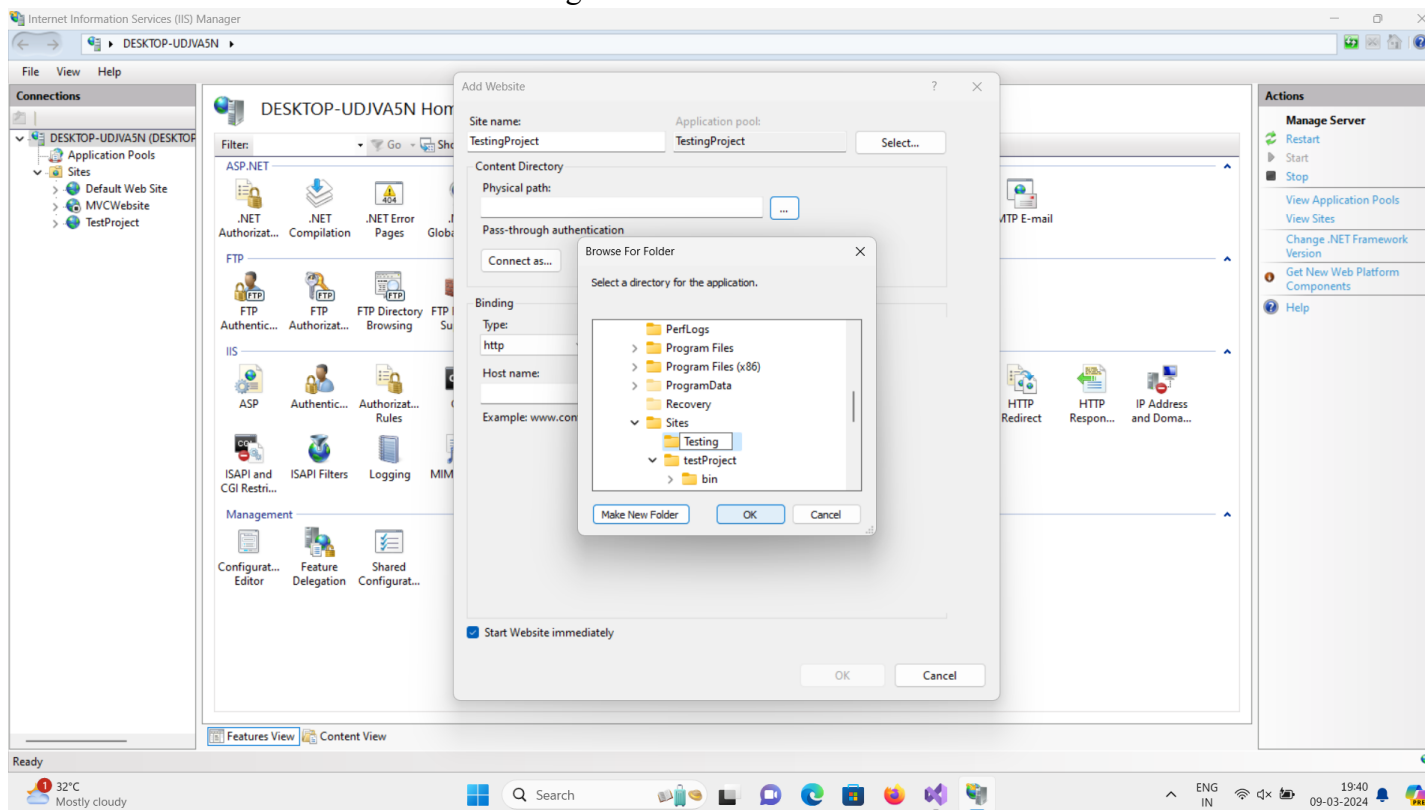
Search IIS in the search bar in your device and select on **Internet Information Services(IIS) Manager**.



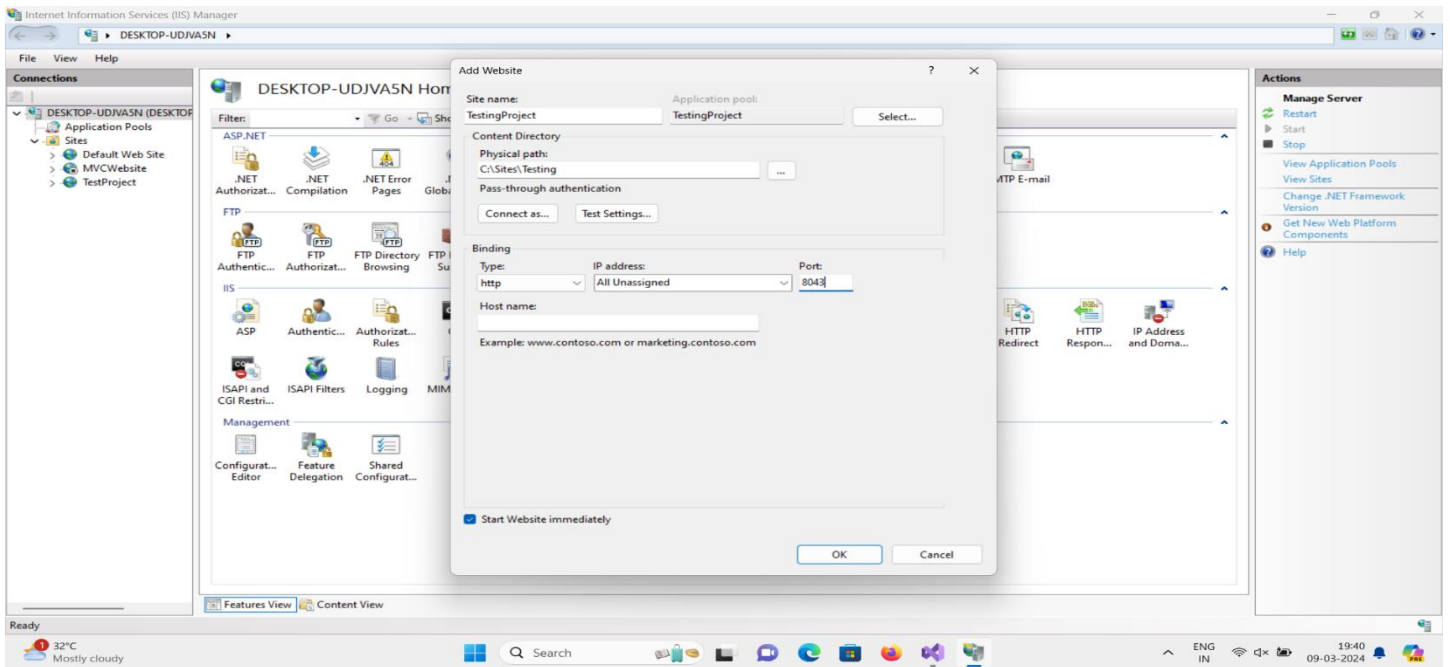
- Right click on the sites(you have to create sites folder in C drive before opening the IIS)
- And select Add Website



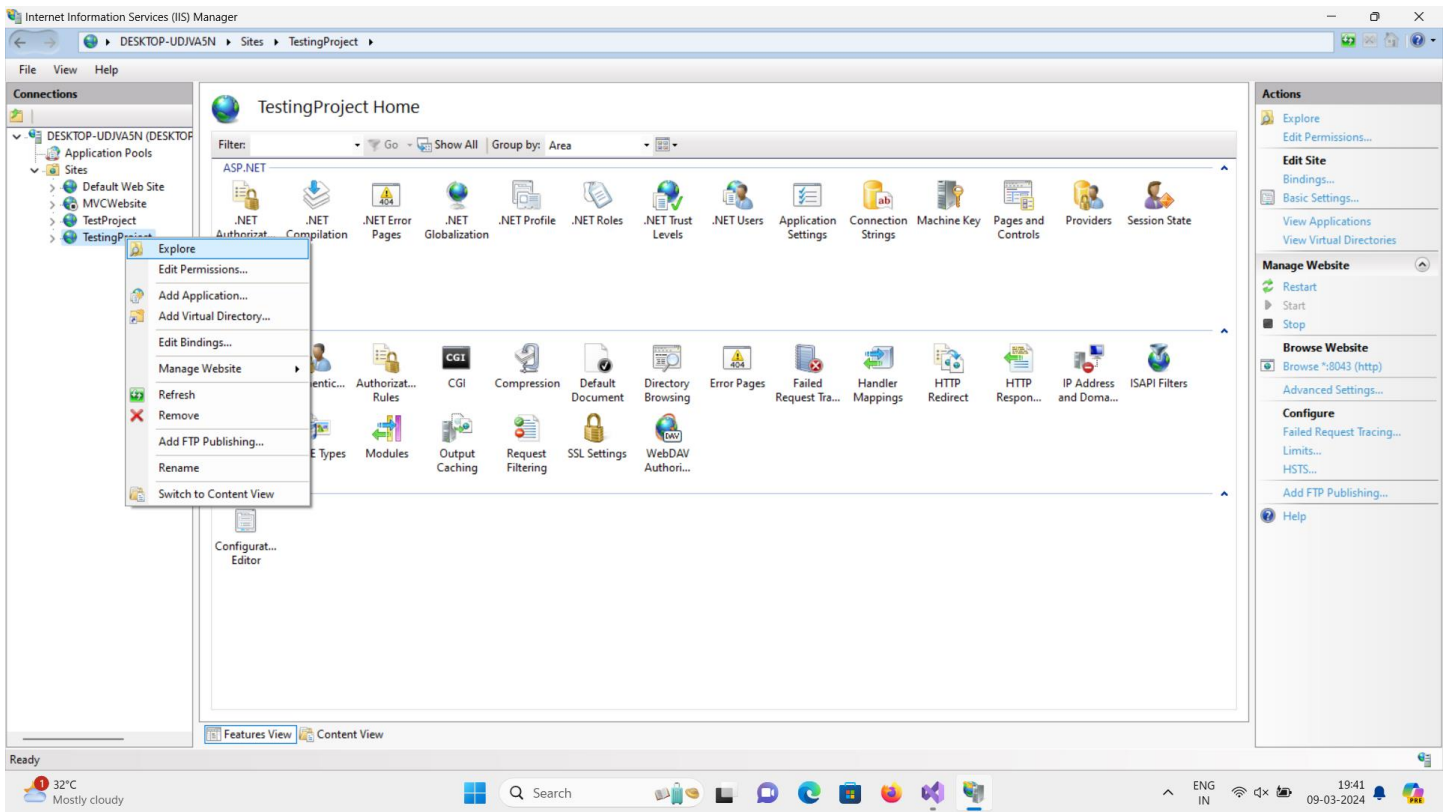
- Enter site name(Ex: TestingProject) < click on physical path [...] <click on sites< click on Make new folder<Give name for new folder Ex:Testing



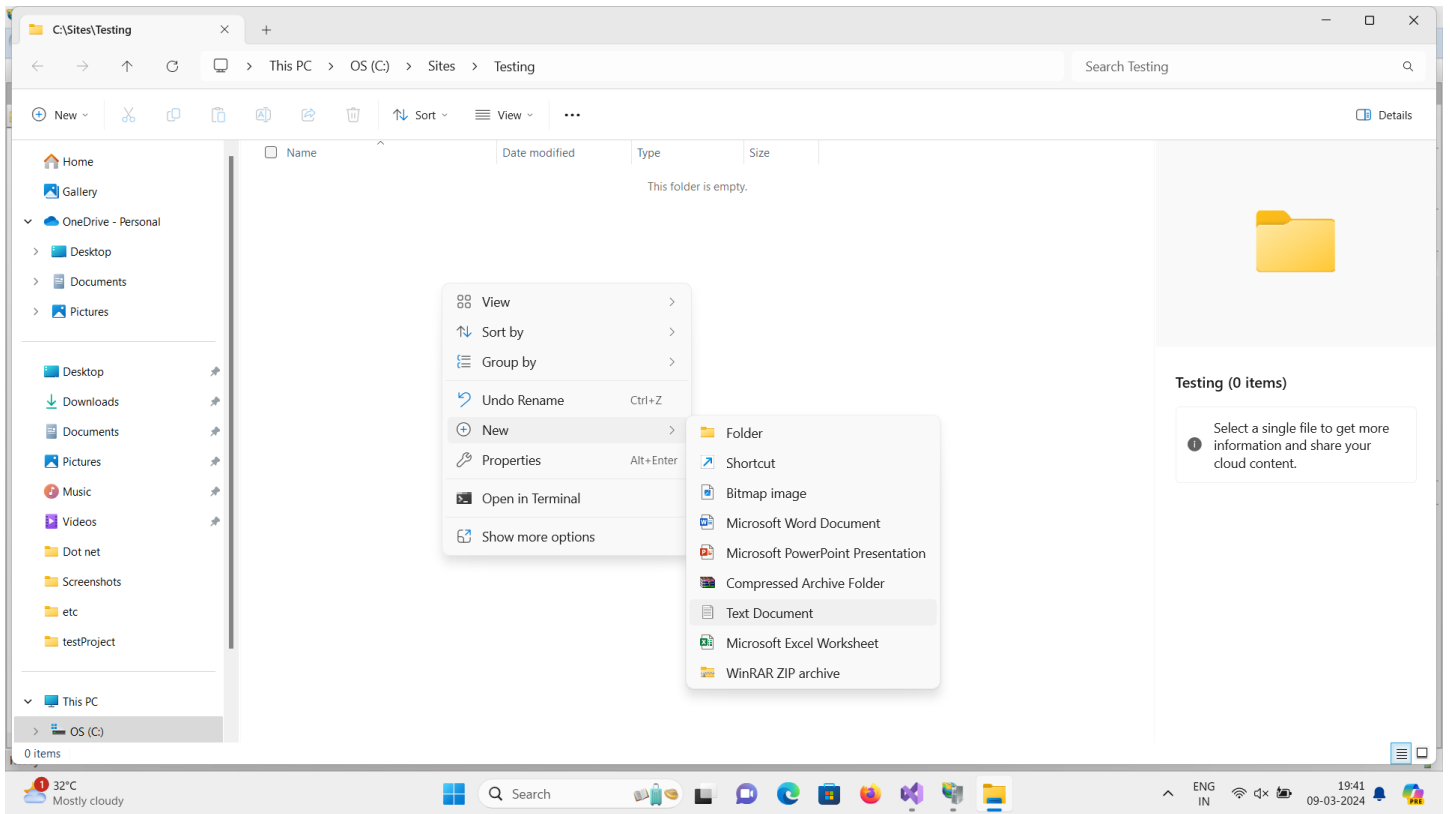
- Click on OK.



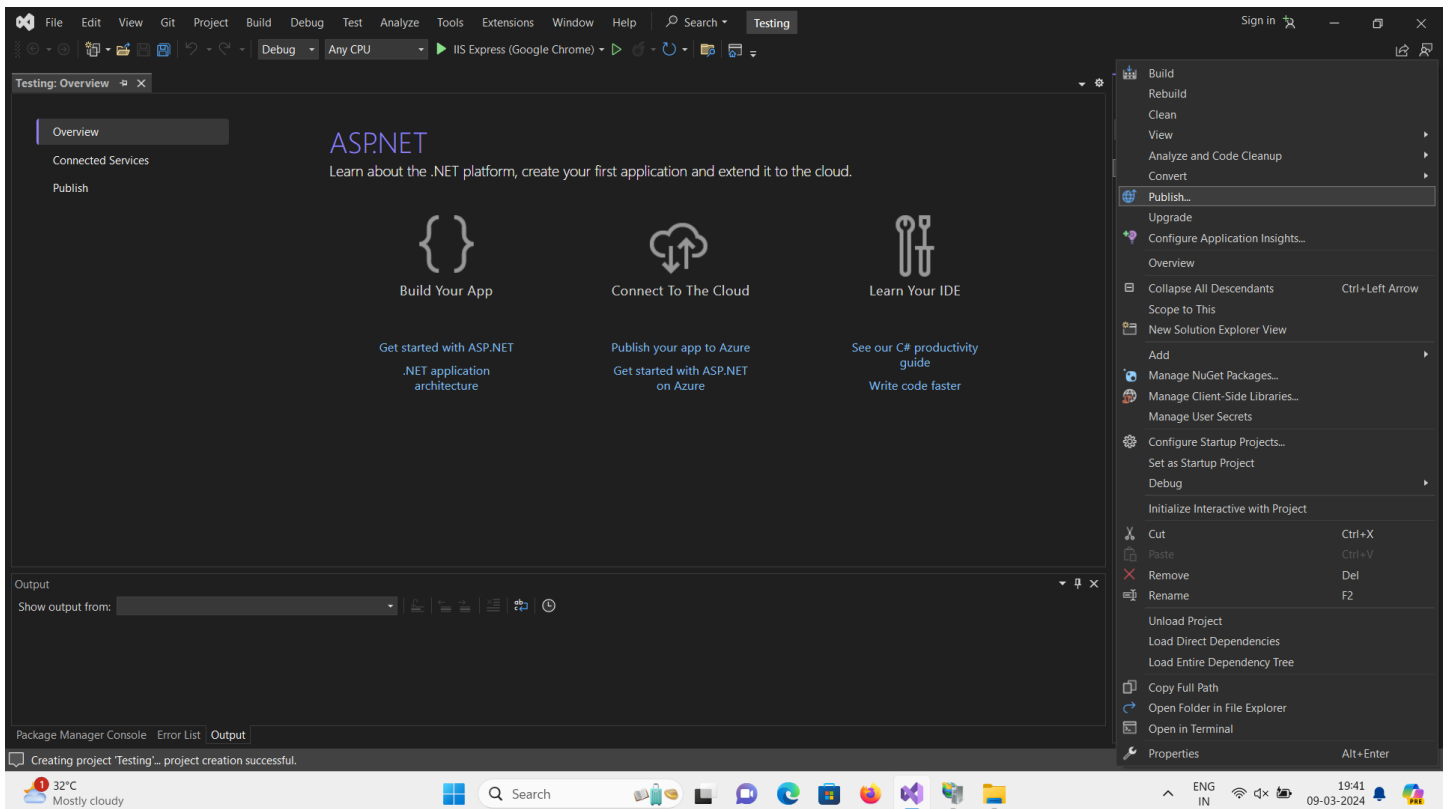
- Right Click on TestingProject and click on Explore.



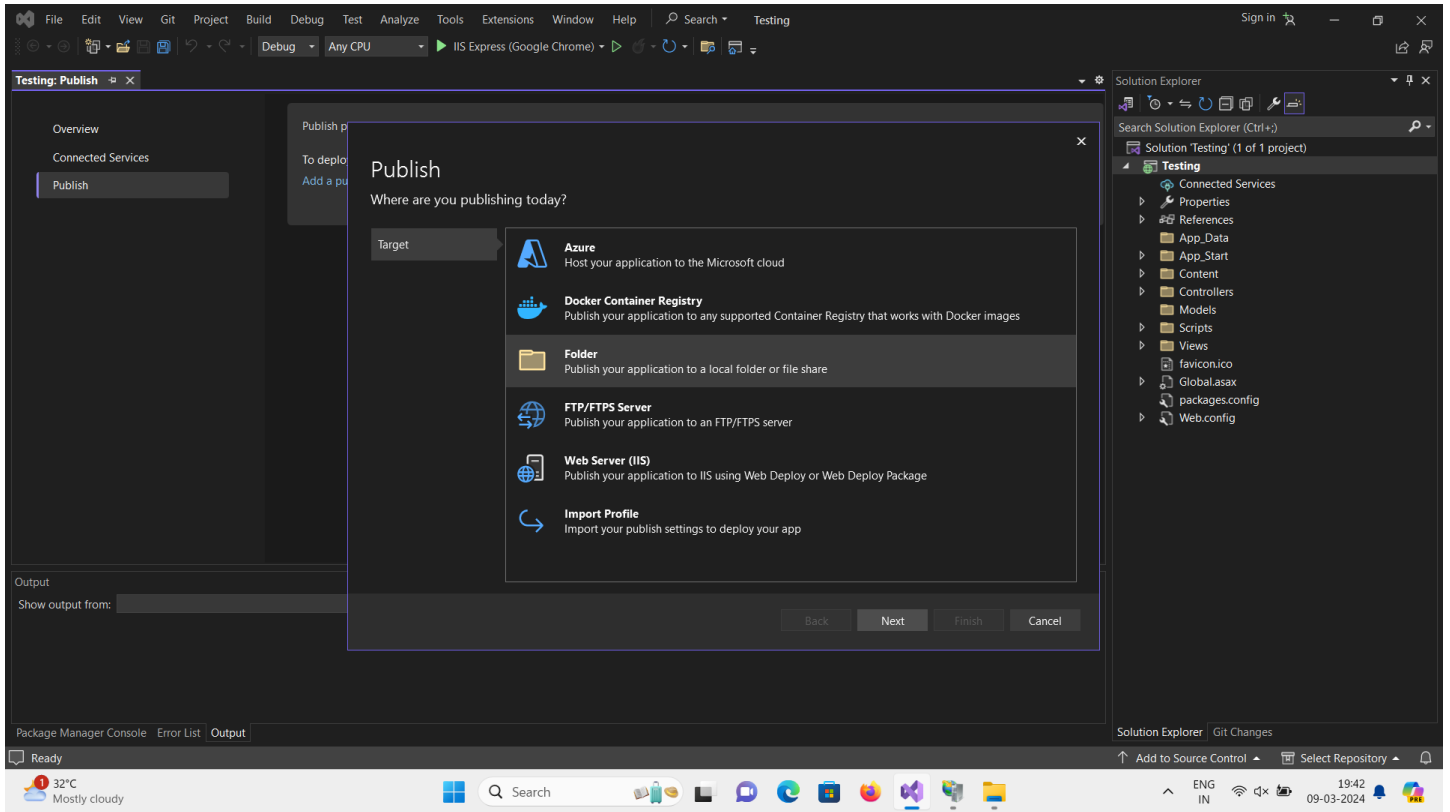
- Create a new text document Ex:Index.html



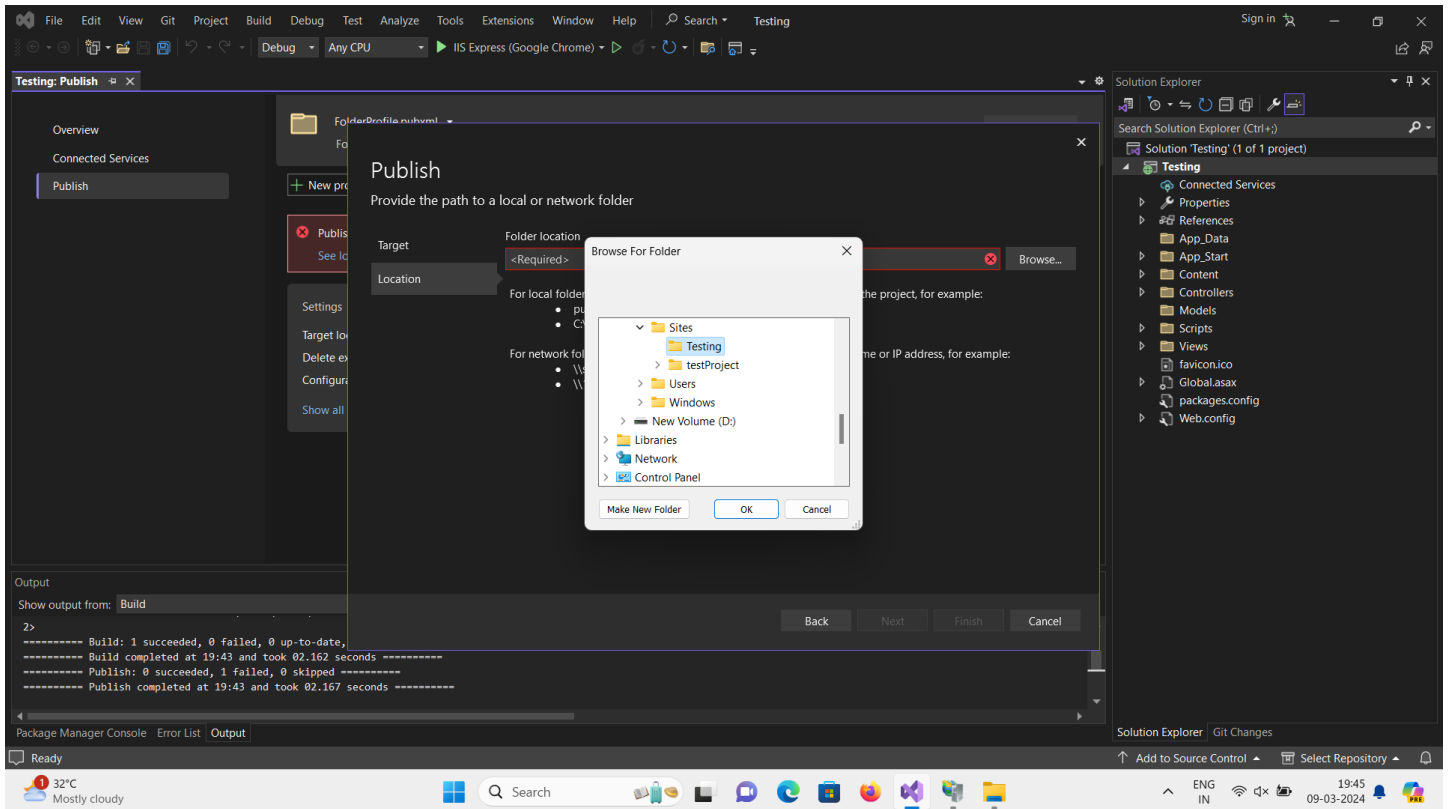
- Open VS code 2022.



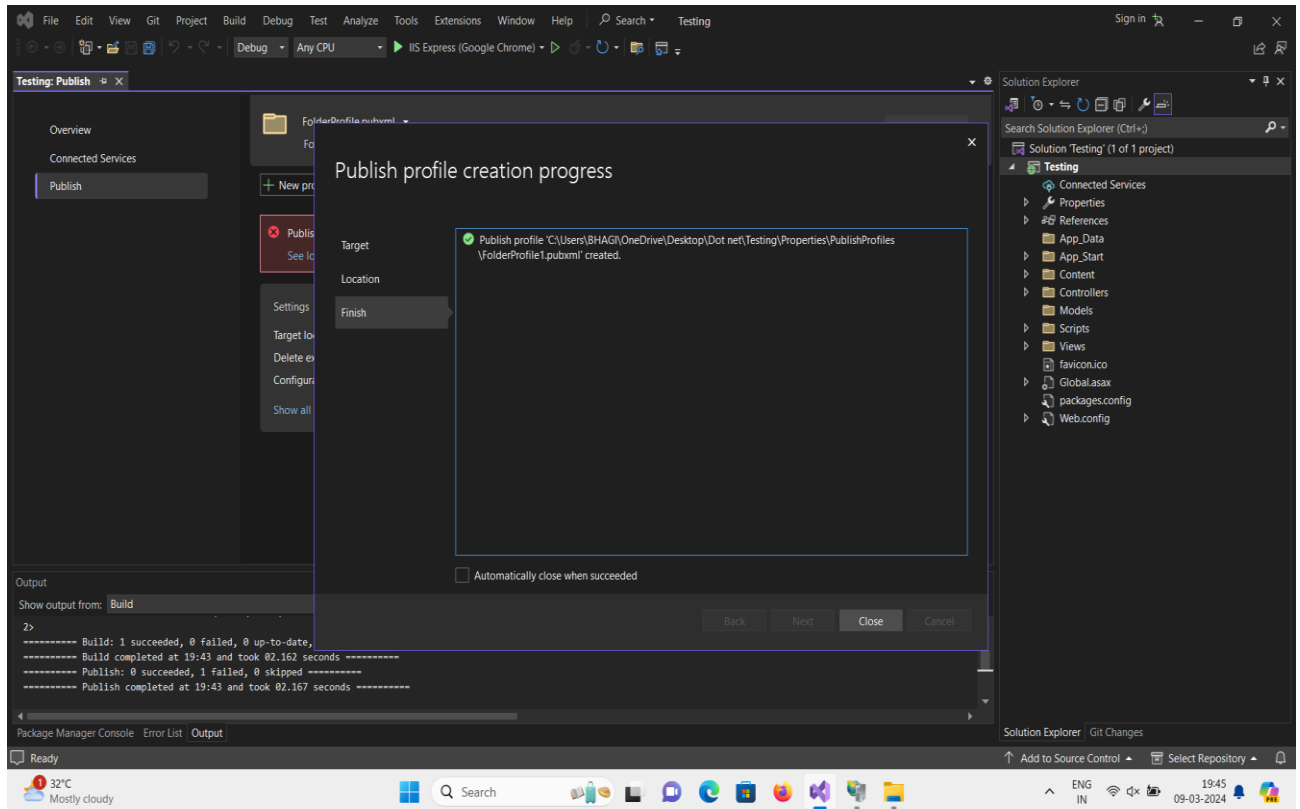
- Right click on Testing and select Folder.



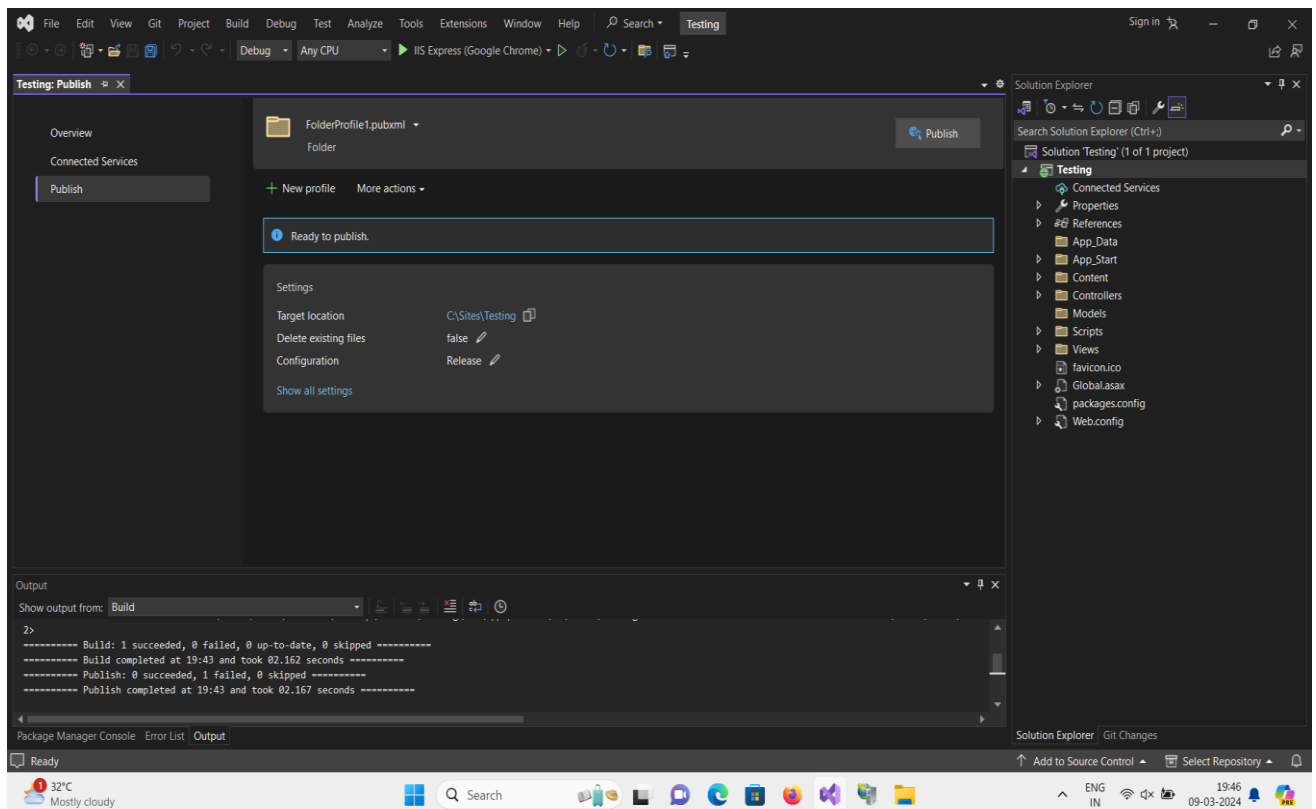
- Give file location.

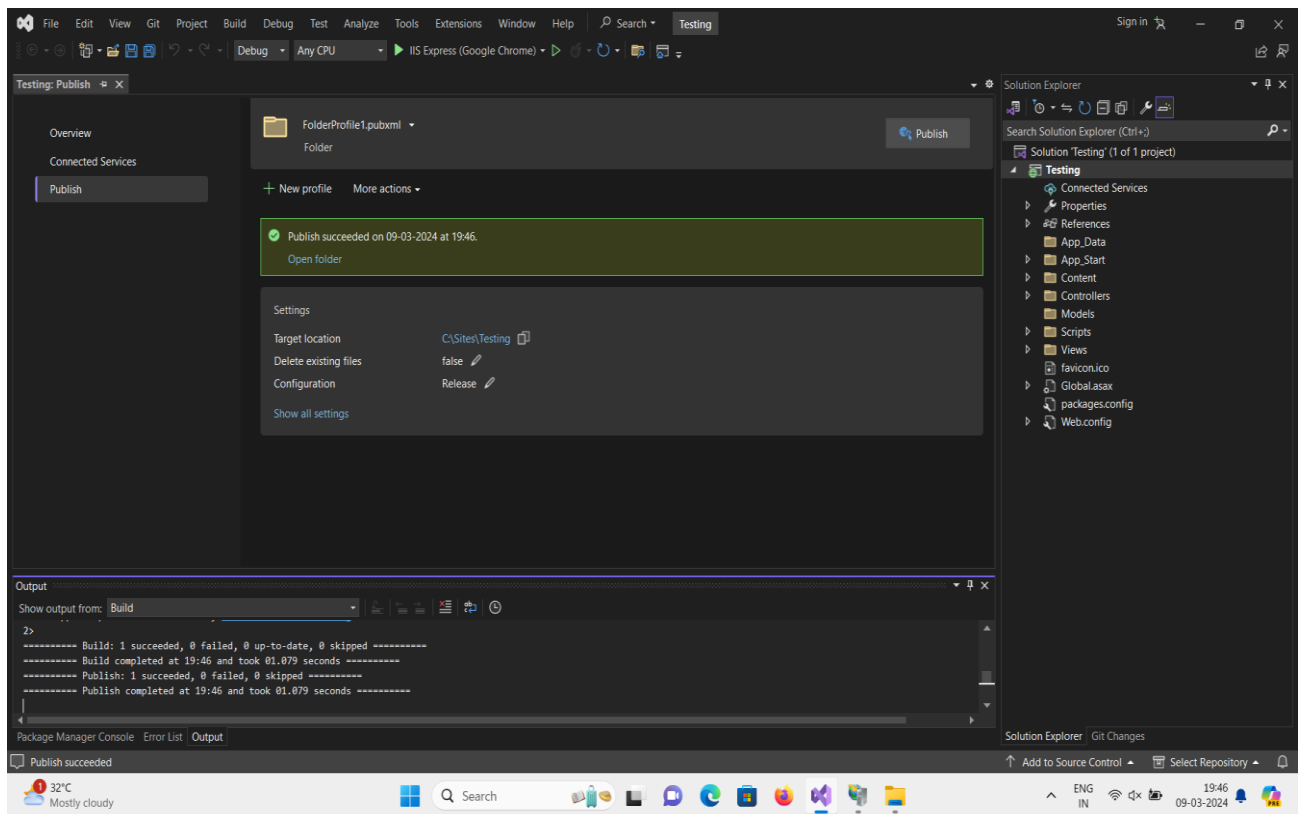


- Click on close

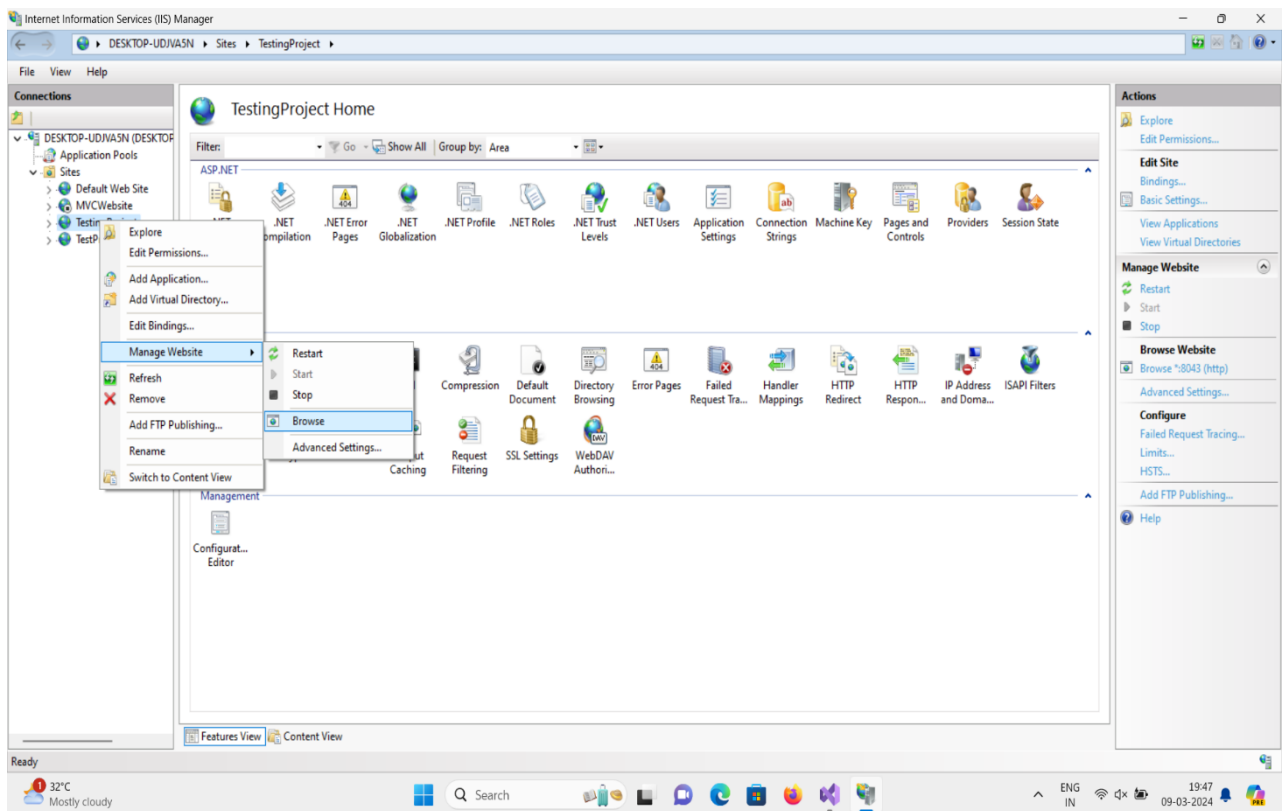


- Click on publish.

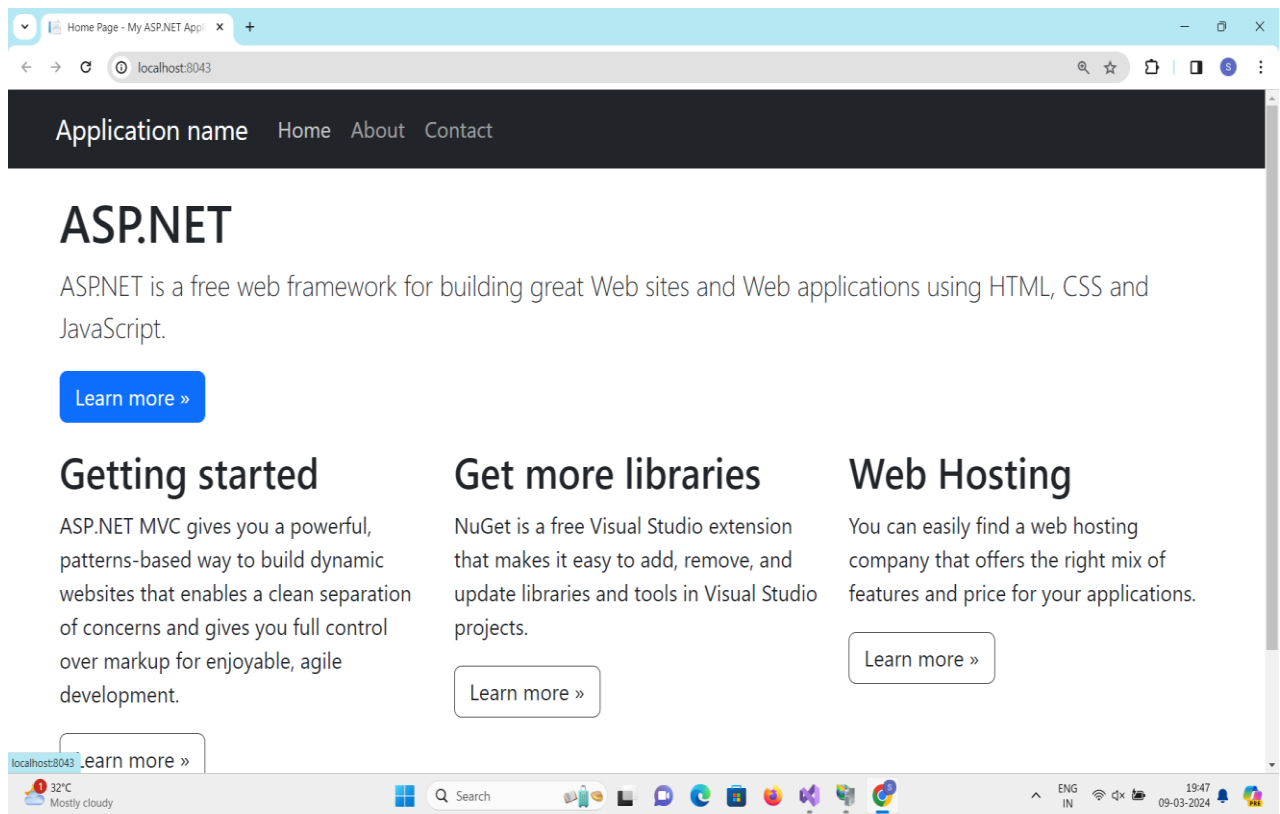




- Right click on TestingProject and click on Manage Website<Brows.



- The web application will run your IIS site in your device.



Deploy an application in Azure:

1. Create an ASP.NET web app

1. Open Visual Studio and then select **Create a new project**.
2. In **Create a new project**, find, and select **ASP.NET Core Web App**, then select **Next**.
3. In **Configure your new project**, name the application *MyFirstAzureWebApp*, and then select **Next**.

Configure your new project

ASP.NET Core Web App C# Linux macOS Windows Cloud Service Web

Project name

MyFirstAzureWebApp

Location

C:\working-directory\

Solution name ⓘ

MyFirstAzureWebApp

☐ Place solution and project in the same directory

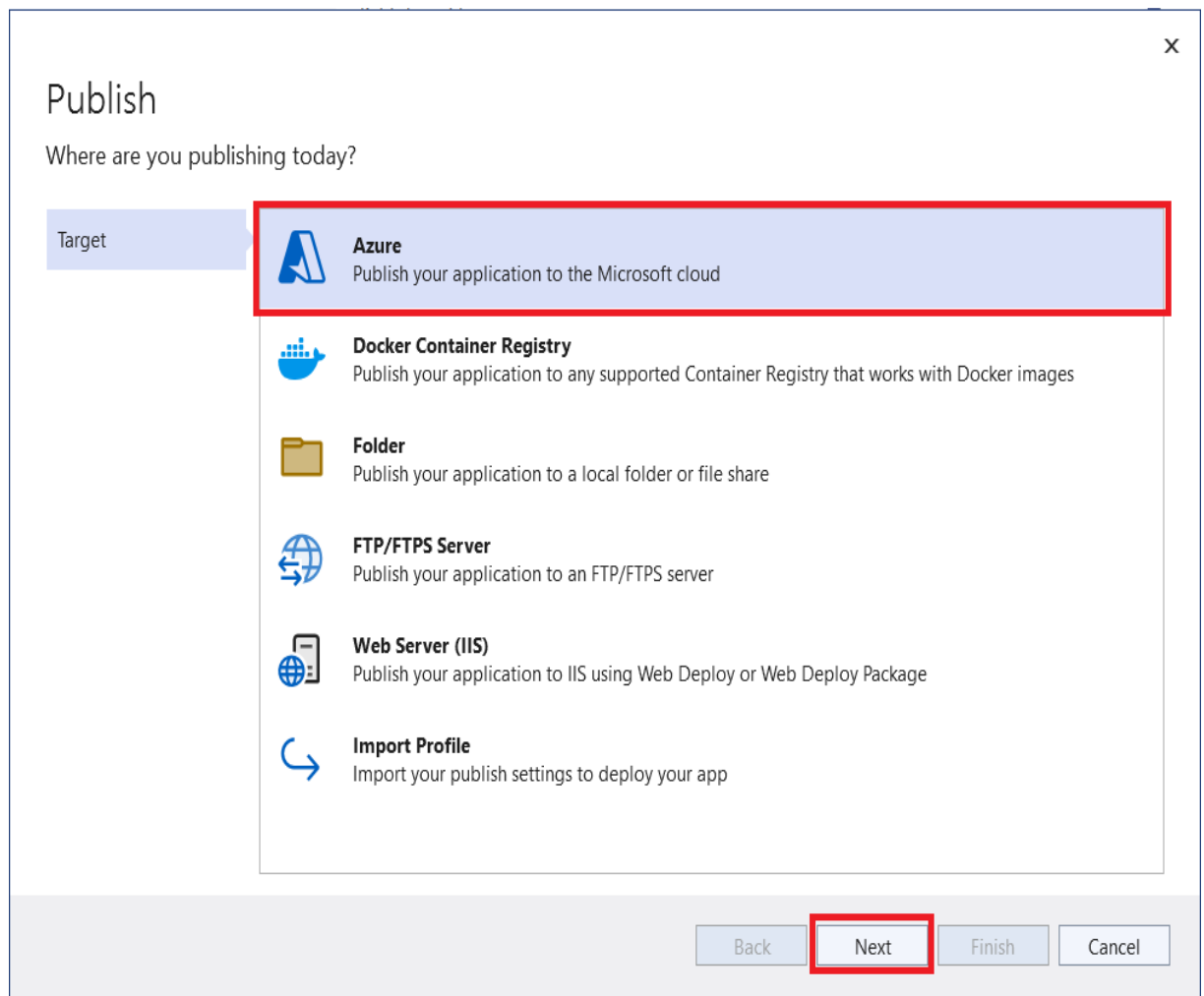
Project will be created in "C:\working-directory\MyFirstAzureWebApp\MyFirstAzureWebApp\"

Back Next

4. Select **.NET 8.0 (Standard-term support)**.
5. Ensure **Authentication Type** is set to **None**. Select **Create**.

2. Publish your web app

1. In **Solution Explorer**, right-click the **MyFirstAzureWebApp** project and select **Publish**.
2. In **Publish**, select **Azure** and then **Next**.



3. Choose the **Specific target**, either **Azure App Service (Linux)** or **Azure App Service (Windows)**. Then, select **Next**.

4. Your options depend on whether you're signed in to Azure already and whether you have a Visual Studio account linked to an Azure account. Select either **Add an account** or **Sign in** to sign in to your Azure subscription. If you're already signed in, select the account you want.

Publish

Select existing or create a new Azure App Service

Target

You need to be signed in with an Azure account

Create your free Azure Account

Specific target

Already have an account?

App Service

[Sign In](#)

[Back](#)

Next

Finish

Cancel

5.To the right of **App Service instances**, select **+**.

Microsoft account

Publish

Select existing or create a new Azure App Service

Target

Subscription name

Visual Studio Enterprise Subscription

Specific target

App Service

Search

+ Create new

There are no existing instances available

[Create a new instance](#)

☐ Deploy as ZIP package

Back

Next

Finish

Cancel

1. For **Subscription**, accept the subscription that is listed or select a new one from the drop-down list.
2. For **Resource group**, select **New**. In **New resource group name**, enter *myResourceGroup* and select **OK**.
3. For **Hosting Plan**, select **New**.
4. In the **Hosting Plan: Create new** dialog, enter the values.
5. In **Name**, enter a unique app name that includes only the valid characters are a-z, A-Z, 0-9, and -. You can accept the automatically generated unique name. The URL of the web app is `http://<app-name>.azurewebsites.net`, where <app-name> is your app name.
6. Select **Create** to create the Azure resources.

 App Service (Windows)
Create new

 Microsoft account

Name

MyFirstAzureWebApp20230327113754

Subscription name

Visual Studio Enterprise Subscription

Resource group

myResourceGroup*

New...

Hosting Plan

MyFirstAzureWebAppPlan* (West Europe, F1)

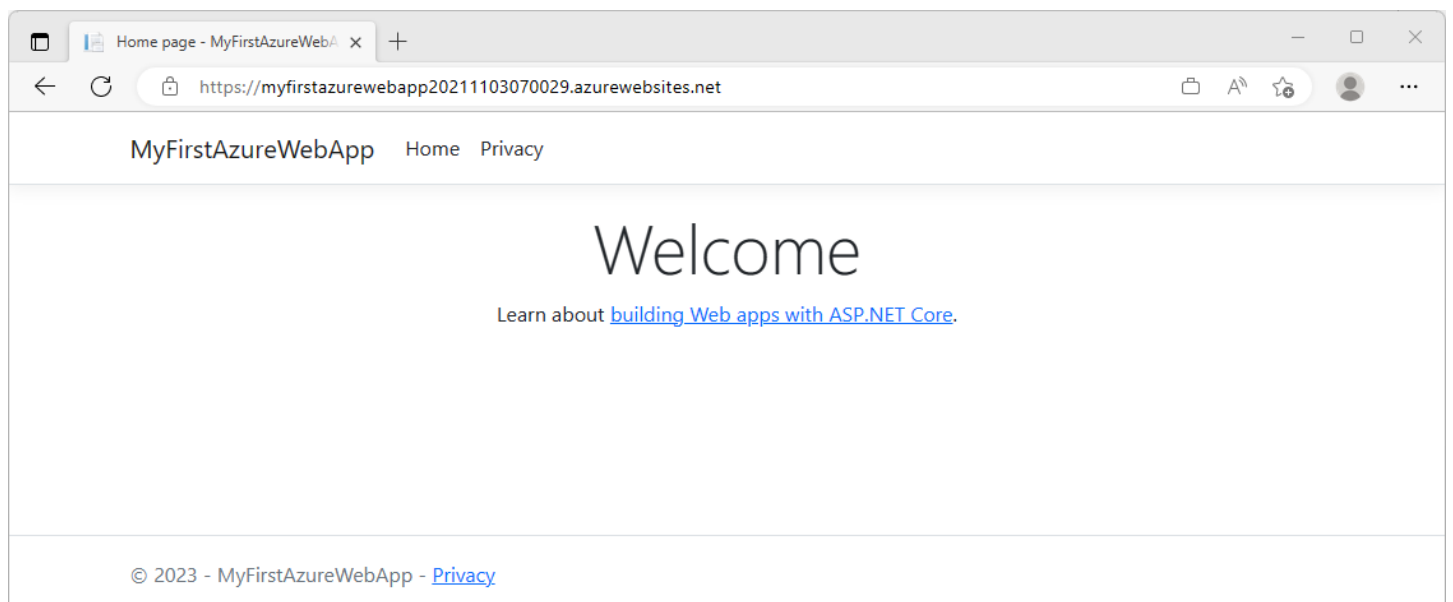
New...

Export...

Create

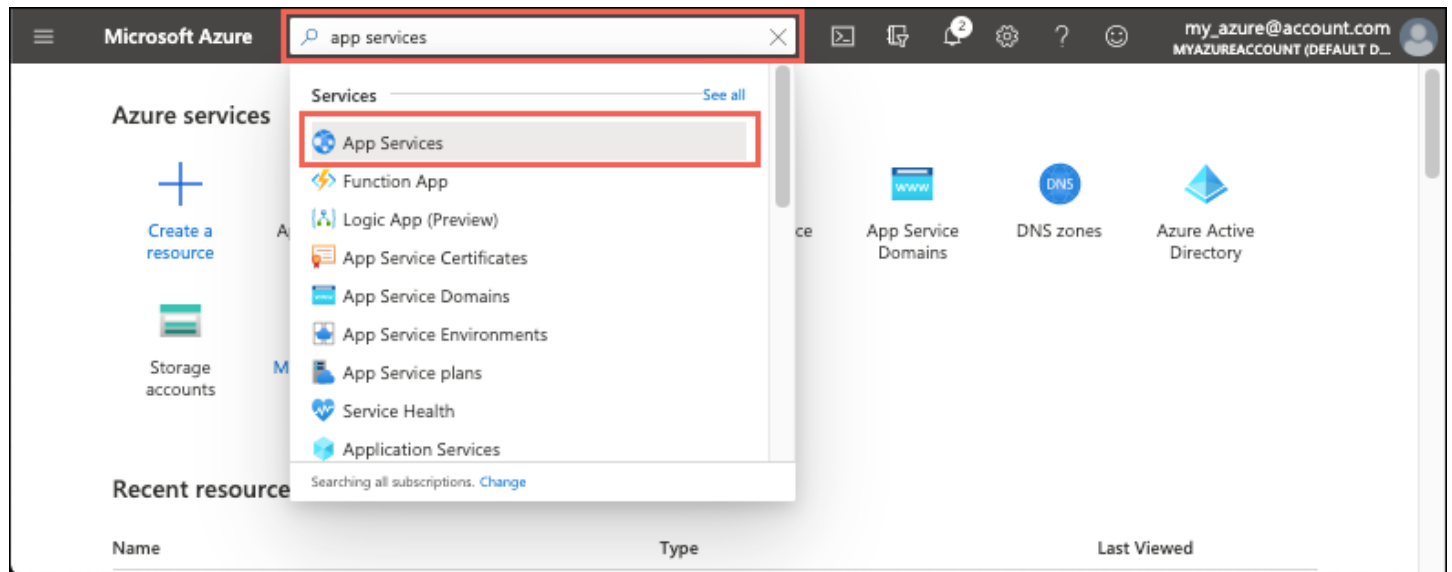
Cancel

1. Once the wizard completes, the Azure resources are created for you, and you're ready to publish your ASP.NET Core project.
2. In the **Publish** dialog, ensure your new App Service app is selected, then select **Finish**, then select **Close**. Visual Studio creates a publish profile for you for the selected App Service app.
3. In the **Publish** page, select **Publish**. If you see a warning message, select **Continue**.

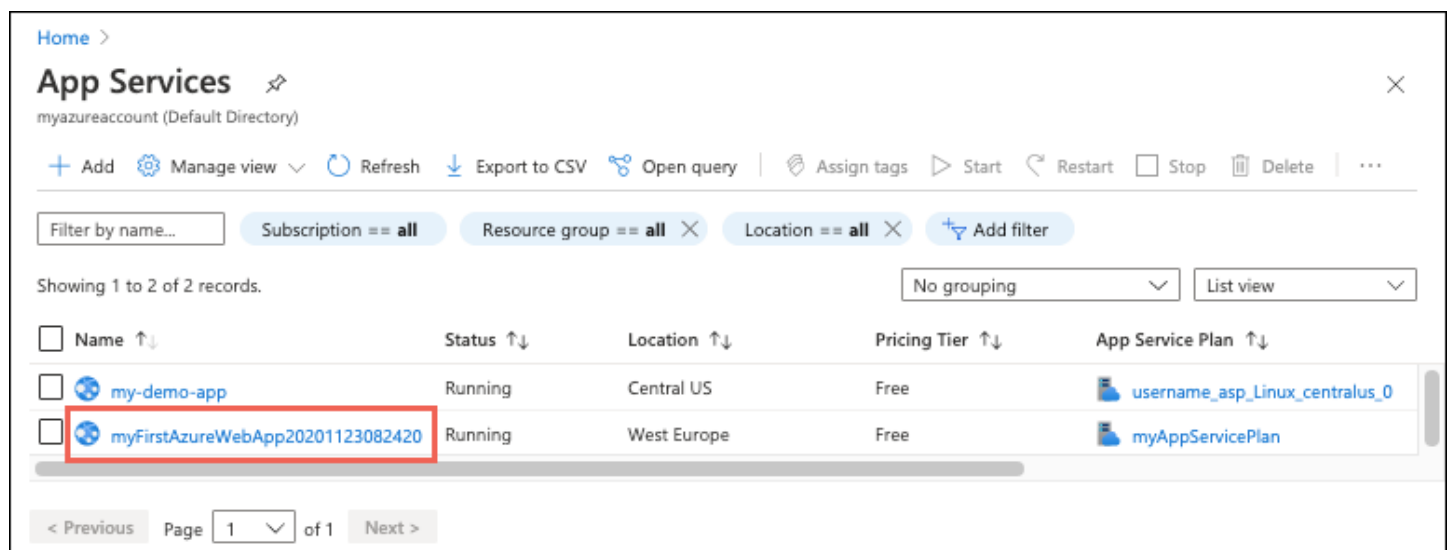


3. Manage the Azure app:

To manage the Azure app go to **Azure portal** and search for and select **App services**.



On the App services page, select the name of your web app.



The overview page for your web app, contains options for basic management like browse, stop, start, restart, and delete. The left menu provides further pages for configuring your app.



myFirstAzureWebApp20201123082420

App Service



Browse



Stop



Swap



Restart



Delete



Refresh



Get publish profile

[Click here to access our Quickstart guide for deploying code to your app →](#)

^ Essentials

Resource group [\(change\)](#)
myResourceGroupStatus
RunningLocation
West EuropeSubscription [\(change\)](#)
[Visual Studio Ultimate with MSDN](#)Subscription ID
00000000-0000-0000-0000-000000000000Tags [\(change\)](#)
[Click here to add tags](#)

URL

<https://myfirstazurewebapp20201123082420.azurewe...>

App Service Plan

[myAppServicePlan \(F1: Free\)](#)

FTP/deployment username

myFirstAzureWebApp20201123082420\deploy-passw...

FTP hostname

ftp://waws-prod-am2-371.ftp.azurewebsites.windows...

FTPS hostname

https://waws-prod-am2-371.ftp.azurewebsites.window...



Diagnose and solve problems

Our self-service diagnostic and troubleshooting experience helps you identify and resolve issues with your web app.



Application Insights

Application Insights helps you detect and diagnose quality issues in your apps, and helps you understand what your users actually do with it.